The Relational Machine Calculus

Anonymous

ABSTRACT

This paper presents the Relational Machine Calculus (RMC): a simple, foundational model of first-order relational programming. The core of the calculus is given by a decomposition of the first-order lambda-calculus which reveals a latent duality, concretely expressed by an involution on syntax. Semantically, this core fragment is sound and complete for string diagrams of Frobenius monoids. A corresponding symmetrization of beta-reduction gives rise to unification as a reduction mechanism. By further including standard operations of Kleene algebra the RMC embeds a range of computational models: the first-order lambda-calculus, logic programming, and automata, among others. These embeddings preserve operational semantics, which for the RMC is given by a generalization of a standard stack machine for the lambda-calculus. The equational theory of the RMC (which supports reasoning about its operational semantics) is conservative over both the first-order lambda-calculus and Kleene algebra, and can be oriented to give a confluent reduction relation.

CCS CONCEPTS

• **Theory of computation → Abstract machines**; **Lambda calculus**; **Regular languages**; **Operational semantics**; **Denotational semantics**; **Categorical semantics**; **Constraint and logic programming**.

KEYWORDS

lambda-calculus, kappa-calculus, Kleene algebra, logic programming, reversible programming, non-determinism, relational calculus, hypergraph category, automata, type systems, Krivine abstract machine, categorical semantics, operational semantics.

ACM Reference Format:

Anonymous. 2024. The Relational Machine Calculus. In *Proceedings of Logic in Computer Science (LICS '24).* ACM, New York, NY, USA, 28 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

The λ-calculus is widely considered the canonical model of functional programming. However, there is no similarly agreed foundation of *relational* programming. Paradigmatic examples of relational programming are logic programming, based on unification [1, 31, 38, 52, 63], and database query languages, based on Tarski's relational algebra [21, 66]. Relations also serve as the (intended) semantics for a wide range of languages. For example, non-deterministic finite automata, Kleene algebra and its variants — some of which subsume propositional Hoare logic [24, 45] — and monadic languages for non-determinism [54]. Furthermore, the category of relations is a simple folk model of (differential) linear logic and thus of the linear λ-calculus; via the Kleisli construction, it is also the basis of a simple quantitative model of the plain λ-calculus [27, 36, 47]. This category can even be seen as underlying game semantics and intersection types [16, 25, 34, 56].

In this paper we present a foundational model of first-order, sequential, relational programming: the *Relational Machine Calculus (RMC)*. We proceed to set out the aims and constraints that informed its design. Then, in the remainder of the introduction, we detail the origin of the RMC: from a first-order λ-calculus through several adaptations made to incorporate the relational paradigm, including *duality*, *unification* and *non-determinism*. The body of the paper is dedicated to justifying our claim of meeting the design criteria we now set out.

1.1 Design criteria

The space of all possible (first-order, sequential) relational models of computation is bewildering, and indeed there is a proliferation of languages in this space, indicating its importance and range of applications. Yet, we find none in the literature which satisfy the following — minimal, but stringent — design criteria that we would expect to be met by a "λ-calculus of relational computation".

Denotational semantics: has a relational and categorical semantics, through quotienting by a local equational theory;
Duality: exhibits a syntactic involution which switches input and output, and denotes the relational converse;
Operational semantics: preserves that of standard models of first-order, sequential (standard and relational) computation;
Confluence: orienting the equational theory gives a confluent reduction relation, sound for the operational semantics

We proceed to carefully motivate these criteria, which serve to tame the design space and establish standards for success in our programme.

Denotational semantics. A minimal qualifying criterion for any relational calculus is to have a denotational semantics in the category of sets and relations. We further expect a categorical semantics in an appropriate symmetric monoidal category which abstracts the structural properties of relations; we shall see momentarily our preferred candidate.

Duality. While the category of sets and relations embeds that of sets and functions, it exhibits very different structural properties; mainly, a perfect duality between input and output given by relational converse. We consider this duality a defining feature of relational computation. A paradigmatic example is the following Prolog program to concatenate two lists.

```
concat([],L,L)
concat([E | L1], L2, [E | L3]) :- concat(L1,L2,L3)
```

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
LICS '24, July 8–12, 2024, Tallinn, Estonia
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

Fixing input lists L1 and L2 in the relation concat(L1,L2,L3) returns their concatenation as L3, but by fixing the value L3 instead, the relation may be run in reverse to (non-deterministically) return every way of splitting L3 into L1 and L2.

Duality is a common theme in programming language research [28, 35, 61], and in mathematics and physics more generally. Its aesthetic appeal barely needs justification; practically, it offers parsimony of expression by identifying a dual program (theorem) with each program (theorem) written. We require of our language that it features duality in an obvious and natural way, via a *syntactic* involution.

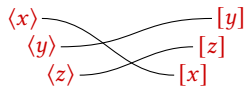
Operational semantics. We expect our language to be expressive enough to encode a range of (standard and relational) models of first-order, sequential computation, including the κ-calculus, reversible programming, logic programming, simple string-diagrammatic calculi, and automata. However, any sufficiently expressive language can encode any other: we would like the embeddings into our calculus to be, in some way, *natural*. As our criterion for this, we specify that operational semantics is preserved, for models where it is defined. This means that (at least, with respect to *operational* semantics — a relatively fine-grained notion) the encoded models may be viewed as *fragments* of our language, built from its more fundamental primitives.

Confluence. We expect an equational theory for our calculus, in order to enable algebraic reasoning about its operational semantics. Following the notion of β-reduction, central to the λ-calculus, we expect the equational theory (modulo certain congruences) to be orientable so as to give a reduction relation, and that this relation is *confluent*: it maintains the possibility for reduction paths from a common source to converge.

1.2 First-order lambda-calculus and duality

Our point of departure is the κ-calculus [39, 58] — that is, the first-order component of the λ-calculus as defined by Hasegawa [3, 4, 39, 40] — which was recently given a new formulation as (a fragment of) the Functional Machine Calculus [4, 40] (FMC). These calculi feature three of our desired properties: a *denotational semantics* (albeit in Cartesian categories), an *operational semantics* in a simple stack machine, and *confluent reduction*. Crucially, they also exhibit the potential for *duality*, and in the formulation of the FMC, this is directly observable in the syntax.

A first-order stack calculus. We begin by introducing the core fragment of the Relational Machine Calculus, as an instruction language for a first-order stack machine which embeds the κ-calculus. Terms are sequences of *push* $[x]$ and *pop* $\langle x \rangle$ operations, separated by sequential composition (;). The following example shows a term and its associated string diagram; the type is given by the size of its input and output stacks.



$$\langle x \rangle ; \langle y \rangle ; \langle z \rangle ; [x] ; [z] ; [y] \ : 3 \Rightarrow 3$$

Operationally, a term evaluates from left to right; $\langle x \rangle$ pops the head off the stack, say a constant c, and substitutes c for x in the remaining computation; $[x]$ pushes the value substituted for x onto the stack. We write our stacks with the head to the right, to match the order of *pushes*; our term then takes a stack $e\,d\,c$ to $c\,e\,d$, where $\langle x \rangle$ pops c, $\langle y \rangle$ pops d, and $\langle z \rangle$ pops e.

In this way, our term implements a relation between input and output stacks, as illustrated by the string diagram, where the wires represent the manipulation of the items on the stack (shown with the head at the top). The simple typing discipline of giving input and output arity tames the asymmetric nature of stacks, resulting in an internal language for symmetric monoidal categories (SMCs) in the linear case (where each *pop* $\langle x \rangle$ is matched by a unique *push* $[x]$ to its right, and vice versa). The κ-calculus, an internal language for Cartesian categories, embeds as follows.

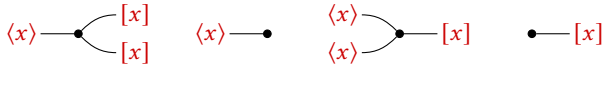
A *pop* $\langle x \rangle ; M$, with M the remaining computation, corresponds to a first-order abstraction $\kappa x.\, M$, written with κ instead of λ following the κ-calculus. A *push* $[x] ; M$ corresponds to an application $M\,x$, restricted to first order by forcing the argument to be a variable. The stack machine is then a simplified Krivine machine [46], replacing environments with substitution.

This correspondence highlights a key aspect. A κ-abstraction is a *binder*, making the variable x *local* to $\kappa x.M$. However, *binding* and *variable scope* are the main notions that will need to be re-visited in light of duality. A pop $\langle x \rangle$ is therefore not a binder, and variables in this fragment of the RMC are *global*. This does not prevent us from encoding the κ-calculus: in the first-order setting, terms are never duplicated, removing any issues with variable capture or α-conversion. We may then simulate binding by stipulating that each *pop* $\langle x \rangle$ has a unique variable, which only occurs in *pushes* to its right (this is *Barendregts convention*). Later, when duplication of terms returns, we will re-introduce local variable scope with a binding *new variable* construct $\exists x.\, M$, justifying the core fragment of the RMC as a decomposition of κ-abstraction into its two distinct roles: *pop* and *new variable*.

Duality. Our formulation of κ-calculus is tailored to reveal its latent duality, effected simply by reversing a term while switching *push* and *pop*. The dual of our previous example term:

$$\langle y \rangle ; \langle z \rangle ; \langle x \rangle ; [z] ; [y] ; [x] \ : 3 \Rightarrow 3$$

The linear fragment, where terms represent *permutations*, is closed under duality. By contrast, the *non-linear* first-order λ-calculus is characterised by *duplication* and *deletion*, embodied by the *diagonal* and *terminal* below left. These yield the dual terms below right, which we label *matching* and *arbitrary*, that feature multiple related *pops*, or zero.



$\langle x \rangle ; [x] ; [x]$ $\quad$ $\langle x \rangle$ $\quad$ $\langle x \rangle ; \langle x \rangle ; [x]$ $\quad$ $[x]$

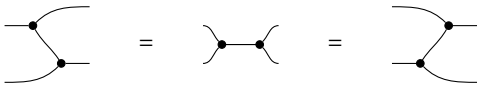
Semantics tells us that the new terms should represent the relational converse of the diagonal and terminal functions, given respectively by the partial function that sends (x, x) to x and is otherwise undefined, and the non-deterministic function sending the empty input to all possible outputs.

Operationally, the required behaviour of *matching* $\langle x \rangle ; \langle x \rangle ; [x]$ for an input stack of two identical values, $c\,c$, is to return c, and for distinct values, $d\,c$, is to fail. We achieve this by generalising

pop to hold also *constants*, $\langle c \rangle$, interpreted as an *assertion* that the head of the stack is c, with failure for any other constant. Then $\langle x \rangle ; \langle x \rangle ; [x]$ evaluates by first popping c, substituting it for x in both *pop* and *push* to get $\langle c \rangle ; [c]$, which then pops and replaces the second value c (or fails for d).

The second term, *arbitrary* $[x]$, illustrates that stack values must include variables, and hence must be substituted for: the term $\langle c \rangle$ for a stack $x\,x$ incurs a substitution of c for the second x. The behaviour of *push* and *pop* thus becomes perfectly symmetric, with substitutions in both the stack and the remaining term.

Altogether these relations are abstracted catgeorically as *Frobenius monoids*, which we take to capture non-linear computation in the presence of duality, given in string diagrams as above and characterised by the *Frobenius equation* below:



Intuitively, this implements the idea that *connectivity = identity*: connected wires represent a single value. In our calculus, this is modeled by *variable names*; and indeed the three diagrams in the above equation may all be represented by the term $\langle x \rangle ; \langle x \rangle ; [x] ; [x]$.

Frobenius monoids have received a huge amount of interest over the last decade or so, since they were identified as a fundamental primitive of quantum computation by the ZX-calculus [22, 23, 42]. Since then, they have been adopted as primitives in a wide range of string diagrammatic languages, including those capturing conjunctive queries, relational algebra, and aspects of logic programming [8, 10, 13, 37]. In programming language theory specifically, they have received significantly less attention, but have been used as a synthetic axiomatization of the operation of *exact conditioning* in probabilistic programming [26, 64] and to model of the (partial) inverse of duplication in reversible languages [43]. The RMC offers a novel *operational* account of Frobenius monoids, thus filling a gap in the literature and bridging the study of string diagrammatic and programming languages.

1.3 The full calculus

The calculus thus far is generated by: *push* $[x]$ and *pop* $\langle x \rangle$ operations, *sequential composition* $M ; N$, and its unit $\star$, the imperative *skip*. Associativity of composition will be implemented via reduction and machine behaviour. To reach our desired expressivity, we extend this with a careful selection of features, outlined below.

Algebraic terms and unification. The β-reduction relation is the interaction of a consecutive *push* and *pop*, say $[y] ; \langle x \rangle$, to incur a (global) substitution $\{y/x\}$. With constants, both $[c].\langle x \rangle$ and $[x].\langle c \rangle$ incur $\{c/x\}$, while $[c].\langle c \rangle$ succeeds and $[d].\langle c \rangle$ fails.

Viewing a redex as a formal equation, these are the familiar rules of *first-order unification*, restricted to variables and constants. We generalize our calculus to allow *algebraic terms* [1] as values, and will implement unification on the machine and through a symmetric version of β-reduction. Evaluating a redex $[t].\langle s \rangle$ will produce the substitutions constituting a most general unifier of t and s, or fail if none exists.

[1] We write "algebraic terms" to avoid name clashing with "first-order terms", which here refer to terms of the κ-calculus, *etc.*

Kleene Algebra. To internalize the partial and non-deterministic semantics of relations, we introduce non-deterministic *sum* $(+)$ and *failure* 0, its unit. Indeed, the presence of non-determinism is often considered a defining feature of relational programming. With sequencing already present, we also desire a Kleene star construction $(-)^*$ to model infinitary behaviour, especially of logic programming. Our full calculus will thus be conservative over Kleene algebra (KA).

The atoms of KA may be interpreted as programs, with the axioms of KA acting as a set of (weak) fundamental laws any standard non-deterministic, sequential language should satisfy [41, 45]. However, an *operational* interpretation of KA is typically only given via its correspondence with finite state automata. Our calculus can be viewed as a modification of KA to include the dual primitives of *push* and *pop*, replacing the atoms of KA and supporting its extension with a *direct* operational semantics.

Local variables. Evaluation for Kleene star, taking a term M^* non-deterministically to a sequence $M ; \ldots ; M$ of any length, reintroduces duplication of terms into the calculus, bringing with it the familiar problems of variable identity and α-conversion. We adopt a standard solution: making the scope of a variable explicit with a *new variable* construct, $\exists x.\, M$, familiar variously from nominal Kleene algebra [33], functional logic programming [38], π-calculus [53], and the ν-calculus [57] as a fragment of ML. This restores locality and gives a proper decomposition of κ-abstraction into its two roles: first, as the binder of new variables, and second, as an instruction to *pop* from the stack.

$$\kappa x.\, M \;=\; \exists x.\, (\langle x \rangle ; M)$$

Locations. We inherit a further feature from the Functional Machine Calculus: multiple stacks (or streams) on the abstract machine, indexed in a set of *locations*. This captures several computational effects: mutable store, as stacks restricted to depth one; input and output, as dedicated streams; and a probabilistic generator, as a stream of random bits — remarkably, *while retaining confluent reduction* [40]. The required generalization of the syntax is a simple parameterization of *push* and *pop* in a location a, as $[x]a$ and $a\langle x \rangle$ respectively, to operate on the indicated stack. Thereby, the FMC subsumes the operational semantics of the given effects, while $\beta\eta$-equivalence captures their algebraic theory [4, 40]. This extension allows the simple modelling of stateful models of computation, such as Turing machines, but can be considered somewhat orthogonal to the main contributions of the paper.

2 THE RELATIONAL MACHINE CALCULUS

We define the *Relational Machine Calculus* (RMC) as follows. We assume a countable set of *locations* $A = \{a, b, c \ldots\}$, which each represent a stack on the abstract machine. Stacks hold algebraic terms over a signature Σ of function symbols f^n of arity n.

Definition 2.1 (Relational Machine Calculus). *Values* s, t and *(computation) terms* M, N are given by the grammars below.

s, t	::=	$x \mid f^n(t_1, \ldots, t_n) \quad (f^n \in \Sigma)$		algebraic terms
M, N	::=	$\star \mid M ; N \mid M^* \mid 0 \mid M + N$		Kleene algebra
	$\mid$	$[t]a \mid a\langle t \rangle$		stack operations
	$\mid$	$\exists x.\, M$		variable scope

From left to right, the computation terms are *skip* or *nil* $\star$, *(sequential) composition* $M\,;N$, *Kleene star* M^*, *zero* or *failure* 0, a *sum* of terms $M+N$, a *push* $[t]a$ of the value t to the location a, a *pop* $a\langle t\rangle$ from the location a to unify with t, and a *new* variable introduction $\exists x.\,M$ which binds x in M.

The calculus exhibits a syntactic involution we call *duality*.

Definition 2.2 (Duality). Define *duality* $(-)^\dagger$ terms as follows.

$$\begin{array}{rclcrcl} \star^\dagger & = & \star & & (M+N)^\dagger & = & M^\dagger + N^\dagger \\ (N\,;M)^\dagger & = & M^\dagger\,;N^\dagger & & [t]a^\dagger & = & a\langle t\rangle \\ (M^*)^\dagger & = & (M^\dagger)^* & & a\langle t\rangle^\dagger & = & [t]a \\ 0^\dagger & = & 0 & & (\exists x.\,M)^\dagger & = & \exists x.\,M^\dagger \end{array}$$

We shall see throughout the paper how various operational, equational, and semantic notions either dualize or respect duality.

2.1 Operational Semantics

The small-step operational semantics is given by a stack machine, the *relational abstract machine* or *relational machine*.

Definition 2.3 (Relational Machine). A *state* is a triple $(\,S_A\,,\,M\,,\,K\,)$ of: a *memory* S_A, a family of stacks indexed in a set of locations A; a term M; and a *continuation stack* K. These are defined as follows.

$$\begin{array}{rcll} K,L & ::= & \varepsilon \mid M\,K & \text{continuation stacks} \\ S,T & ::= & \varepsilon \mid S\,t & \text{operand stacks} \\ S_A & ::= & \{S_a \mid a \in A\} & \text{memories} \end{array}$$

The addition of a stack S_a to a memory S_A ($a \notin A$) is written $S_A \cdot S_a$. We abbreviate $f(t_1,\ldots,t_n)$ to $f(T)$ where $T = t_1 \ldots t_n$, and $[t_1]a\,;\ldots;[t_n]a$ to $[T]a$ and $a\langle t_n\rangle\,;\ldots;a\langle t_1\rangle$ to $a\langle T\rangle$ (note the inversion). The *transitions* of the machine are given in Figure 1, read top–to–bottom, and are non-deterministic: a state transitions to a formal sum of states, represented by branching of the transitions. A *run* of the machine is a single rooted path in the machine tree, not necessarily to a leaf, shown with a double line as below.

$$\dfrac{(\,S_A\,,\,M\,,\,K\,)}{\;\;(\,T_A\,,\,N\,,\,L\,)\;\;}$$

A run is *successful* if it terminates in a state $(\,T_A\,,\,\star\,,\,\varepsilon\,)$. A state where no transition rules apply represents *failure*, and is considered to have *zero* branches.

Machine evaluation of a term M for an input memory S_A gives a (possibly infinite) number of successful runs, each with a return memory T_A. The big-step evaluation function $(S_A \Downarrow M)$ will collect these as a multiset. Since variables are global and machine steps substitute into the continuation stack, evaluation returns also a finite substitution map σ with each return memory T_A, as a pair (T_A, σ). In a composition $M\,;N$, the subsitutions from M can then be applied to N.

Notation. We denote the empty multiset by $[]$, a singleton by $[(T_A,\sigma)]$, and multiset union by $\sqcup$. Substitution maps σ and τ are applied to a term as σM and composed as $(\sigma\tau)M = \sigma(\tau M)$. The empty map is ε, and $\sigma \setminus y$ is as σ except undefined on y. The n-fold composition of a term M is M^n where $M^0 = \star$ and $M^{n+1} = M\,;M^n$.

$$\dfrac{(\,S_A\,,\;\star\,,\,M\,K\,)}{(\,S_A\,,\,M\,,\;\;K\,)} \qquad \dfrac{(\,S_A\,,\,M\,;N\,,\;\;K\,)}{(\,S_A\,,\;\;M\,,\,N\,K\,)}$$

$$\dfrac{(\,S_A\,,\,M^*\,,\,K\,)}{(\,S_A\,,\,\star\,,\,K\,)\quad(\,S_A\,,\,M\,;M^*\,,\,K\,)}$$

$$\dfrac{(\,S_A\,,\,M+N\,,\,K\,)}{(\,S_A\,,\,M\,,\,K\,)\quad(\,S_A\,,\,N\,,\,K\,)}$$

$$\dfrac{(\,S_A\,,\;\;\exists x.\,M\,,\,K\,)}{(\,S_A\,,\,\{y/x\}M\,,\,K\,)}\;(y \text{ fresh}) \qquad \dfrac{(\,S_A\cdot S_a\;\;,\,[t]a\,,\,K\,)}{(\,S_A\cdot S_a\,t\,,\;\;\star\,,\,K\,)}$$

$$\dfrac{(\,S_A\cdot S_a\,x\,,\,a\langle x\rangle\,,\,K\,)}{(\,S_A\cdot S_a\;\;,\;\;\star\,,\,K\,)}$$

$$\dfrac{(\,S_A\cdot S_a\,x\;\;\;\;,\,a\langle t\rangle\,,\;\;\;\;K\,)}{(\,\{t/x\}(S_A\cdot S_a)\,,\;\;\star\,,\,\{t/x\}K\,)}\;(x \notin t)$$

$$\dfrac{(\,S_A\cdot S_a\,t\;\;\;\;,\,a\langle x\rangle\,,\;\;\;\;K\,)}{(\,\{t/x\}(S_A\cdot S_a)\,,\;\;\star\,,\,\{t/x\}K\,)}\;(x \notin t)$$

$$\dfrac{(\,S_A\cdot S_a\,f(R)\,,\,a\langle f(T)\rangle\,,\,K\,)}{(\,S_A\cdot S_a\,R\;\;\;,\;\;\;a\langle T\rangle\,,\,K\,)}$$

Figure 1: Transitions of the Relational Machine

Definition 2.4 (Big-step operational semantics). The big-step operational semantics of the RMC is given by the evaluation function $(-\Downarrow -)$, defined as follows:

$$\begin{array}{rll} S_A \Downarrow \star & = [(S_A,\varepsilon)] & \\ S_A \Downarrow M\,;N & = [\,(U_A,\tau\sigma) \mid (T_A,\sigma) \in S_A \Downarrow M, & \\ & \qquad (U_A,\tau) \in T_A \Downarrow \sigma N\,] & \\ S_A \Downarrow M^* & = \bigsqcup_{n\in\mathbb{N}} (S_A \Downarrow M^n) & \\ S_A \Downarrow M+N & = (S_A \Downarrow M) \sqcup (S_A \Downarrow N) & \\ S_A\cdot S_a \Downarrow [t]a & = [(S_A\cdot S_a\,t, \varepsilon)] & \\ S_A\cdot S_a\,x \Downarrow a\langle x\rangle & = [(S_A\cdot S_a, \varepsilon)] & \\ S_A\cdot S_a\,x \Downarrow a\langle t\rangle & = [(\{t/x\}(S_A\cdot S_a), \{t/x\})] & (x \notin t) \\ S_A\cdot S_a\,t \Downarrow a\langle x\rangle & = [(\{t/x\}(S_A\cdot S_a), \{t/x\})] & (x \notin t) \\ S_A\cdot S_a\,f(R) \Downarrow a\langle f(T)\rangle & = S_A\cdot S_a\,R \Downarrow a\langle T\rangle & \\ S_A \Downarrow \exists x.\,M & = [\,(T_A,\sigma\setminus y) \mid (T_A,\sigma) \in S_A \Downarrow \{y/x\}M\,] & \\ S_A \Downarrow M & = [] & \text{(otherwise)} \end{array}$$

where y is globally *fresh* in the $\exists x.\,M$ case.

We assume that fresh variables are generated globally, without clashing. In particular, in $(\exists x.\,M)\,;N$ the variable instantiating the $\exists x$ must be free for N as well as M. One may address this practically by carrying along the *formal parameters* of $(\exists x.\,M)\,;N$ into $\exists x.\,M$; we omit this for brevity.

Proposition 2.5 (Big-step semantics is well defined). *The evaluation function $(-\Downarrow -)$ is a total function.*

Proposition 2.6 (Small-step and big-step semantics agree). *For every memory S_A, term M and continuation K, there is a bijection*

between the elements (T_A, σ) *of* $(S_A \Downarrow M)$ *and successful runs*

$$\frac{(\ S_A\ ,\ M\ ,\ \ K\ )}{(\ T_A\ ,\ \star\ ,\ \sigma K\ )}\ .$$

The function $\lfloor - \rfloor$ takes a multiset to its underlying set, and the function $\mathrm{ret}(-)$ projects onto only *return memories* in $\lfloor S_A \Downarrow M \rfloor$.

3 ENCODING COMPUTATIONAL MODELS, I

We embed a number of computational models (or algorithms, in the case of unification) into the RMC to illustrate the origins and purposes of its various constructs. Observe, in particular, that operational semantics is preserved, often in a strong sense. We make use of only one stack, and so we have only one unnamed location, writing $[t]$ and $\langle t \rangle$. Further encodings, using locations, are given in Section 8.

Notation. We introduce vector notation for stacks of variables $\overrightarrow{x} = x_1 \ldots x_n$, and emphasize reversal, as in $\langle \overleftarrow{x} \rangle = \langle x_n \rangle\,;\,\ldots\,;\langle x_1 \rangle$ by pointing the arrow left. Concatenation is given by juxtaposition. In particular, we abbreviate $\exists x_1 \ldots \exists x_n.\, M$ as $\exists \overrightarrow{x}.\, M$. Abusing notation, we denote a singleton memory given by a stack S at location a (and otherwise empty) by S_a.

3.1 Regular expressions

Regular expressions over an alphabet Σ of constants $a, b, c, \ldots$ are given by the following grammar.

$$E, F \ ::=\ \varepsilon \ \mid\ E\,F \ \mid\ E^* \ \mid\ \varnothing \ \mid\ (E|F) \ \mid\ c \in \Sigma$$

Their embedding $[\![-]\!]$ into RMC-terms is as follows.

$$M, N \ ::=\ \star \ \mid\ M\,;N \ \mid\ M^* \ \mid\ 0 \ \mid\ M + N \ \mid\ [c]$$

Here, constant values c are *pushed* to the stack. The regular language $\mathcal{L}_E$ defined by E — a set of words (stacks) over Σ — is thus given directly by the evaluation of $[\![E]\!]$.

Proposition 3.1 (Regular expressions embed). *For a regular expression* E,

$$\mathcal{L}_E \ =\ \mathrm{ret}\lfloor \varepsilon \Downarrow [\![E]\!] \rfloor\ .$$

There is a *dual* embedding $[\![-]\!]^\dagger$ of regular expressions into RMC-terms of the grammar below, given by composition of the original embedding with duality.

$$M, N \ ::=\ \star \ \mid\ M\,;N \ \mid\ M^* \ \mid\ 0 \ \mid\ M + N \ \mid\ \langle c \rangle$$

Here, constant values c are *popped* from the stack. The embedding can be considered as defining a computation which *tests* words (stacks), with a word S accepted if $\varepsilon \in \mathrm{ret}\lfloor S \Downarrow [\![E]\!]^\dagger \rfloor$. The characteristic (or indicator) function of a regular language $\mathcal{L}_E$ is thus given directly by evaluation of $[\![E]\!]^\dagger$.

3.2 Unification

A (first-order) unification algorithm [50] takes a set of formal equations $E = \{s_1 \doteq t_1, \ldots, s_n \doteq t_n\}$ and returns a most general unifier (MGU), a minimal subsitution σ such that $\sigma s_i = \sigma t_i$ for all i, if one exists. In the RMC, we may encode such equations as redexes: $[\![s \doteq t]\!] = [s]\,;\langle t \rangle$. Unification (as a general concept) can then be viewed as embedding in the RMC as the following fragment.

$$M, N \ ::=\ \star \ \mid\ M\,;N \ \mid\ [t] \ \mid\ \langle t \rangle$$

Terms are sequences of pushes and pops of first-order values to and from this location. A set of equations E is encoded as follows.

$$[\![\{s_1 \doteq t_1, \ldots, s_n \doteq t_n\}]\!] = [s_n]\,;\,\ldots\,;[s_1]\,;\langle t_1 \rangle\,;\,\ldots\,;\langle t_n \rangle$$

Note that we have chosen nested redexes, since that is also what the machine reduction for constants (for $[f(s_1, \ldots, s_n)]\,;\langle f(t_1, \ldots, t_n) \rangle$) produces; but a sequence of redexes would have been equally valid. We then observe that evaluation returns an MGU if one exists.

Proposition 3.2 (First-order unification embeds). *For a set of formal equations over algebraic terms* E,

$$\varepsilon \Downarrow [\![E]\!] = [(\varepsilon, \sigma)]$$

where σ *is an MGU for* E*, if one exists, and otherwise* $\varepsilon \Downarrow [\![E]\!] = [\,]$.

For machine evaluation, which does not return a substitution explicitly, we may obtain σ by populating the initial stack with the free variables $x_1 \ldots x_n$ in E. This is the domain of σ; the machine then substitutes into each variable x_i to return σx_i, as follows, so that the relation from input to output stack captures the MGU σ.

$$\frac{(\ x_1\ \ldots\ x_n\ \ \ ,\ [\![E]\!]\ ,\ \varepsilon\ )}{(\ \sigma x_1\ \ldots\ \sigma x_n\ ,\ \ \ \star\ ,\ \varepsilon\ )}$$

3.3 Kappa-calculus

Hasegawa's κ-calculus [39], which featured in the introduction, as formulated by Power and Thielecke [58], embeds as follows.

$$E, E' \ ::=\ E; E' \ \mid \mathsf{PUSH}\, x \mid \qquad \kappa x.E$$

$$M, N \ ::=\ M\,;N \mid \quad [x] \quad \mid \exists x.\, (\langle x \rangle\,;M)$$

A κ-abstraction (first-order λ-abstraction) decomposes into a *new* $\exists x$ and a *pop* $\langle x \rangle$, so that x acts as a binder as well as taking input. The asymmetry of this construct, which rules out terms such as $\langle x \rangle\,;\langle x \rangle\,;[x]$, gives the Cartesian semantics. The machine behaviour illustrates that the decomposition gives the correct behaviour: pop and substitute.

$$\begin{array}{r}
\hline
(\ S\,z\ ,\ \exists x.\, (\langle x \rangle\,;M)\ ,\ \ \ K\ ) \\
\hline
(\ S\,z\ ,\ \langle y \rangle\,;\{y/x\}M\ ,\ \ \ K\ ) \\
\hline
(\ S\,z\ ,\ \ \ \langle y \rangle\ ,\ (\{y/x\}M)\,K\ ) \\
\hline
(\ S\ \ ,\ \ \ \star\ ,\ (\{z/x\}M)\,K\ ) \\
\hline
(\ S\ \ ,\ \ \{z/x\}M\ ,\ \ \ K\ )
\end{array}$$

3.4 Pattern-matching

In a λ-calculus with *pattern-matching* [14, 44], abstractions are over *patterns*, which generally are algebraic terms, instead of over variables. Since we don't have higher-order abstraction, we will stay within κ-calculus. We will use *pairing* as our only pattern, as is standard in a Cartesian setting.

$$E, E' \ ::=\ E; E' \ \mid \mathsf{PUSH}\, p \ \mid\ \kappa p.E \qquad p, q \ ::=\ x \ \mid\ (p, q)$$

In $\kappa p.E$, the free variables of the pattern p bind in E. The embedding in the RMC captures this by introducing each variable as *new*:

$$M, N \ ::=\ M\,;N \ \mid\ [p] \ \mid\ \exists \overrightarrow{x}.\, (\langle p \rangle\,;M) \qquad (\overrightarrow{x} = \mathsf{fv}(p))$$

The behaviour of the RMC-encoding is slightly more general than that of the original pattern-matching calculus, since it can deal with situations such as $[x]\,;\exists y.\,\exists z.\,\langle (y, z) \rangle$ (by substituting into x), which the latter cannot. Pattern-matching calculi address this by

introducing explicit product types. With that constraint, the overall behaviour agrees, as expected.

3.5 Symmetric pattern matching

We embed a first-order fragment of *Theseus*, a *reversible* programming language which is both forwards and backwards deterministic [17, 19, 60]. Its *values* are generated from pairing and injections. Computation is performed by application of *isomorphisms*, expressed using *symmetric pattern-matching* syntax below, where the set of values v_i (respectively, w_i) are restricted by the type system to be *exhaustive* and *non-overlapping*, guaranteeing reversibility. We omit presentation of the type system here for brevity.

$$\{\, v_1 \leftrightarrow w_1 \mid \ldots \mid v_n \leftrightarrow w_n \,\}$$

We interpret values using corresponding function symbols, and embed isomorphisms as below.

$$\exists \vec{x_1}.\, (\langle v_1 \rangle ; [w_1]) + \ldots + \exists \vec{x_n}.\, (\langle v_n \rangle ; [w_n])$$

Each $\vec{x_i}$ contains exactly the variables of v_i and w_i. Isomorphism *inversion* is given by $(-)^\dagger$. The application of isomorphism M to value v is encoded as $[v] ; M$.

3.6 Prolog

We embed pure Prolog, as defined by the following grammar.

Terms:	$t \coloneqq x \mid f(t_1, \ldots, t_n)$	Atoms:	$A \coloneqq P(t_1, \ldots, t_n)$
Clauses:	$C \coloneqq A \coloneq A_1 \ldots A_n$	Programs:	$L \coloneqq C_1 \ldots C_n$

Prolog by defualt evaluates using a *top-down* evaluation strategy, which takes the query — that is, a chosen atom — as a *goal* to be proved and non-deterministically applies clauses, decomposing the set of goals into sub-goals, succeeding when the set is empty.

We consider atoms, as well as terms, to be modelled in the RMC by algebraic terms. The embedding of a clause C with free variables $\vec{x}$ is then given as follows.

$$[\![A \coloneq A_1 \ldots A_n]\!] \;=\; \exists \vec{x}.\, (\langle A \rangle ; [A_1] ; \ldots ; [A_n])$$

We will use the stack to hold the set of goals, with the non-deterministic sum of clauses modelling the program itself. To select the correct solution among the non-deterministic outputs, we add a new constant END to the input stack, which can only be removed when the computation is successfully completed. The embedding of a query-program pair is thus given below.

$$[\![(Q, C_1 \ldots C_n)]\!] \;=\; [\mathsf{END}] ; [Q] ; \left(\sum_{i=1}^{n} [\![C_i]\!] \right)^* ; \langle \mathsf{END} \rangle$$

As we did with unification, by populating the initial stack with the query Q we collect the substitutions generated by the machine run. The completed computation then returns the same instantiation of Q as does Prolog.

Proposition 3.3 (Prolog embeds). *Given as input a query Q and program L, the Prolog abstract interpreter [65] outputs the instance Q' if and only if*

$$\frac{(\ Q\ ,\ [\![(Q, L)]\!]\ ,\ \varepsilon\ )}{(\ Q'\ ,\ \ \star\ ,\ \varepsilon\ )}$$

In contrast to top-down evaluation, *bottom-up* evaluation starts from the set of *facts* (*i.e.*, clauses with a head A but no body A_i) and non-deterministically applies rules *in reverse*, building up the set of logical consequences of facts, succeeding when the query is reached. Remarkably, the RMC reveals a syntactic duality between the two strategies: the dual embedding $[\![(Q, L)]\!]^\dagger$ evaluates the query-program pair via the bottom-up strategy.

4 EQUATIONAL THEORY AND REDUCTION

In this section, we define the equational theory of the RMC and show that it is sound with respect to a natural notion of observational equivalence induced by the operational semantics. We then recover a reduction relation through a natural orientation of (an appropriate subset of) the equations, which we prove confluent. In the star-free fragment, this relation is strongly normalising, giving rise to normal forms.

4.1 The Equational Theory

The equational theory we define is conservative over Kleene algebra, but contains the following additional axioms. For the *new* variable construct, there are axioms reminiscent of those from *nominal* Kleene algebra [33]. There are then axioms internalizing unification, with each corresponding to a step in the standard Martelli and Montanari algorithm for unification (including its "occurs check") [50] . In particular, these axioms include a symmetric version of the β-law of the κ-calculus. There is also a weaker, inequational version of the η-law; and we shall see later that, in the typed setting, we recover the stronger, standard η-law. Finally, there is an axiom allowing the permutation of locations, familiar from the Functional Machine Calculus [40]. Observe that the equational theory respects the duality of the calculus, as expected.

Definition 4.1 (Equational theory). We define the *equational theory* $(=)$ over terms of the RMC to be the least congruence generated by the following axioms.

- Those of Kleene algebra: that is, $(M, ;, \star, +, 0)$ forms an idempotent semiring[2] together with

$$M^* = \star + M ; M^* \qquad M ; N \leq N \rightarrow M^* ; N \leq N$$
$$M^* = \star + M^* ; M \qquad M ; N \leq M \rightarrow M ; N^* \leq M$$

  where $\leq$ is the natural partial order: $M \leq N$ iff $M + N = N$. In the sequel, we work modulo associativity wherever possible.
- Those axiomatizing the *new* variable constructor:

$$\begin{array}{lll} \exists x.\, M & =_\nu M & (x \notin \mathrm{fv}(M)) \\ M ; (\exists x.\, N) & =_\nu \exists x.\, (M ; N) & (x \notin \mathrm{fv}(M)) \\ (\exists x.\, M) ; N & =_\nu \exists x.\, (M ; N) & (x \notin \mathrm{fv}(N)) \\ \exists x.\, (M + N) & =_\nu \exists x.\, M + \exists x.\, N & \\ \exists x.\, \exists y.\, M & =_\nu \exists y.\, \exists x.\, M & \end{array}$$

[2] That is, $(M, ;, \star)$ is a monoid and $(M, +, 0)$ is an idempotent commutative monoid, such that $;$ distributes over $+$ and 0 is annihilative.

- Those axiomatizing β- and η-equivalence and unification:

$$\begin{aligned}
\exists x.\,(N\,;[t]a\,;a\langle x\rangle\,;M) &=_\beta \{t/x\}(N\,;M) && (x \notin t)\\
\exists x.\,(N\,;[x]a\,;a\langle t\rangle\,;M) &=_\beta \{t/x\}(N\,;M) && (x \notin t)\\
\exists x.\,(a\langle x\rangle\,;[x]a) &\leq_\eta \star\\
a[x]\,;\langle x\rangle a &=_\upsilon \star\\
[f(S)]a\,;a\langle f(T)\rangle &=_\upsilon [S]a\,;a\langle T\rangle\\
[f(S)]a\,;a\langle g(T)\rangle &=_\upsilon 0 && (f \neq g)\\
[f(S)]a\,;a\langle x\rangle &=_\omega 0 && (x \in S)\\
[x]a\,;a\langle f(S)\rangle &=_\omega 0 && (x \in S)
\end{aligned}$$

- Those axiomatizing permutation of locations:

$$[s]a\,;b\langle t\rangle =_\pi b\langle t\rangle\,;[s]a \qquad (a \neq b)$$

The permutation law suffices to derive the fact that "push" and "pop" on distinct locations commute with each other.

Lemma 4.2 (Permutation). *The following equations are derivable.*

$$[s]a\,;[t]b = [t]b\,;[s]a \quad a\langle s\rangle\,;b\langle t\rangle = b\langle t\rangle\,;a\langle s\rangle \quad (a \neq b)$$

Proof. Derivable from π together with η, ν and β. □

A β-redex of the κ-calculus encodes as below, left, assuming a chosen, unnamed location. As such, its β-law decomposes into the following two axioms.

$$[t]\,;\exists x.\,(\langle x\rangle\,;M) =_\nu \exists x.\,([t]\,;\langle x\rangle\,;M) =_\beta \{t/x\}M$$

The β-axiom of the RMC generalizes and symmetrizes the second equality above: the $\exists x$ construct may additionally be separated from the sub-term $[t]\,;\langle x\rangle$ by some term N, which is also open for substitution.

We now define a notion of observational equivalence of terms, which we call *machine equivalence*, via the big-step operational semantics. In the following, we take S_A and σM to be open in general, and possibly sharing same variables. Open memories and substitutions are considered equivalent up to a choice of globally fresh variables. Practically, this means machine runs and memories are equipped with a context; we omit this for brevity.

Definition 4.3 (Machine equivalence). *Machine equivalence* is the relation $(\sim)$ on terms defined by

$$M \sim N \quad \text{if} \quad \forall S_A, \sigma.\ \lfloor S_A \Downarrow \sigma M\rfloor = \lfloor S_A \Downarrow \sigma N\rfloor\ .$$

Proposition 4.4 (Congruence). *Machine equivalence is a congruence.*

We can define a similar *machine refinement* $(\lesssim)$ relation on terms as follows.

$$M \lesssim N \quad \text{if} \quad \forall S_A, \sigma.\ \lfloor S_A \Downarrow \sigma M\rfloor \subseteq \lfloor S_A \Downarrow \sigma N\rfloor$$

It is easy to see machine refinement satisfies the property that $M \lesssim N$ if and only if $M + N \sim N$, and so $(\lesssim)$ stands in the same relation to $(\sim)$ as $(\leq)$ does to $(=)$ in the algebraic theory. In particular, it is a partial order closed under all contexts.

Theorem 4.5 (Soundness of equational theory). *The equational theory is sound with respect to machine equivalence:*

$$M = N \implies M \sim N\ .$$

4.2 Reduction and Confluence

It is possible to recover a reduction relation $(\twoheadrightarrow)$ from the equational theory by orientation of the equations. The β and υ equations have an obvious orientation and the ν equations are oriented to bring $\exists x$ to the front of a term, or remove it when it does not bind any variables. We do not consider the η inequation in the untyped case. To maintain confluence in the presence of the Kleene star, we find it necessary to break the symmetry of the equational theory: we can include either the left-handed *unfolding* reduction $M^* \twoheadrightarrow \star + M\,;M^*$ or its dual, but not both. We opt, arbitrarily, for the former. We do not find it sensible to consider the conditional equation $M\,;N \leq N \longrightarrow M^*\,;N \leq N$ or its dual as part of reduction.

We consider the main interest of confluence for reduction to be in dealing with the directed β, υ and ν equations. We thus make several further adaptations to the KA fragment of reduction which simplify the confluence proof. We choose to omit the idempotence equation, in keeping with the more general multiset operational semantics; we omit consideration of associativity and commutativity of $(+)$, although the method of proof should extend straightforwardly to these cases; and we assume right-associativity of sequencing, so terms are of the form $M\,;(N\,;\ldots\,;(P\,;\star)\ldots)$, which also has the pleasant consequence of eliminating the need to consider $(\exists x.\,M)\,;N \twoheadrightarrow \exists x.\,(M\,;N)$ where $x \notin \mathrm{fv}(N)$ and $M\,;\star \twoheadrightarrow M$. For this reason, β-reduction will take place in (also right-associative) *linear contexts*, defined as follows

$$L\{\ \} ::= \{\ \} \mid M\,;L\{\ \} \mid L\{\ \}\,;M$$

and υ-reductions will contain an arbitrary post-fixed term. We also enforce a prioritization of left- over right-distributivity, although we expect this can be lifted. Finally, the permutation equation is oriented left–to–right. The reduction relation is overall clearly *sound* (but not complete) with respect to the equational theory, and thus the operational semantics.

Our proof of confluence makes use of techniques from the field of first-order term rewriting by treating the atomic terms $[t]a$, $a\langle t\rangle$, $\star$ and 0 as constants. Local confluence is shown by analysis of critical pairs, so we observe the star-free fragment is strongly normalising (by the measure given in the proof of Proposition 2.5) and hence confluent by Newman's Lemma. Confluence of the full reduction relation then follows from an application of the Hindley-Rosen Lemma.

Definition 4.6 (Reduction). The reduction relation $(\twoheadrightarrow)$ on terms is given by the following set of rewrite rules.

- KA-reduction:

$$\begin{aligned}
M^* &\twoheadrightarrow \star + M\,;M^*\\
(M+N)\,;P &\twoheadrightarrow (M\,;P)+(N\,;P) && (P \neq M+N)\\
P\,;(M+N) &\twoheadrightarrow (P\,;M)+(P\,;N)\\
M+0 &\twoheadrightarrow M\\
0+M &\twoheadrightarrow M\\
\star\,;M &\twoheadrightarrow M\\
M\,;0 &\twoheadrightarrow 0\\
0\,;M &\twoheadrightarrow 0
\end{aligned}$$

- ν-reductions:

$$\begin{aligned}
\exists x.\,M &\twoheadrightarrow_\nu M && (x \notin M)\\
\exists x.\,(M+N) &\twoheadrightarrow_\nu (\exists x.\,M)+(\exists x.\,N)\\
M\,;\exists x.\,N &\twoheadrightarrow_\nu \exists x.\,(M\,;N)
\end{aligned}$$

- β- and υ-reduction:

$$\begin{array}{rcll}
\exists x.\, L\{[t]a\,;a\langle x\rangle\,;M\} & \to_\beta & \exists x.\, \{t/x\}L\{M\} & (x \notin t)\\
\exists x.\, L\{[x]a\,;a\langle t\rangle\,;M\} & \to_\beta & \exists x.\, \{t/x\}L\{M\} & (x \notin t)\\
[x]a\,;a\langle x\rangle\,;M & \to_\upsilon & M &\\
[f(T)]a\,;a\langle f(S)\rangle\,;M & \to_\upsilon & [T]a\,;a\langle S\rangle\,;M &\\
[f(T)]a\,;a\langle g(S)\rangle\,;M & \to_\upsilon & 0 & (f \neq g)\\
[f(S)]a\,;a\langle x\rangle\,;M & \to_\upsilon & 0 & (x \in f)\\
[x]a\,;a\langle f(S)\rangle\,;M & \to_\upsilon & 0 & (x \in f)
\end{array}$$

- π-reduction:

$$[t]a\,;b\langle s\rangle\,;M \quad \to_\pi \quad b\langle s\rangle\,;[t]a\,;M \quad (a \neq b)$$

Theorem 4.7 (Confluence). *The reduction relation $(\twoheadrightarrow)$ is confluent.*

In the star-free fragment, which is easily shown to be strongly normalising, we thus recover the existence of *normal forms*. Let $\twoheadrightarrow^*$ be the reflexive, transitive closure of $\twoheadrightarrow$. We state the result for the case of one location, but in general *push* and *pop* actions on distinct locations can be arranged according to a chosen ordering.

Corollary 4.8 (Normal forms). *For any closed, star-free RMC-term M, we have that $M \twoheadrightarrow^* N$, where*

$$N \equiv N_1 + \ldots + N_n\,,$$

where each N_i is sum-free and of the form

$$N_i \equiv \exists \vec{x}.\, (\langle s_n\rangle\,;\ldots;\langle s_1\rangle\,;[t_1]\,;\ldots;[t_m])\,,$$

where $\vec{x}$ are the variables of s_j and t_k.

Note, normal forms are taken modulo associativity and commutativity of $+$ and permutation of $\exists x$.

We close this section with a theorem about the equational theory (and thus about operational semantics) proved easily using the existence of normal forms.

Theorem 4.9 (Equations for duality). *We have the following:*

- *for any closed, star-free term M, we have $M \leq M\,;M^\dagger\,;M$;*
- *if M is also sum-free then $M = M\,;M^\dagger\,;M$;*

The first statement is familiar as the unique law added to KA in Kleene algebra *with converse* [15]; the second statement is familiar as the weakening of invertibility in the definition of *inverse monoids* and *inverse categories* [20].

5 THE SIMPLY-TYPED RMC

In this section, we introduce a simple typing discipline giving the input and output arity of terms. This tames the asymmetric nature of stacks and results in a natural langauge for string diagrams, which we explicate in the subsequent section on categorical semantics.

The typing discipline has the pleasant consequence of restricting the modes of failure of the machine. The untyped machine run may fail in a number of ways: either by failure of unification, attempting to evaluate a 0, or by attempting to pop from an empty stack. We consider the first two forms of failure to be intrinsic to the relational nature of the calculus; however, typing ensures that a term, when given an appropriately typed input, never fails due to attempting to pop from an empty stack. This result allows the sound strengthening the η-axiom, which is necessary for a monoidal categorical semantics.

$$\frac{x\colon \rho \in \Gamma}{\Gamma \vdash_{\mathsf{v}} x\colon \rho}\ \mathsf{var} \qquad \frac{\Gamma \vdash_{\mathsf{v}} t_i\colon \rho \quad f^n \in \Sigma}{\Gamma \vdash_{\mathsf{v}} f^n(t_1,\ldots,t_n)\colon \rho}\ \mathsf{const}$$

$$\frac{}{\Gamma \vdash_{\mathsf{c}} \star\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\sigma}_A}\ \mathsf{id} \qquad \frac{\Gamma \vdash_{\mathsf{c}} M\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A \quad \Gamma \vdash_{\mathsf{c}} N\colon \overleftarrow{\tau}_A \Rightarrow \overrightarrow{\upsilon}_A}{\Gamma \vdash_{\mathsf{c}} M\,;N\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\upsilon}_A}\ \mathsf{seq}$$

$$\frac{}{\Gamma \vdash_{\mathsf{c}} 0\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}\ \mathsf{zero} \qquad \frac{\Gamma \vdash_{\mathsf{c}} M\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A \quad \Gamma \vdash_{\mathsf{c}} N\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}{\Gamma \vdash_{\mathsf{c}} M + N\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}\ \mathsf{sum}$$

$$\frac{\Gamma \vdash_{\mathsf{v}} t\colon \tau}{\Gamma \vdash_{\mathsf{c}} [t]a\colon \sigma_a \cdot \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\sigma}_A \cdot \sigma_a\tau}\ \mathsf{push} \qquad \frac{\Gamma \vdash_{\mathsf{v}} t\colon \tau}{\Gamma \vdash_{\mathsf{c}} a\langle t\rangle\colon \tau\sigma_a \cdot \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\sigma}_A \cdot \sigma_a}\ \mathsf{pop}$$

$$\frac{\Gamma, x\colon \rho \vdash_{\mathsf{c}} \quad M\colon \overrightarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}{\Gamma \quad \vdash_{\mathsf{c}} \exists x.\, M\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}\ \mathsf{new}$$

Figure 2: Simply-Typed Relational Machine Calculus.

Let a signature $\Sigma = (\Sigma_0, \Sigma_1)$ of the simply-typed RMC be given by a set Σ_0 of base (or value) types and a set Σ_1 of *typed* function symbols — that is, with an *arity* given by an associated word of Σ_0^*.

Definition 5.1 (Simply-Typed Relational Machine Calculus). A *stack type* is defined by the grammar below, left, and a *memory type* is an A-indexed family of stack types as below, right.

$$\vec{\sigma}, \vec{\tau} ::= \varepsilon \mid \vec{\sigma}\tau \qquad \vec{\sigma}_A, \vec{\tau}_A ::= \{\vec{\tau}_a \mid a \in A\}$$

A *computation type* is an implication between memory types: $\overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$, with the *reverse* of a stack type denoted $\overleftarrow{\tau}$, and the reverse of a memory type $\overleftarrow{\tau}_A$ defined pointwise. We denote *stack type concatenation* as $\vec{\rho} \cdot \vec{\sigma}$ and *memory type concatenation* as $\vec{\rho}_A \cdot \vec{\sigma}_A$, again defined pointwise. The addition of a stack type σ_a to a memory type $\vec{\sigma}_A$ ($a \notin A$) is written $\sigma_A \cdot \sigma_a$.

We define *value* and *computation typing judgements* by the typing relations in Figure 2: they are relations between a *typing context* $\Gamma = x_1\colon \tau_1, \ldots, x_n\colon \tau_n$ — a finite sequence of variables, each associated with a base type — and a term and type of the respective sort, as follows.

$$\Gamma \vdash_{\mathsf{v}} t\colon \tau \qquad \Gamma \vdash_{\mathsf{c}} M\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$$

We will sometimes omit the subscript on the typing relations or leave the context and typing relation implicit and write $M\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$.

Additionally, we judge $\Gamma \vdash t_1 \ldots t_n : \sigma_1 \ldots \sigma_n$ if each $\Gamma \vdash_{\mathsf{v}} t_i\colon \sigma_i$, and similarly we judge $\Gamma \vdash S_A : \vec{\sigma}_A$ if $\Gamma \vdash S_a : \vec{\sigma}_a$ for each $a \in A$. However, for a (multi)set of memories $\mathcal{T}$ we will write $\mathcal{T} : \vec{\tau}_A$ to denote that for each $T_A \in \mathcal{T}$ there exists some context Γ such that $\Gamma \vdash T_A : \vec{\tau}_A$.

Notation. We extend vector notation to contexts, writing $\vec{x}\colon \vec{\tau} = x_1\colon \tau_1, \ldots, x_n\colon \tau_n$, and to simultaneous substitutions, writing $\{S/\vec{x}\} = \{t_1/x_1, \ldots, t_n/x_n\}$ where $S = t_1 \ldots t_n$.

Lemma 5.2. *The following properties of the type system hold:*

- *For each judgement, there is at most one typing derivation.*
- *If $\Gamma \vdash_c M\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$ and $M \twoheadrightarrow N$ then $\Gamma \vdash_c N\colon \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$.*

- *The rules of weakening, exchange, substitution, stack expansion, and duality (the latter three shown below) are admissible.*

$$\frac{\Gamma \vdash_v t : \rho \qquad \Gamma \vdash_c M : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}{\Gamma \vdash_c \{t/x\}M : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}\ \text{subst}$$

$$\frac{\Gamma \vdash_c M : \qquad \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}{\Gamma \vdash_c M : \overleftarrow{\sigma}_A \cdot \overleftarrow{\rho}_A \Rightarrow \overrightarrow{\rho}_A \cdot \overrightarrow{\tau}_A}\ \text{exp} \qquad \frac{\Gamma \vdash_c M \ : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A}{\Gamma \vdash_c M^{\dagger} : \overleftarrow{\tau}_A \Rightarrow \overrightarrow{\sigma}_A}\ \text{dual}$$

Machine progress follows as a corollary of the proof of the following theorem, which is a simple, direct induction on type derivations.

Theorem 5.3 (Machine respects types). *For any term M,*

$$\vdash M : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A \ \Rightarrow\ \forall(\vdash S : \overrightarrow{\sigma}_A).\ \mathrm{ret}\lfloor S_A \Downarrow M \rfloor : \overrightarrow{\tau}_A\ .$$

Practically, a machine run is to be evaluated in a context, into which the $\exists x$ can introduce new, globally fresh variables. Thus, each memory in the output may be associated with a *different* context.

We say a machine state $(\, S_A\,,\, M\,,\, K\,)$ makes *progress* if none of its branches reach a state $(\, S_{A'} \cdot \varepsilon_a\,,\, a\langle x\rangle\,,\, K\,)$, for $A' = A \setminus \{a\}$.

Corollary 5.4 (Machine progress). *For any closed typed term $M : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$ and memory $S_A : \overrightarrow{\sigma}_A$, $(\, S_A\,,\, M\,,\, \varepsilon\,)$ makes progress.*

We define a typed notion of machine equivalence, where we expect input stacks to respect typing.

Definition 5.5 (Typed machine equivalence). *Typed machine equivalence* is defined as the relation $\overrightarrow{x} : \overrightarrow{\rho} \vdash M \sim N : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$ on similarly typed terms, which holds if:

$$\forall(\overrightarrow{x'} : \overrightarrow{\rho'} \vdash R : \overrightarrow{\rho}, S_A : \overrightarrow{\sigma}_A).\ \lfloor S_A \Downarrow \{R/\overrightarrow{x}\}M \rfloor = \lfloor S_A \Downarrow \{R/\overrightarrow{x}\}N \rfloor\ .$$

Due to the property of machine progress, we can soundly strengthen the η-axiom for the *typed* equational theory as follows.

Definition 5.6 (Typed equational theory). The *typed equational theory* $(=)$ relates similarly typed terms of the RMC by the least congruence generated by the axioms of the untyped equational theory, together with the strengthened η-axiom where, given $x : \tau$,

$$\exists x.\ (a\langle x\rangle\,;[x]a) =_\eta \star\ .$$

The η-law of the κ-calculus is decomposed in the RMC as follows

$$\exists x.\ (\langle x\rangle\,;[x]\,;M) =_v \exists x.\ (\langle x\rangle\,;[x])\,;M =_\eta \ \star\ ;M = M \quad (x \notin \mathrm{fv}(M))$$

and indeed our symmetric formulation of the η-axiom is equivalent.

Theorem 5.7 (Soundness of typed equational theory). *The typed equational theory is sound with respect to typed machine equivalence:*

$$\Gamma \vdash M = N : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A \ \Rightarrow\ \Gamma \vdash M \sim N : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A\ .$$

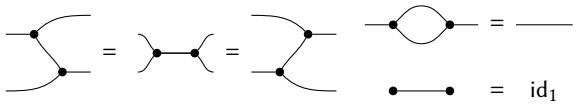
6 CATEGORICAL SEMANTICS

We give a sound and complete categorical semantics for the sum-free, star-free and function-symbol-free fragment of the RMC, illustrating how the core calculus provides an operational interpretation of Frobenius monoids and thus of a natural class of string diagrams. The extension of the categorical semantics to include non-deterministic sum follows easily, but we leave to future work the categorical interpretation of iteration and of algebraic terms.

Frobenius monoids are used as a categorical abstraction for modelling a range of computational phenomena, many of interest to programming language theorists. While the literature contains computational interpretations of compact closed categories [18] and certain accounts of operational semantics of string diagrams [11, 12], to the knowledge of the authors, the RMC gives the first syntactic account of Frobenius monoids via a notion of β-reduction, and the first operational account of Frobenius monoids via a stack machine. To highlight the correspondence with string diagrams, we work with a single location, but the results extend easily to multiple locations [3].

Notation. We denote atomic types using $A \in \Sigma_0$, and for the purposes of this section we consider Σ_0 to contain a unique base type, although the results generalize easily. Given a category C and objects $X, Y \in C$ we denote by $C(X, Y)$ the corresponding homset. We write the identity morphism on X as id_X, or simply X. Composition is written in diagrammatic order, using infix $(;)$. We denote the tensor product of a *symmetric monoidal category (SMC)* C by $\otimes$, its unit by 1 and its symmetry natural transformation as sym [49]. We elide all associativity and unit isomorphisms associated with monoidal categories. Recall that a commutative monoid in a given SMC is an object X together with a multiplication $\mu : X \otimes X \to X$ and a unit $\eta : 1 \to X$ satisfying the expected associativity, commutativity and unit equations, and a cocommutative comonoid is defined dually. We denote by CMon the category of commutative monoids. We will use string diagrams throughout this section to describe (co-)monoids in SMCs.

Definition 6.1 (Frobenius monoid). An *extra-special commutative Frobenius monoid* in an SMC $(C, \otimes, 1)$ is given by a tuple $(X, \delta, \epsilon, \mu, \eta)$ consisting of a cocommutative comonoid (X, δ, ϵ) and a commutative monoid (X, μ, η) that further satisfy the *Frobenius*, *special*, and *extra* equations, which are given below in the string diagrammatic language of Figure 3.



The categorical abstraction of relations which is modelled by the RMC is that of *extra-hypergraph categories*, as defined below. This is a slight variant of the usual definition of *hypergraph category* [29], which additionally requires that the relevant Frobenius structures satisfy the *extra* equation. Indeed, a paradigmatic example of such a catgeory is that of sets and relations.

Definition 6.2. [Extra-hypergraph category] An *extra-hypergraph category* is a SMC C in which each object $X \in C$ is equipped with an extra-special commutative Frobenius structure $(\delta_X, \epsilon_X, \mu_X, \eta_X)$, which together satisfy the following coherences:

$$\delta_{X\otimes Y} = (\delta_X \otimes \delta_Y);(X \otimes \mathrm{sym}_{X,Y} \otimes Y) \qquad \epsilon_{X\otimes Y} = \epsilon_X \otimes \epsilon_Y$$
$$\mu_{X\otimes Y} = (X \otimes \mathrm{sym}_{X,Y} \otimes Y);(\mu_A \otimes \mu_Y) \qquad \eta_{X\otimes Y} = \eta_X \otimes \eta_Y$$

and the unit coherence that $\eta_1 = \mathrm{id}_1 = \epsilon_1$. A functor of extra-hypergraph categories is a strong symmetric monoidal functor that additionally preserves the Frobenius structures.

We can define the hypergraph categorical structure of the RMC by assigning terms to string diagrams as in Figure 3. The wires represent the input and output stacks, with the head of the stack at the top. In contrast to the λ-calculus, categorical composition of terms is given by sequential compostion $M\,;N$ rather than by

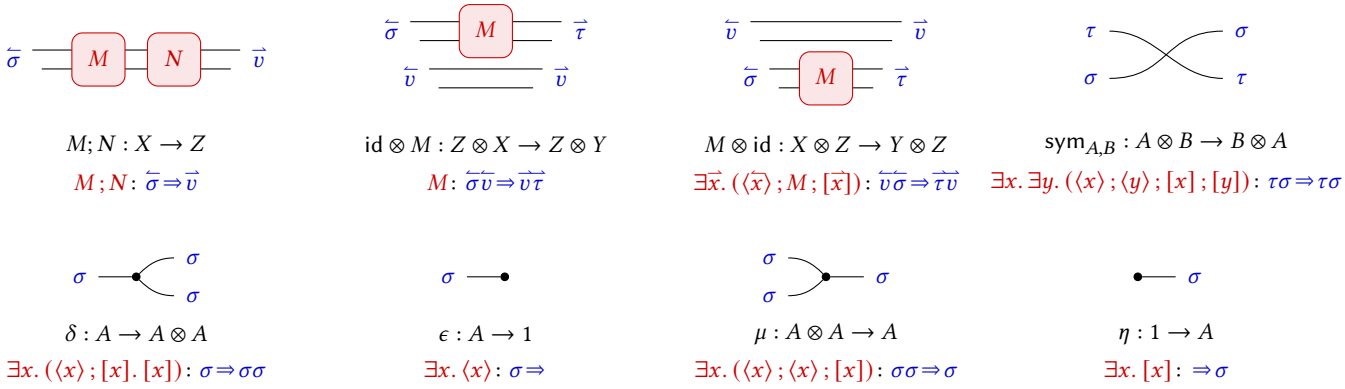


Figure 3: Equipment of the Relational Machine Category: string diagrams, categorical combinators, and RMC-terms.

substitution. Right-action id ⊗ M of the tensor product is given by stack expansion (as described in Lemma 5.2). Left-action M ⊗ id lifts the arguments for υ from the stack as variables $\overleftarrow{x}$, to restore them after evaluating M on the remaining stack. The remaining equipment is familiar from the discussion in the introduction of this paper.

Definition 6.3. The *Relational Machine Category*, RMC[Σ], of RMC-terms, is defined by the following data.

- Objects: stack types $\vec{\tau}$ over Σ_0
- Morphisms: RMC[Σ]$(\vec{\sigma}, \vec{\tau})$ is the set of closed simply-typed terms $\vdash_c M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$ over Σ, modulo (=).
- Identity: at type $\vec{\tau}$, the term $\star : \vec{\tau} \Rightarrow \vec{\tau}$
- Composition: for terms $M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$ and $N : \overleftarrow{\tau} \Rightarrow \vec{\upsilon}$, their composition is $M\,;N : \overleftarrow{\sigma} \Rightarrow \vec{\upsilon}$
- Tensor product: on objects $\vec{\sigma} \otimes \vec{\tau} = \vec{\sigma} \cdot \vec{\tau}$, with unit $1 = \varepsilon$.
- The right- and left-action of the tensor product on morphisms, the symmetry, and the Frobenius monoids at base type are given by Figure 3. The associators and unitors are given by the identity.

Note that, because the objects of RMC[Σ] are inductively generated from Σ_0, it suffices to define a Frobenius monoid on each base type, with Frobenius monoids at compound types defined using the compatibility conditions of Definition 6.2.

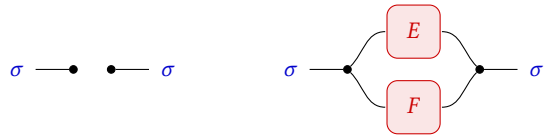
Theorem 6.4 (Soundness). *The Relational Machine Category RMC[Σ] is an extra-hypergraph category.*

It is immediate from the axioms of Kleene algebra that RMC[Σ] is also *CMon-enriched*, *i.e.*, every homset RMC[Σ]$(\vec{\sigma}, \vec{\tau})$ is a commutative monoid with addition (+) and unit 0 and composition of morphisms distributing over addition. This is an easy and appropriate extension of the model to incorporate non-determinism.

Completeness of the categorical semantics for the core fragment follows an easy normal form argument. Abusing notation, let Σ_0 denote a signature with a unique base type and no function symbols.

Theorem 6.5 (Completeness). *The subcategory of sum-free, star-free terms of RMC[Σ_0] is equivalent to the free extra-hypergraph category.*

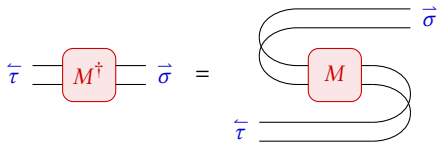
We conclude this section by giving some simple examples of RMC-terms and their corresponding string diagrams. Following existing work on string diagrams for (the positive fragment of) Tarski's calculus of relations [8, 10, 13, 37, 66], we give terms and diagrams whose anticipated relational denotation is given by *top* (*i.e.*, everything is related), *intersection*, and *converse*. The remaining operations of the calculus of relations are already included in the grammar of regular expressions (see subsection 3.1). Top and intersection are encoded by the terms below.



We can characterize the converse of a term in the typed RMC as follows, with its corresponding string diagram given below.

Proposition 6.6 (Typed Duality). *For any closed term $M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$ we have the following, where $\vec{x} : \vec{\tau}$ and $\vec{y} : \vec{\sigma}$:*

$$M^\dagger \;=\; \exists \vec{x}\vec{y}.\, (\langle \vec{x} \rangle\,;[\vec{y}]\,;M\,;\langle \overleftarrow{x} \rangle\,;[\vec{y}]) : \overleftarrow{\tau} \Rightarrow \vec{\sigma}\,.$$



Note, in the above diagram we denote the compositions $\eta;\delta$ and $\mu;\epsilon$ as cups and caps, since it is well-known that they induce a (self-dual) compact closed structure.

7 RELATIONAL SEMANTICS

In this section, we investigate in detail the relational semantics of the RMC. Although the denotational semantics is very close to the input/output behaviour of the big-step semantics, there are some differences: forgetting the output substitutuion, and the interpretation of $\exists x.\, M$ and $\langle t \rangle$. The former is interpreted semantically as

the union of all possible interpretations of x as some closed term t, leaving the latter to become a simple equality check predicate; there is no need for unification since free variables are dealt with by (possibly infinite) non-determinism of the interpretation of $\exists x$.

For simplicity, we again work with a single base type in Σ_0 and a single location, but the results generalize easily. Now, let us recall the extra-hypergraph structure of the category of sets and relations.

Definition 7.1. The category Rel of sets and relations forms an extra-hypergraph category with the tensor product is given by the Cartesian product, the unit given by the singleton set $\{\varepsilon\}$, and a Frobenius monoid on each object A given by $\delta_A = \{(x,(x,x)) \mid x \in A\}$, $\epsilon_A = \{(x,\varepsilon) \mid x \in A\}$ and with μ_A and η_A by their respective converses.

The relational semantics of typed RMC-terms can easily be read off their normal forms: for example, compare the relational semantics of δ, ϵ, μ and η with the RMC-terms interpreting the same morphisms in Figure 3.

We define the *relational semantics* of the RMC as an extra-hypergrpah functor $[\![-]\!] : \mathrm{RMC}[\Sigma] \to \mathrm{Rel}$. Let $T0$ be the set of *closed* algebraic terms built over Σ, *i.e.* containing no variables. To interpret terms in a context Γ, we define a valuation $v : \Gamma \to T0$; the valuation $v\{x \leftarrow t\}$ assigns t to x and otherwise behaves as v. We elide the isomorphism between closed algebraic terms t in the syntax of RMC-terms and $t \in T0$. We extend the action of v to algebraic terms with variables Γ so that $v(f(t_1,\ldots,t_n)) = f(v(t_1),\ \ldots,\ v(t_n))$. In the following, we consider relations $A \to B$ as functions $A \to \mathcal{P}(B)$, where $\mathcal{P}$ is the powerset functor.

Definition 7.2 (Relational semantics). The *relational semantics* of the typed RMC is defined inductively on types by

$$[\![e]\!] = 1 \qquad [\![\vec{\sigma}\tau]\!] = [\![\vec{\sigma}]\!] \otimes T0$$

We define the relation $[\![\Gamma \vdash M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_v$ inductively on the type derivation of terms as follows, and may abbreviate this as $[\![M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_v$ or simply $[\![M]\!]_v$.

$$\begin{aligned}
[\![\Gamma \vdash \star : \overleftarrow{\sigma} \Rightarrow \vec{\sigma}]\!]_v(S) &= [S] \\
[\![\Gamma \vdash M;N : \overleftarrow{\sigma} \Rightarrow \vec{v}]\!]_v(S) &= [U \mid T \in [\![M]\!]_v(S),\ U \in [\![N]\!]_v(T)] \\
[\![\Gamma \vdash 0 : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_v(S) &= [\,] \\
[\![\Gamma \vdash M+N : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_v(S) &= [\![M]\!]_v(S) \sqcup [\![N]\!]_v(S) \\
[\![\Gamma \vdash [t] : \overleftarrow{\sigma} \Rightarrow \vec{\sigma}\tau]\!]_v(S) &= [S\, v(t)] \\
[\![\Gamma \vdash \langle t\rangle : \tau\overleftarrow{\sigma} \Rightarrow \vec{\sigma}]\!]_v(S\,u) &= \begin{cases} [\vec{s}] & \text{if } t = v(u) \\ [\,] & \text{otherwise} \end{cases} \\
[\![\Gamma \vdash \exists x.\, M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_v(S) &= \bigsqcup_{t \in T0} [\![\Gamma, x : \rho \vdash M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_{v\{x \leftarrow t\}}(S)
\end{aligned}$$

Given a closed term $\vdash M$, its relational semantics $[\![M]\!]$ (written without a subscript) is given by $\lfloor [\![M]\!]_\varepsilon \rfloor$, where ε is the empty valuation.

From the following lemma, soundness of the semantics follows.

Lemma 7.3 (Substitution). *For any $\Gamma, x : \rho \vdash M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$, we have*

$$[\![\Gamma, x : \rho \vdash M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_{v\{x \leftarrow t\}} = [\![\Gamma \vdash \{t/x\}M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_v\,.$$

Lemma 7.4 (Relational semantics is hypergraph functor). *The relational semantics $[\![-]\!] : \mathrm{RMC}[\Sigma] \to \mathrm{Rel}$ is a well-defined extra-hypergraph functor.*

In fact, if we additionally consider $\mathrm{RMC}[\Sigma]$ and Rel as CMon-enriched categories, with the monoid on homsets in Rel given by the union of relations, it is easy to see the relational semantics functor is also CMon-enriched functor.

8 ENCODING COMPUTATIONAL MODELS, II

We conclude with further encodings of computational models into the RMC, this time demonstrating the use of locations. This innovation of the Functional Machine Calculus [4, 40] can be used to capture stateful behaviour.

8.1 Turing Machines

We demonstrate an encoding of a Turing machine $\mathcal{M}$ with a set of *states* S, initial state $s \in S$, halting state $h \in S$, alphabet Σ, blank symbol $0 \in \Sigma$, and transition function $\delta : (S \setminus \{h\}) \times \Sigma \to S \times \Sigma \times \{\boldsymbol{L}, \boldsymbol{R}\}$. To do so, we work with an RMC signature consisting of $S \cup \Sigma$ and three locations l, r, and q, used to record the tape to the left of the head, the tape under and to the right of the head (in reverse), and the current state, respectively. We thus encode a tape move $m \in \{\boldsymbol{L}, \boldsymbol{R}\}$ as follows.

$$[\![m]\!] = \begin{cases} \exists x.\, (l\langle x\rangle\,;[x]r) & \text{if } m = \boldsymbol{L} \\ \exists x.\, (r\langle x\rangle\,;[x]l) & \text{if } m = \boldsymbol{R} \end{cases}$$

A Turing machine $\mathcal{M}$ then encodes as an iterated non-deterministic sum of transitions.

$$[\![\mathcal{M}]\!] = \left(\sum_{\delta(s,a)=(s',a',m)} q\langle s\rangle\,;[s']q\,;r\langle a\rangle\,;[a']r\,;[\![m]\!] \right)^*$$

Instantiating S_l and S_r with streams of blank symbols,[3] we have the following result.

Proposition 8.1 (The RMC implements Turing machines). *A Turing machine $\mathcal{M}$ halts with output tapes T_l and T_r to the left and right of the head, respectively, if and only if there exists a machine run*

$$\frac{(\ S_l \cdot S_r \cdot s_q\ ,\ [\![\mathcal{M}]\!]\ ,\ \varepsilon\ )}{(\ T_l \cdot T_r \cdot h_q\ ,\quad \star\ ,\ \varepsilon\ )}\,.$$

8.2 Arithmetic

We will use the example of arithmetic to demonstrate how to encode recursive computation on algebraic terms using Kleene star and new variables. We additionally show the use of multiple locations. Values are composed of *zero*, *successor*, *addition*, and *multiplication*:

$$s, t \ ::=\ Z \mid S(t) \mid +(s,t) \mid *(s,t)$$

We show how to compute the normal form (a term over only *zero* and *successor*) of an arbitrary arithmetic expression in the RMC.

[3] It also easy enough to simulate an infinite tape with stacks, using special end-of-stack markers $\bot$ and $\top$. The encoding of a read of $a \in \Sigma$ then additionally checks for the end of either stack, pushing markers along as needed: that is, in the case of $a = 0$, using $r\langle a\rangle + (r\langle\bot\rangle\,;[\bot]l) + (r\langle\top\rangle\,;[\top]r)$ instead of $r\langle a\rangle$.

First, the term A implements the laws of Peano arithmetic as *generalised axioms*: terms of the form $\langle s \rangle ; [t]$.

$$A = \begin{pmatrix} \exists x. \quad \langle +(Z,x) \rangle ; [x] \\ + \exists xyz. \langle +(S(x),y) \rangle ; [S(+(x,y))] \\ + \exists x. \quad \langle *(Z,x) \rangle ; [Z] \\ + \exists xyz. \langle *(S(x),y) \rangle ; [+(y,*(x,y))] \end{pmatrix}$$

Since we cannot apply rules inside terms, we will transfer every outer successor to a separate *answer* location a. This is the purpose of the term B, which further removes a final *zero* from the input.

$$B = \langle Z \rangle + \exists xy. \langle S(x) \rangle ; [x] ; a\langle y \rangle ; [S(y)]a$$

To select the correct solution among the non-deterministic outputs of A and B, we add a new constant END to the input stack. This will need to be removed in evaluation, but that can only happen when the computation is complete. The implementation is then the following.

Proposition 8.2 (The RMC implements arithmetic). *For a value t in the language of arithmetic, we have the following evaluation, where n is the normal form of t.*

$$\mathsf{END}\, t \cdot Z_a \Downarrow (A+B)^* ; \langle \mathsf{END} \rangle = [(n_a, \sigma)]$$

Observe that our implementation is *tail recursive*, not general recursive. That is because the Kleene star creates a loop, equivalent to tail recursion, while general recursion is much less easy to encode.

9 CONCLUSION AND FURTHER RESEARCH

We believe that by exposing the duality of relational programming in syntax we have arrived at a natural and convincing foundational model of the paradigm. We hope that the RMC will allow the development of tools and reasoning techniques for relational programming, similar to those founded in the λ-calculus which have proved so important for functional programming: type systems and denotational semantics, operational semantics, equational reasoning and confluent reduction. In this paper, we proved fundamental results supporting this new calculus, focussing on showing it meets the design criteria set out and justified in the introduction. We now outline several areas of future work.

9.1 Further Research

Foundations for functional logic programming. It appears straightforward to conceive of a higher-order RMC, based on the higher-order κ-calculus and Functional Machine Calculus (FMC) [4, 40, 58], which would have potential as a foundational model of *functional* logic programming. Already, the FMC, which seeded this work, has shown how to seamlessly integrate the higher-order κ-calculus with global state, by the use of locations — which are also present in the RMC. Thus, there is potential, even, for a unification of logic, functional *and imperative* programming.

Bridging programming languages and string diagrams. String diagrams have found practical applications in an increasing range of domains [22, 42, 51, 55]. This often involves their implementation at scale: for example, the ZX-diagrams used for quantum circuit optimization may have on the order of 10^4–10^7 nodes. As string diagrams grow, tools to represent and reason about them become more and more important [6, 7]. This can be seen by the introduction of circuit description languages which allow the high-level specification of algorithms which can yet be compiled down to circuit (or diagram) level [32, 48] . The RMC gives a principled foundation to programming languages for string diagrams — at least for the class given by Frobenius monoids. A higher-order RMC would allow the factorization of large diagrams and thus their compact representation.

Although the class of string diagrams this paper deals with is limited, we believe it is possible to extend the RMC in a way which gives a unifying account of — and uniform syntax and operational semantics to — a wide class of string diagrams via *unification modulo theory*. For example, by considering values from the theory of commutative monoids, and performing unification modulo this theory, we can represent as computations string diagrams of *graphical resource algebra* [9]; by considering values from the theory of Abelian groups, we can represent as computations string diagrams of *graphical linear algebra* and the *phase-free ZX calculus*. In each of these cases we have a constant 0 and a binary function symbol $+$, and terms

$$\begin{array}{ll} [0] & \exists x. \exists y. (\langle x \rangle ; \langle y \rangle ; [x+y]) \\ \langle 0 \rangle & \exists x. \exists y. (\langle x+y \rangle ; [y] ; [x]) \end{array}$$

providing a second commutative monoid structure and its dual, with the expected equational theory induced by the operational semantics of unification modulo. We hope further for links to (differential) linear logic and monoidal differential categories via the observation that the free commutative (co-)monoid (co-)monad models the exponential modality [5].

Weighted relations. Although there are a wide range of languages with a relational semantics, the range of languages taking a *weighted* relational semantics [47, 67] of some form is vast. Broadly conceived, these include linear-algebraic [2, 68], probabilistic and quantum λ-calculi [62], and classes of string diagrams modelling similar domains [8, 10, 13, 22, 23, 37, 42]. In fact, the RMC generalizes easily to include *weighted* terms and thus weighted non-determinism. Restricting this general non-determinism in order to give accounts of stochastic or unitary processes in probabilistic or quantum domains, respectively, is less immediate and will require more sophisticated type systems. However, Frobenius monoids have already been used to axiomatize the exact conditioning operation of probabilistic programming languages (albeit in the finite dimensional case) [26], and thus investigation into a *probabilistic machine calculus* appears warranted [59]. Quantum processes also admit Frobenius monoids as a fundamental primitive[22, 23, 42]; furthermore, they include as essential aspects both non-determinism and reversiblity, and thus investigation into a *quantum machine calculus* appears warranted, too.

REFERENCES

[1] Michael Arntzenius and Neelakantan R. Krishnaswami. Datafun: a functional datalog. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, Nara, Japan, September 18-22, 2016*, pages 214–227. ACM, 2016.
[2] Ali Assaf, Alejandro Díaz-Caro, Simon Perdrix, Christine Tasson, and Benoît Valiron. Call-by-value, call-by-name and the vectorial behaviour of the algebraic λ-calculus. *Log. Methods Comput. Sci.*, 10(4), 2014.
[3] Chris Barrett. On the simply-typed functional machine calculus: Categorical semantics and strong normalisation, ph.d. thesis. arXiv.2305.16073, 2023.

[4] Chris Barrett, Willem Heijltjes, and Guy McCusker. The functional machine calculus II: semantics. In *31st EACSL Annual Conference on Computer Science Logic, CSL 2023, February 13-16, 2023, Warsaw, Poland*, volume 252 of *LIPIcs*, pages 10:1–10:18, 2023.

[5] Richard Blute, J. Robin B. Cockett, and Robert A. G. Seely. Differential categories. *Math. Struct. Comput. Sci.*, 16(6):1049–1083, 2006.

[6] Filippo Bonchi, Fabio Gadducci, Aleks Kissinger, Pawel Sobocinski, and Fabio Zanasi. String diagram rewrite theory i: Rewriting with frobenius structure. *J. ACM*, 69(2), mar 2022.

[7] Filippo Bonchi, Fabio Gadducci, Aleks Kissinger, Pawel Sobocinski, and Fabio Zanasi. String diagram rewrite theory ii: Rewriting with symmetric monoidal structure. *Mathematical Structures in Computer Science*, 32(4):511–541, 2022.

[8] Filippo Bonchi, Alessandro Di Giorgio, and Alessio Santamaria. Deconstructing the calculus of relations with tape diagrams. *Proceedings of the ACM on Programming Languages*, 7(POPL):1864–1894, 2023.

[9] Filippo Bonchi, Joshua Holland, Robin Piedeleu, Paweł Sobociński, and Fabio Zanasi. Diagrammatic algebra: from linear to concurrent systems. *Proc. ACM Program. Lang.*, 3(POPL), jan 2019.

[10] Filippo Bonchi, Robin Piedeleu, Pawel Sobocinski, and Fabio Zanasi. Graphical Affine Algebra. In *2019 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–12, Vancouver, BC, Canada, June 2019. IEEE.

[11] Filippo Bonchi, Robin Piedeleu, Pawel Sobocinski, and Fabio Zanasi. Contextual equivalence for signal flow graphs. In Jean Goubault-Larrecq and Barbara König, editors, *Foundations of Software Science and Computation Structures - 23rd International Conference, FOSSACS 2020, ETAPS 2020, April 25-30, 2020, Proceedings*, volume 12077 of *Lecture Notes in Computer Science*, pages 77–96. Springer, 2020.

[12] Filippo Bonchi, Robin Piedeleu, Paweł Sobociński, and Fabio Zanasi. Bialgebraic foundations for the operational semantics of string diagrams. *Information and Computation*, 281:104767, December 2021.

[13] Filippo Bonchi, Jens Seeber, and Pawel Sobocinski. Graphical conjunctive queries. In *27th EACSL Annual Conference on Computer Science Logic, CSL 2018*, volume 119 of *LIPIcs*, pages 13:1–13:23, 2018.

[14] V. Breazu-Tannen, D. Kesner, and L. Puel. A typed pattern calculus. In *[1993] Proceedings Eighth Annual IEEE Symposium on Logic in Computer Science*, pages 262–274, 1993.

[15] Paul Brunet and Damien Pous. Algorithms for kleene algebra with converse. *J. Log. Algebraic Methods Program.*, 85(4):574–594, 2016.

[16] Ana C. Calderon and Guy McCusker. Understanding Game Semantics Through Coherence Spaces. *Electronic Notes in Theoretical Computer Science*, 265:231–244, September 2010.

[17] Kostia Chardonnet, Alexis Saurin, and Benoît Valiron. A curry-howard correspondence for linear, reversible computation. In Bartek Klin and Elaine Pimentel, editors, *31st EACSL Annual Conference on Computer Science Logic, CSL 2023, February 13-16, 2023, Warsaw, Poland*, volume 252 of *LIPIcs*, pages 13:1–13:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

[18] Chao-Hong Chen and Amr Sabry. A computational interpretation of compact closed categories: reversible programming with negative and fractional types. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, January 2021.

[19] Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. Symmetries in reversible programming: from symmetric rig groupoids to reversible programming languages. *Proceedings of the ACM on Programming Languages*, (POPL):1–32, January 2022.

[20] J. Robin B. Cockett and Stephen Lack. Restriction categories I: categories of partial maps. *Theor. Comput. Sci.*, 270(1-2):223–259, 2002.

[21] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.

[22] Bob Coecke and Ross Duncan. Interacting Quantum Observables: Categorical Algebra and Diagrammatics. *New Journal of Physics*, 13(4):043016, April 2011.

[23] Bob Coecke, Dusko Pavlovic, and Jamie Vicary. A new description of orthogonal bases. *Mathematical Structures in Computer Science*, 23(3):555–567, June 2013.

[24] J. H. Conway. Regular algebra and finite machines, 1971. Chapman and Hall Mathematics Series (1971).

[25] Daniel de Carvalho. Execution time of λ-terms via denotational semantics and intersection types. *Mathematical Structures in Computer Science*, 28(7):1169–1203, 2018.

[26] Elena Di Lavore and Mario Román. Evidential Decision Theory via Partial Markov Categories. In *2023 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–14, Boston, MA, USA, June 2023. IEEE.

[27] Thomas Ehrhard and Laurent Regnier. The differential lambda-calculus. *Theoretical Computer Science*, 309(1-3):1–41, December 2003.

[28] Andrzej Filinski. Declarative continuations: an investigation of duality in programming language semantics. In David H. Pitt, David E. Rydeheard, Peter Dybjer, Andrew M. Pitts, and Axel Poigné, editors, *Category Theory and Computer Science, Manchester, UK, September 5-8, 1989, Proceedings*, volume 389 of *Lecture Notes in Computer Science*, pages 224–249. Springer, 1989.

[29] Brendan Fong and David I. Spivak. Hypergraph Categories, January 2019. arXiv:1806.08304 [cs, math].

[30] Brendan Fong and Fabio Zanasi. Universal constructions for (co)relations: categories, monoidal categories, and props. *Log. Methods Comput. Sci.*, 14(3), 2018.

[31] Jason Hemann Daniel P. Friedman. Kanren : A minimal functional core for relational programming. 2013.

[32] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. Proto-quipper with dynamic lifting. *CoRR*, abs/2204.13041, 2022.

[33] Murdoch James Gabbay and Vincenzo Ciancia. Freshness and name-restriction in sets of traces with names. In Martin Hofmann, editor, *Foundations of Software Science and Computational Structures - 14th International Conference, FOSSACS 2011*, volume 6604, pages 365–380, 2011.

[34] Dan R. Ghica and Guy McCusker. Reasoning about idealized ALGOL using regular languages. In *Automata, Languages and Programming, 27th International Colloquium, ICALP 2000, Geneva, Switzerland, July 9-15, 2000, Proceedings*, volume 1853 of *Lecture Notes in Computer Science*, pages 103–115. Springer, 2000.

[35] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50:1–102, 1987.

[36] Jean-Yves Girard. Normal functors, power series and λ-calculus. *Annals of Pure and Applied Logic*, 37(2):129–177, February 1988.

[37] Tao Gu and Fabio Zanasi. Functorial semantics as a unifying perspective on logic programming. In *9th Conference on Algebra and Coalgebra in Computer Science, CALCO 2021, August 31 to September 3, 2021, Salzburg, Austria*, volume 211 of *LIPIcs*, pages 17:1–17:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[38] Michael Hanus. Functional Logic Programming: From Theory to Curry. In Andrei Voronkov and Christoph Weidenbach, editors, *Programming Logics: Essays in Memory of Harald Ganzinger*, Lecture Notes in Computer Science, pages 123–168. Springer, Berlin, Heidelberg, 2013.

[39] Masahito Hasegawa. Decomposing typed lambda calculus into a couple of categorical programming languages. In *Category Theory and Computer Science, 6th International Conference, CTCS '95, Cambridge, UK, August 7-11, 1995, Proceedings*, volume 953 of *Lecture Notes in Computer Science*, pages 200–219. Springer, 1995.

[40] Willem Heijltjes. The functional machine calculus. In *Proceedings of the 38th Conference on the Mathematical Foundations of Programming Semantics, MFPS XXXXVIII*, volume 1 of *ENTICS*, 2022.

[41] Tony Hoare and Stephan van Staden. The laws of programming unify process calculi. *Sci. Comput. Program.*, 85:102–114, 2014.

[42] Emmanuel Jeandel, Simon Perdrix, and Renaud Vilmart. Completeness of the zx-calculus. *Log. Methods Comput. Sci.*, 16(2), 2020.

[43] Robin Kaarsgaard and Mathys Rennela. Join Inverse Rig Categories for Reversible Functional Programming, and Beyond. *Electronic Proceedings in Theoretical Computer Science*, 351:152–167, December 2021.

[44] Jan Willem Klop, Vincent van Oostrom, and Roel de Vrijer. Lambda calculus with patterns. *Theoretical Computer Science*, 398(1):16–31, May 2008.

[45] Dexter Kozen. On hoare logic and kleene algebra with tests. *ACM Trans. Comput. Log.*, 1(1):60–76, 2000.

[46] Jean-Louis Krivine. A call-by-name lambda-calculus machine. *Higher-Order and Symbolic Computation*, 20(3):199–207, September 2007.

[47] Jim Laird, Giulio Manzonetto, Guy McCusker, and Michele Pagani. Weighted Relational Models of Typed Lambda-Calculi. In *2013 28th Annual ACM/IEEE Symposium on Logic in Computer Science*, pages 301–310, New Orleans, LA, USA, June 2013. IEEE.

[48] Bert Lindenhovius, Michael W. Mislove, and Vladimir Zamdzhiev. Enriching a linear/non-linear lambda calculus: A programming language for string diagrams. In Anuj Dawar and Erich Grädel, editors, *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018*, pages 659–668. ACM, 2018.

[49] Saunders Mac Lane. *Categories for the working mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, second edition, 1998.

[50] Alberto Martelli and Ugo Montanari. An Efficient Unification Algorithm. *ACM Transactions on Programming Languages and Systems*, 4(2):258–282, April 1982.

[51] Paul-André Melliès. Local states in string diagrams. In *Rewriting and Typed Lambda Calculi - Joint International Conference, RTA-TLCA 2014*, volume 8560, pages 334–348, 2014.

[52] Dale Miller and Gopalan Nadathur. Higher-order logic programming. In Ehud Shapiro, editor, *Third International Conference on Logic Programming, July 14-18, 1986, Proceedings*, volume 225, pages 448–462, 1986.

[53] Robin Milner. *Communicating and mobile systems - the Pi-calculus*. Cambridge University Press, 1999.

[54] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

[55] Keiko Nakata and Masahito Hasegawa. Small-step and big-step semantics for call-by-need. *CoRR*, abs/0907.4640, 2009.

[56] C.-H. Luke Ong. Quantitative semantics of the lambda calculus: Some generalisations of the relational model. In *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–12, June 2017.

[57] Andrew M. Pitts and Ian David Bede Stark. Observable properties of higher order functions that dynamically create local names, or what's new? In *Mathematical Foundations of Computer Science 1993, 18th International Symposium, MFCS'93, Gdansk, Poland, August 30 - September 3, 1993, Proceedings*, volume 711 of *Lecture*

Notes in Computer Science, pages 122–141. Springer, 1993.
[58] John Power and Hayo Thielecke. Closed freyd- and kappa-categories. In *Automata, Languages and Programming, 26th International Colloquium, ICALP'99, Prague, Czech Republic, July 11-15, 1999, Proceedings*, volume 1644 of *Lecture Notes in Computer Science*, pages 625–634. Springer, 1999.
[59] Wojciech Rozowski, Tobias Kappé, Dexter Kozen, Todd Schmid, and Alexandra Silva. Probabilistic guarded KAT modulo bisimilarity: Completeness and complexity. In Kousha Etessami, Uriel Feige, and Gabriele Puppis, editors, *50th International Colloquium on Automata, Languages, and Programming, ICALP 2023, July 10-14, 2023, Paderborn, Germany*, volume 261 of *LIPIcs*, pages 136:1–136:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.
[60] Amr Sabry, Benoît Valiron, and Juliana Kaizer Vizzotto. From symmetric pattern-matching to quantum control. In *Foundations of Software Science and Computation Structures FOSSACS*, pages 348–364. Springer, 2018.
[61] PETER SELINGER. Control categories and duality: on the categorical semantics of the lambda-mu calculus. *Mathematical Structures in Computer Science*, 11(2):207–260, 2001.
[62] Peter Selinger and Benoît Valiron. Quantum Lambda Calculus. In Simon Gay and Ian Mackie, editors, *Semantic Techniques in Quantum Computation*, pages 135–172. Cambridge University Press, 1 edition, November 2009.
[63] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. *The Journal of Logic Programming*, 29(1):17–64, October 1996.
[64] Dario Stein and Sam Staton. Compositional semantics for probabilistic programs with exact conditioning. In *36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021*, pages 1–13. IEEE, 2021.
[65] L. Sterling and E. Shapiro. *The Art of Prolog*. MIT Press, Cambridge, MA, 1986.
[66] Alfred Tarski. On the calculus of relations. *Journal of Symbolic Logic*, 6(3):73–89, 1941.
[67] Takeshi Tsukada and Kazuyuki Asada. Linear-Algebraic Models of Linear Logic as Categories of Modules over Sigma-Semirings, April 2022. arXiv:2204.10589 [cs].
[68] Lionel Vaux. The algebraic lambda calculus. *Math. Struct. Comput. Sci.*, 19(5):1029–1059, 2009.

A PROOFS FOR SECTION 2: THE RELATIONAL MACHINE CALCULUS

Proposition A.1 (Proposition 2.5 restatement). *Definition 2.4 makes* $(- \Downarrow -)$ *a total function.*

Proof. For each rule defining $S_A \Downarrow M$ the following measure on M decreases:

$$(\mathsf{K_nesting}, \mathsf{new}, \mathsf{var}, \mathsf{const}, \mathsf{size})$$

where K_nesting is the nesting depth of Kleene-star constructs in M; new is the number of $\exists$ constructs in M; var is the number of distinct free variables in M; const is the number of first-order function symbols in M; and size is the size of M.

The one non-trivial case of Definition 2.4 is $S_A \Downarrow M\,;N$, where we need to show that σN has a smaller measure than $M\,;N$ for every (T_A, σ) in $S_A \Downarrow M$. There are three cases. If σ is empty, then size decreases, while the first four measures don't increase. If M contains an $\exists$, then new decreases from $M\,;N$ to σN. For the remaining case, we need the following two observations about σ which are immediate by induction on $(- \Downarrow -)$: the domain of σ and the free variables in its range are disjoint, and if there are no $\exists$ in M, then the free variables in the range of σ are a subset of those of M. With these observations it follows that var reduces from $M\,;N$ to N in the case where M has no $\exists$ and σ is non-empty. □

Proposition A.2 (Proposition 2.6 restatement). *For every memory* S_A*, term* M *and continuation* K*, there is a bijection between the elements* (T_A, σ) *of* $(S_A \Downarrow M)$ *and successful runs*

$$\frac{(\ S_A\ ,\ M\ ,\ \ K\ )}{(\ T_A\ ,\ \star\ ,\ \sigma K\ )}\ .$$

Proof. The bijection is given by annotating each run with a *trace*, a word w in $\{0,1\}^*$ that represents the choice made at each branch. □

B PROOFS FOR SECTION 3: ENCODING COMPUTATIONAL MODELS, I

Proposition B.1 (Proposition 3.2 restatement). *For a set of formal equations over algebraic terms* E*,*

$$\varepsilon \Downarrow [\![E]\!] = [(\varepsilon, \sigma)]$$

where σ *is a most general unifier for* E*, if one exists, and otherwise* $\varepsilon \Downarrow [\![E]\!] = []$.

Proof. Let $E = \{s_1 \doteq t_1, \ldots, s_n \doteq t_n\}$, which we abbreviate as $\{\vec{s} \doteq \vec{t}\}$. Then

$$\varepsilon \Downarrow [\![E]\!] = \varepsilon \Downarrow [\vec{s}]\,;\langle\vec{t}\rangle = \vec{s} \Downarrow \langle\vec{t}\rangle\ ,$$

which we require to show equal to $[(\varepsilon, \sigma)]$ or $[]$ if no unifier exists. We prove a slightly more general statement, which will prove a useful lemma later: for any memory $S_A \cdot S_a$, and values $\vec{s}$ and $\vec{t}$,

$$S_A \cdot S_a\, \vec{s} \Downarrow a\langle\overleftarrow{t}\rangle = [(\sigma(S_A \cdot S_a), \sigma) \mid \sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t})]\ .$$

Proceed by induction, using the lexicographic ordering of (var, const, size) as the measure, where var is the number of distinct free variables in $\vec{s}$ and $\vec{t}$ together, const is the number of function symbols (including constants) appearing in $\vec{s}$ and size is the length n of the stack $\vec{s}$. The measure is $(0,0,0)$ if and only if the stacks are empty. In this base case, the empty set of equations has mgu $\sigma = \varepsilon$, and we are done. Otherwise, we have a non-empty set of equations $\{\vec{s}s_n \doteq \vec{t}t_n\}$ and, for the inductive case, we proceed by case analysis on the pair (s_n, t_n).

- $(s_n, t_n) \equiv (x, x)$

$$\begin{aligned} &S_A \cdot S_a\, \vec{s}\, x \Downarrow a\langle x\rangle\,;a\langle\overleftarrow{t}\rangle \\ &= S_A \cdot S_a\, \vec{s} \Downarrow a\langle\overleftarrow{t}\rangle \\ &(\text{I.H.}) = [(\sigma(S_A \cdot S_a), \sigma) \mid \sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t})] \end{aligned}$$

  where, to use the inductive hypothesis, we note the overall measure decreases, since size decreases, while const remains the same and var remains the same (or, in fact, decreases if there are no occurrences of x in the remaining equations). To see the output is correct, note

$$\sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t}) \Rightarrow \sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t}, x \doteq x)\ .$$

- $(s_n, t_n) \equiv (u, x),\ x \notin u$

$$\begin{aligned} &S_A \cdot S_a\, \vec{s}u \Downarrow a\langle x\rangle\,;a\langle\overleftarrow{t}\rangle \\ &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \cdot S_a\, \vec{s}u \Downarrow a\langle x\rangle,\ (U_A, v) \in T_A \Downarrow \tau a\langle\overleftarrow{t}\rangle] \\ &= [(U_A, v\{u/x\}) \mid (U_A, v) \in \{u/x\}(S_A \cdot S_a\, \vec{s}) \Downarrow \{u/x\}a\langle\overleftarrow{t}\rangle] \\ &= [(U_A, v\{u/x\}) \mid (U_A, v) \in \{u/x\}(S_A \cdot S_a)\ \{u/x\}\vec{s} \Downarrow a\langle\{u/x\}\overleftarrow{t}\rangle] \\ &(\text{I.H.}) = [(\sigma\{u/x\}(S_A \cdot S_a), \sigma\{u/x\}) \mid \sigma = \mathrm{mgu}(\{u/x\}\vec{s} \doteq \{u/x\}\vec{t})] \end{aligned}$$

  where, to use the inductive hypothesis, we note the measure decreases since var decreases (since x no longer appears free in $\vec{s}$ or $\vec{t}$). To see the output is correct, note that

$$\sigma = \mathrm{mgu}(\{u/x\}\vec{s} \doteq \{u/x\}\vec{t}) \Rightarrow \sigma\{u/x\} = \mathrm{mgu}(\vec{s} \doteq \vec{t}, u \doteq x)\ .$$

- $(s_n, t_n) \equiv (x, u),\ x \notin u$

$$S_A \cdot S_a\,\vec{s}x \Downarrow a\langle u\rangle\,;a\langle\vec{t}\rangle$$
$$= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \cdot S_a\,\vec{s}x \Downarrow a\langle u\rangle,\ (U_A, v) \in T_A \Downarrow \tau a\langle\vec{t}\rangle]$$
$$= [(U_A, v\{u/x\}) \mid (U_A, v) \in \{u/x\}(S_A \cdot S_a\,\vec{s}) \Downarrow \{u/x\}a\langle\vec{t}\rangle]$$
$$= [(U_A, v\{u/x\}) \mid (U_A, v) \in \{u/x\}(S_A \cdot S_a)\ \{u/x\}\vec{s} \Downarrow a\langle\{u/x\}\vec{t}\rangle]$$
$$\text{(I.H.)} = [(\sigma\{u/x\}(S_A \cdot S_a), \sigma\{u/x\}) \mid \sigma = \mathrm{mgu}(\{u/x\}\vec{s} \doteq \{u/x\}\vec{t})]$$

with the argument following exactly as in the previous case.

- $(s_n, t_n) \equiv (f(R), f(T))$

$$S_A \cdot S_a\,\vec{s}\,f(R) \Downarrow a\langle f(T)\rangle\,;a\langle\vec{t}\rangle$$
$$= (S_A \cdot S_a\,R) \Downarrow a\langle T\rangle\,;a\langle\vec{t}\rangle$$
$$\text{(I.H.)} = \{(\sigma(S_A \cdot S_a) \mid \sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t}, R \doteq T))\}$$

where, to use the inductive hypothesis, we note the measure decreases because const decreases, while var remains the same. To see the output is correct, note that

$$\sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t}, R \doteq T) \Rightarrow \sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t}, f(R) \doteq f(T))\ .$$

- $(s_n, t_n) \equiv (f(R), g(T)),\ f \neq g.$

$$S_A \cdot S_a\,\vec{s}\,f(R) \Downarrow a\langle g(T)\rangle\,;a\langle\vec{t}\rangle = \{\}$$
$$= \{(\sigma(S_A \cdot S_a), \sigma) \mid \sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t}, f(R) \doteq g(T))\}$$

since there is no $\mathrm{mgu}(\vec{s} \doteq \vec{t}, f(R) \doteq g(T))$ because $f \neq g$.

- $(s_n, t_n) \equiv (f(R), x),\ x \in R$

$$S_A \cdot S_a\,\vec{s}\,f(R) \Downarrow a\langle x\rangle\,;a\langle\vec{t}\rangle = \{\}$$
$$= \{(\sigma(S_A \cdot S_a), \sigma) \mid \sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t}, f(R) \doteq x)\}$$

since there is no $\mathrm{mgu}(\vec{s'} \doteq \vec{t'}, f(R) \doteq x)$ because $x \in R$.

- $(s_n, t_n) \equiv (x, f(R)),\ x \in R$

$$S_A \cdot S_a\,\vec{s}\,x \Downarrow a\langle f(R)\rangle\,;a\langle\vec{t}\rangle = \{\}$$
$$= \{(\sigma(S_A \cdot S_a), \sigma) \mid \sigma = \mathrm{mgu}(\vec{s} \doteq \vec{t}, x \doteq f(R))\}$$

since there is no $\mathrm{mgu}(\vec{s} \doteq \vec{t}, x \doteq f(R))$ because $x \in R$. □

Definition B.2 (Prolog abstract interpreter). We define a version of the top-down abstract interpreter for Prolog, taken from [65].

Input: A query $?- A.$ and program L
Output: An instance of A that is a logical consequence of L, otherwise *fail*
$R \leftarrow [A]$
while $R \neq \emptyset$ **do**
  pop a goal B' from R
  if there exists a clause $B :- B_1 \ldots B_m$ in L such that B and B' unify with mgu σ **then**
    assign fresh variables to $B :- B_1 \ldots B_m$
    $R \leftarrow$ tail $R : B_1 : \cdots : B_m$
    apply σ to A and each element of R
  else
    exit while loop
  end if
end while

if $R = \emptyset$ **then**
  output A
else
  output *fail*
end if

Proposition B.3 (Proposition 3.3 restatement). *Given an given input query Q and program L, the Prolog abstract interpreter [65] outputs the instance Q' if and only if*

$$\frac{(\ Q\ ,\ [\![(Q, L)]\!]\ ,\ \varepsilon\ )}{(\ Q'\ ,\ \ \ \ \ \ \ \ \star\ ,\ \varepsilon\ )}$$

Proof. We consider the interpreter of Definition B.2. We begin by noting

$$(S, \sigma) \in Q\,END \Downarrow [A]\,;\left(\sum_{i=1}^{n} [\![C_i]\!]\right)^{\star};\langle END\rangle \iff$$
$$(S, \sigma) \in \bigsqcup_{C \in \mathcal{C}} (Q\,END\,A \Downarrow C)$$

where $\mathcal{C}$ is the set of all finite sequential compositions of embeddings of clauses in L, postcomposed with $\langle END\rangle$. Suppose C is such a term, and that A and $P(\vec{t})$ have mgu θ. Then by Proposition 3.2, we have

$$\frac{(\ S\,P(\vec{t})\ \ \ \ \ \ \ \ \ \ \ \ ,\ \exists\vec{x}.\ (\langle A\rangle\,;[A_n]\,;\ldots\,;[A_1])\ ,\ C\ )}{(\ \sigma(S\,A_n \ldots A_1)\ ,\ *\ ,\ C\ )}$$

By induction, each state in a successful run of $(\ Q\,END\,A\ ,\ C\ ,\ e\ )$ must eventually reach a state of the form $(\ Q'\,END\,B_m \ldots B_1\ ,\ \star\ ,\ C'\ )$, where $C' \in \mathcal{C}$. Mapping states of this form to resolvant stacks $\{B_m, \ldots, B_1\}$ gives a straightforward bijection between runs ending with $(\ Q'\ ,\ \star\ ,\ \varepsilon\ )$ in the execution of $(\ Q\ ,\ [\![(Q, L)]\!]\ ,\ \varepsilon\ )$ on the branching machine and successful instances of the abstract interpreter over all possible scheduling policies. □

C PROOFS FOR SECTION 4: EQUATIONAL THEORY AND REDUCTION

C.1 Proofs for Section 4.1: The Equational Theory

Proposition C.1 (Proposition 4.4 restatement). *Machine equivalence is a congruence.*

Proof. It is immediate from the well-definedness of $\lfloor - \Downarrow - \rfloor$ that $(\sim)$ is an equivalence relation. It remains to show that $(\sim)$ is closed under contexts. We in fact prove the following, stronger, statement involving multisets rather than sets: for every context K,

$$\forall S_A.\ S_A \Downarrow M = S_A \Downarrow N \ \Rightarrow\ \forall S_A.\ S_A \Downarrow K\{M\} = S_A \Downarrow K\{N\}\ .$$

We proceed by induction on the size of K. The base case, given by the trivial context, is immediate. The inductive cases follow, assuming that $M \sim M'$ and $N \sim N'$.

- $M\,;N \sim M'\,;N'$

$$S_A \Downarrow M\,;N = [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau N]$$
$$\text{(I.H.)} = [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M',\ (U_A, v) \in T_A \Downarrow \tau N']$$
$$= S_A \Downarrow M'\,;N'$$

- $M^* \sim M'^*$

$$S_A \Downarrow M^* = \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M^n = \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M'^n = S_A \Downarrow M'^*$$

where the second equality holds by repeatedly applying the previous case of sequential composition to derive $M^n \sim M'^n$.

- $M + N \sim M' + N'$

$$\begin{aligned} S_A \Downarrow M + N &= (S_A \Downarrow M) \sqcup (S_A \Downarrow N) \\ \text{(I.H.)} &= (S_A \Downarrow M') \sqcup S_A \Downarrow N') = S_A \Downarrow M' + N' \end{aligned}$$

- $\exists x.\, M \sim \exists x.\, M'$

$$\begin{aligned} S_A \Downarrow \exists x.\, M &= [(T_A, \tau \setminus x) \mid (T_A, \tau) \in S_A \Downarrow \{y/x\}M] \quad (y \text{ fresh}) \\ \text{(I.H.)} &= [(T_A, \tau \setminus x) \mid (T_A, \tau) \in S_A \Downarrow \{y/x\}M'] \\ &= S_A \Downarrow \exists x.\, M' \end{aligned}$$

□

Proposition C.2. *For any terms M and N, we have*

$$\forall S_A.\ \lfloor S_A \Downarrow M \rfloor = \lfloor S_A \Downarrow N \rfloor \iff M \sim N$$

Proof. Let $\vec{x} = \mathrm{dom}(\sigma)$ and $\sigma = \{T_a/\vec{x}\}$ an arbitrary substitution, and let a be a fresh location. Note, a similar proof applies without the use of a fresh location — we use select one for notational convenience. Then the implication is given by

$$\begin{aligned} &\forall S_A.\ \lfloor S_A \Downarrow M \rfloor = \lfloor S_A \Downarrow N \rfloor \implies \\ &\forall S_A.\ \lfloor S_A \Downarrow \exists\vec{x}.\, (a\langle\vec{x}\rangle ; M) \rfloor = \lfloor S_A \Downarrow \exists\vec{x}.\, (a\langle\vec{x}\rangle ; N) \rfloor \implies \\ &\forall S'_A \cdot T_a.\ \lfloor S'_A \cdot T_a \Downarrow \exists\vec{x}.\, (a\langle\vec{x}\rangle ; M) \rfloor = \lfloor S'_A \cdot T_a \Downarrow \exists\vec{x}.\, (a\langle\vec{x}\rangle ; N) \rfloor \end{aligned}$$

where the first implication is given by application of congruence (Propopsition 4.4) given a trivial substitution, the second implication is given by the instantiation of the universal quantifer S'_A is arbitrary but T_a is as above.

$$\begin{aligned} & S'_A \cdot T_a \Downarrow \exists\vec{x}.\, (a\langle\vec{x}\rangle ; M) \\ (y \text{ fresh}) &= [(U_A, v \setminus \vec{y}) \mid (U_A, v) \in S'_A \cdot T_a \Downarrow \{\vec{y}/\vec{x}\}(a\langle\vec{x}\rangle ; M)] \\ &= [(U_A, v \setminus \vec{y}) \mid (U_A, v) \in S_A \cdot T_a \Downarrow a\langle\vec{y}\rangle ; \{\vec{y}/\vec{x}\}M] \\ &= [(V_A, vv' \setminus \vec{y}) \mid (U_A, v') \in S'_A \cdot T_a \Downarrow a\langle\vec{y}\rangle, \\ &\qquad (V_A, v) \in V_A \Downarrow v'\{\vec{y}/\vec{x}\}M] \\ &= [(V_A, vv' \setminus \vec{y}) \mid (U_A, v') \in S'_A \cdot T_a \Downarrow a\langle\vec{y}\rangle, \\ &\qquad (V_A, v) \in V_A \Downarrow v'\{\vec{y}/\vec{x}\}M] \\ &= [(V_A, v\{T_a/\vec{y}\} \setminus \vec{y}) \mid (V_A, v) \in S'_A \Downarrow \{T_a/\vec{y}\}\{\vec{y}/\vec{x}\}M] \\ &= [(V_A, v\{T_a/\vec{y}\} \setminus \vec{y}) \mid (V_A, v) \in S'_A \Downarrow \{T_a/\vec{y}\}\{\vec{y}/\vec{x}\}M] \\ &= [(V_A, v) \mid (V_A, v) \in S'_A \Downarrow \{T_a/\vec{x}\}M] \\ &= S'_A \Downarrow \{T_a/\vec{x}\}M \end{aligned}$$

where the penultimate step assumes the y is globally fresh. A similar calculation shows $S'_A \cdot T_a \Downarrow \exists\vec{x}.\, (a\langle\vec{x}\rangle ; N) = S'_A \Downarrow \{T_a/\vec{x}\}N$. Thus, we can conclude that $\forall S_A.\ \lfloor S_A \Downarrow M \rfloor = \lfloor S_A \Downarrow N \rfloor$ implies that $\forall S_A.\ S_A \Downarrow \{T_a/\vec{x}\}M = S_A \Downarrow \{T_a/\vec{x}\}N$, *i.e.*, that $M \sim N$. The converse is immediate by choosing σ to be the empty substitution. □

We prove the result separately β cases and the rest of the equational theory.

Lemma C.3. *The equational theory is sound with respect to machine equivalence, for all equations except β:*

$$M = N \implies M \sim N\,.$$

Proof. Because machine equivalence is a congruence (Lemma 4.4), it suffices to check the statement for each generator of the equational theory. We in fact prove a stronger consequent, $\forall S_A.\ S_A \Downarrow \sigma M = S_A \Downarrow \sigma N$, where possible (*i.e.*, except for the case of idempotence). We further use the proposition that $\lfloor S_A \Downarrow M \rfloor = \lfloor S_A \Downarrow N \rfloor$ implies $M \sim N$, to simplify the form of the statement to be proven. This claim is itself proved in Proposition C.2.

- $M\,;(N\,;P) \sim (M\,;N)\,;P$

$$\begin{aligned} & S_A \Downarrow M\,;(N\,;P) \\ &= [(V_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (V_A, v) \in T_A \Downarrow \tau(N\,;P)] \\ &= [(V_A, v'v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau N, \\ &\qquad (V_A, v') \in U_A \Downarrow v\tau P] \\ &= [(V_A, v'v') \mid (U_A, v') \in S_A \Downarrow M\,;N,\ (V_A, v') \in U_A \Downarrow v'P] \\ &= S_A \Downarrow (M\,;N)\,;P \end{aligned}$$

- $\star\,;M \sim M$

$$\begin{aligned} S_A \Downarrow \star\,;M &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow \star,\ (U_A, v) \in T_A \Downarrow \tau M] \\ &= [(U_A, v) \mid (U_A, v) \in S_A \Downarrow M] = S_A \Downarrow \sigma M \end{aligned}$$

- $M\,;\star \sim M$

$$\begin{aligned} S_A \Downarrow M\,;\star &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau\star] \\ &= [(U_A, v) \mid (U_A, v) \in S_A \Downarrow M] = S_A \Downarrow M \end{aligned}$$

- $(M + N) + P \sim M + (N + P)$

$$\begin{aligned} S_A \Downarrow (M + N) + P &= (S_A \Downarrow (M + N)) \sqcup (S_A \Downarrow P) \\ &= (S_A \Downarrow M) \sqcup (S_A \Downarrow N) \sqcup (S_A \Downarrow P) \\ &= (S_A \Downarrow M) \sqcup (S_A \Downarrow (N + P)) \\ &= S_A \Downarrow (M + (N + P)) \end{aligned}$$

- $M + 0 \sim M$

$$S_A \Downarrow M + 0 = (S_A \Downarrow M) \sqcup (S_A \Downarrow 0) = (S_A \Downarrow M) \sqcup [\,] = S_A \Downarrow M$$

- $M + N \sim N + M$

$$S_A \Downarrow M + N = (S_A \Downarrow M) \sqcup (S_A \Downarrow N) = S_A \Downarrow N + M$$

- $M + M \sim M$

$$\lfloor S_A \Downarrow M + M \rfloor = \lfloor S_A \Downarrow M \rfloor \cup \lfloor S_A \Downarrow M \rfloor = \lfloor S_A \Downarrow M \rfloor$$

- $M\,;(N + P) \sim M\,;N + M\,;P$

$$\begin{aligned} & S_A \Downarrow M\,;(N + P) \\ &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau(N + P)] \\ &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau N] \sqcup \\ &\quad [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau P] \\ &= (S_A \Downarrow M\,;N) \sqcup (S_A \Downarrow M\,;P) = S_A \Downarrow M\,;N + M\,;P \end{aligned}$$

- $(M + N)\,;P \sim M\,;P + M\,;P$

$$\begin{aligned} & S_A \Downarrow (M + N)\,;P \\ &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M + N,\ (U_A, v) \in T_A \Downarrow \tau P] \\ &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau P] \sqcup \\ &\quad [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow N,\ (U_A, v) \in T_A \Downarrow \tau P] \\ &= (S_A \Downarrow M\,;P) \sqcup (S_A \Downarrow N\,;P) = S_A \Downarrow M\,;P + N\,;P \end{aligned}$$

- $0\,;M \sim 0$

$$\begin{aligned} S_A \Downarrow 0\,;M &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow 0,\ (U_A, v) \in T_A \Downarrow M] \\ &= [\,] = S_A \Downarrow 0 \end{aligned}$$

- $M\,;0 \sim 0$

$$S_A \Downarrow M\,;0 = [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow 0] = [\,] = S_A \Downarrow 0$$

- $M^* \sim \star + M\,;M^*$

$$S_A \Downarrow M^* = \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M^n$$
$$= (S_A \Downarrow M^0) \sqcup \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M^{n+1} = (S_A \Downarrow \star) \sqcup \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M\,;M^n$$
$$= (S_A \Downarrow \star) \sqcup \bigsqcup_{n\in\mathbb{N}} [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau M^n]$$
$$= (S_A \Downarrow \star) \sqcup [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in \bigsqcup_{n\in\mathbb{N}} T_A \Downarrow \tau M^n]$$
$$= (S_A \Downarrow \star) \sqcup [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau M^*]$$
$$= (S_A \Downarrow \star) \sqcup (S_A \Downarrow M\,;M^*)$$
$$= S_A \Downarrow \star + M\,;M^*$$

- $M^* \sim \star + M^*\,;M$

$$S_A \Downarrow M^* = \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M^n$$
$$= (S_A \Downarrow M^0) \sqcup \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M^{n+1}\ (*) = (S_A \Downarrow \star) \sqcup \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M^n\,;M$$
$$= (S_A \Downarrow \star) \sqcup \bigsqcup_{n\in\mathbb{N}} [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M^n,\ (U_A, v) \in T_A \Downarrow \tau M]$$
$$= (S_A \Downarrow \star) \sqcup [(U_A, v\tau) \mid (T_A, \tau) \in \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M^n,\ (U_A, v) \in T_A \Downarrow \tau M]$$
$$= (S_A \Downarrow \star) \sqcup [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M^*,\ (U_A, v) \in T_A \Downarrow \tau M]$$
$$= (S_A \Downarrow \star) \sqcup (S_A \Downarrow M^*\,;M)$$
$$= S_A \Downarrow \star + M^*\,;M$$

where $(*)$ is achieved from inductively applying associativity, proved in a previous case, to the definition $M^{n+1} = M\,;M^n$ to see $M^{n+1} = M^n\,;M$.

- $M\,;N \lesssim N \Rightarrow M^*\,;N \lesssim N$

That is to say, we must prove:

$$\forall S_A.\ S_A \Downarrow M\,;N \sqsubseteq S_A \Downarrow N \Rightarrow \forall S_A.\ S_A \Downarrow M^*\,;N \sqsubseteq S_A \Downarrow N$$

Call the premise $(*)$. Unfolding the definition of $-\Downarrow-$, we have that $S_A \Downarrow M^*\,;N = \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M^n\,;N$. So it suffices to prove, for each $n \in \mathbb{N}$, that $(*)$ implies $S_A \Downarrow M^n\,;N \sqsubseteq S_A \Downarrow N$. The base case is immediate, and the inductive case is given below:

$$S_A \Downarrow M^{n+1}\,;N = S_A \Downarrow M\,;M^n\,;N$$
$$= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau(M^n\,;N)]$$
$$(\text{I.H.}) \sqsubseteq [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau N]$$
$$= S_A \Downarrow M\,;N\ (*) \sqsubseteq S_A \Downarrow N$$

□

- $M\,;N \lesssim M \Rightarrow M\,;N^* \lesssim M$

That is to say, we must prove:

$$\forall S_A.\ S_A \Downarrow M\,;N \sqsubseteq S_A \Downarrow M \Rightarrow \forall S_A.\ S_A \Downarrow M\,;N^* \sqsubseteq S_A \Downarrow M$$

Call the premise $(*)$. Unfolding the definition of $-\Downarrow-$, we have that $S_A \Downarrow M\,;N^* = \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow M\,;N^n$. So it suffices to prove, for each $n \in \mathbb{N}$, that $(*)$ implies $S_A \Downarrow M\,;N^n \sqsubseteq S_A \Downarrow M$. The base case is immediate, and the inductive case is given below:

$$S_A \Downarrow M\,;N^{n+1} = S_A \Downarrow M\,;N^n\,;N$$
$$= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M\,;N^n,\ (U_A, v) \in T_A \Downarrow \tau N]$$
$$(\text{I.H.}) \sqsubseteq [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau N]$$
$$= S_A \Downarrow M\,;N\ (*) \sqsubseteq S_A \Downarrow N$$

Note, the first equality is achieved by inductively applying associativity, proved in a previous case, to the definition $N^{n+1} = N\,;N^n$ to see $N^{n+1} = N^n\,;N$.

- $\exists x.\star \sim \star$

$$S_A \Downarrow \exists x.\star$$
$$(y \text{ fresh}) = [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \Downarrow \{y/x\}\star]$$
$$= [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \Downarrow \star]$$
$$= [(S_A, \varepsilon)] = S_A \Downarrow \star$$

- $\exists x.\,(M\,;N) \sim \exists x.\,M\,;N,\ x \notin \mathsf{fv}(N)$

$$S_A \Downarrow \exists x.\,(M\,;N)$$
$$(y \text{ fresh}) = [(U_A, v' \setminus y) \mid (U_A, v') \in S_A \Downarrow \{y/x\}(M\,;N)]$$
$$(x \notin N) = [(U_A, v' \setminus y) \mid (U_A, v') \in S_A \Downarrow \{y/x\}M\,;N]$$
$$= [(U_A, v\tau \setminus y) \mid (T_A, \tau) \in S_A \Downarrow \{y/x\}M,\ (U_A, v) \in T_A \Downarrow \tau N]$$
$$= [(U_A, v(\tau \setminus y)) \mid (T_A, \tau) \in S_A \Downarrow \{y/x\}M,\ (U_A, v) \in T_A \Downarrow (\tau \setminus y)N]$$
$$= [(U_A, v\tau') \mid (T_A, \tau') \in S_A \Downarrow \exists x.\,M,\ (U_A, v) \in T_A \Downarrow \tau N]$$
$$= S_A \Downarrow \exists x.\,M\,;N$$

using that $\tau N = (\tau \setminus y)N$ since $y \notin N$, and the assumption that y is globally fresh and thus need not be passed to any further continuation.

- $\exists x.\,(M\,;N) \sim M\,;\exists x.\,N,\ x \notin \mathsf{fv}(M)$

$$S_A \Downarrow \exists x.\,(M\,;N)$$
$$(y \text{ fresh}) = [(U_A, v' \setminus y) \mid (U_A, v') \in S_A \Downarrow \{y/x\}(M\,;N)]$$
$$(x \notin M) = [(U_A, v' \setminus y) \mid (U_A, v') \in S_A \Downarrow M\,;\{y/x\}N]$$
$$= [(U_A, v\tau \setminus y) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau\{y/x\}N]$$
$$= [(U_A, (v \setminus y)\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \{y/x\}(\tau \setminus x)N]$$
$$= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \exists x.\,(\tau \setminus x)N]$$
$$= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \Downarrow M,\ (U_A, v) \in T_A \Downarrow \tau\exists x.\,N]$$
$$S_A \Downarrow M\,;\exists x.\,N$$

using that y is not in the domain or range of τ, by freshness.

- $\exists x.\,0 \sim 0$

$$S_A \Downarrow \exists x.\,0 = [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \Downarrow \{y/x\}0] = [\,] = S_A \Downarrow 0$$

- Case: $\exists x.\,(M + N) = \exists x.\,M + \exists x.\,N$

$$\begin{aligned} S_A \Downarrow \exists x.\,(M+N) &= [(T_A, \tau \setminus y) \mid (T_A,\tau) \in S_A \Downarrow \{y/x\}(M+N)] \\ &= [(T_A, \tau \setminus y) \mid (T_A,\tau) \in S_A \Downarrow \{y/x\}M] \sqcup \\ &\quad\ [(T_A, \tau \setminus y) \mid (T_A,\tau) \in S_A \Downarrow \{y/x\}N] \\ &= (S_A \Downarrow \exists x.\,M) \sqcup (S_A \Downarrow \exists x.\,N) \\ &= S_A \Downarrow \exists x.\,M + \exists x.\,N \end{aligned}$$

- $\exists x.\,[t]a \sim [t]a \quad (x \notin t)$

$$\begin{aligned} S_A \cdot S_a \Downarrow \exists x.\,[t]a &= [(T_A, \tau \setminus y) \mid (T_A,\tau) \in S_A \cdot S_a \Downarrow \{y/x\}[t]a] \\ (x \notin t) &= [(T_A, \tau \setminus y) \mid (T_A,\tau) \in S_A \cdot S_a \Downarrow [t]a] \\ &= [(S_A \cdot S_a\, t, \varepsilon)] \\ &= S_A \cdot S_a \Downarrow [t]a \end{aligned}$$

- $\exists x.\,a\langle t\rangle \sim a\langle t\rangle \ (x \notin t)$
  In the case $S_A \equiv S'_A \cdot S_a\, s$:

$$\begin{aligned} & S_A \Downarrow \exists x.\,a\langle t\rangle \\ (y \text{ fresh}) &= [(T_A, \tau \setminus y) \mid (T_A,\tau) \in S'_A \cdot S'_a\, s \Downarrow \{y/x\}a\langle t\rangle] \\ &= [(T_A, \tau \setminus y) \mid (T_A,\tau) \in S'_A \cdot S'_a\, s \Downarrow a\langle t\rangle] \\ (\text{Prop 3.2}) &= [(\sigma(S'_A \cdot S_a), \sigma \setminus y) \mid \sigma = \mathrm{mgu}(s \doteq t)] \\ y \notin \mathrm{dom}(\sigma) &= [(\sigma(S'_A \cdot S_a), \sigma) \mid \sigma = \mathrm{mgu}(s \doteq t)] \\ (\text{Prop 3.2}) &= S_A \Downarrow a\langle t\rangle \end{aligned}$$

  and otherwise: $S_A \Downarrow \exists x.\,\langle t\rangle = [] = S_A \Downarrow a\langle t\rangle$.
- $\exists x.\,\exists y.\,M \sim \exists y.\,\exists x.\,M$

$$\begin{aligned} & S_A \Downarrow \exists x.\,\exists y.\,M \\ (u \text{ fresh}) &= [(T_A, \tau \setminus u) \mid (T_A,\tau) \in S_A \Downarrow \{u/x\}\exists y.\,M] \\ &= [(T_A, \tau \setminus u) \mid (T_A,\tau) \in S_A \Downarrow \exists y.\,\{u/x\}M] \\ (v \text{ fresh}) &= [(T_A, \tau \setminus u \setminus v) \mid (T_A,\tau) \in S_A \Downarrow \{v/y\}\{u/x\}M] \\ &= [(T_A, \tau \setminus v \setminus u) \mid (T_A,\tau) \in S_A \Downarrow \{u/x\}\{v/y\}M] \\ &= [(T_A, \tau \setminus v) \mid (T_A,\tau) \in S_A \Downarrow \exists x.\,\{v/y\}M] \\ &= [(T_A, \tau \setminus v) \mid (T_A,\tau) \in S_A \Downarrow \{v/y\}\exists x.\,M] \\ &= S_A \Downarrow \exists y.\,\exists x.\,M \end{aligned}$$

- $\exists x.\,(a\langle x\rangle\,;[x]a) \lesssim \star$
  In the case $S_A \equiv S'_A \cdot S'_a\, t$, we have:

$$\begin{aligned} & S'_A \cdot S'_a\, t \Downarrow \exists x.\,a\langle x\rangle\,;[x]a \\ (y \text{ fresh}) &= [(T_A, \tau \setminus y) \mid (T_A,\tau) \in S'_A \cdot S'_a\, t \Downarrow a\langle y\rangle\,;[y]a] \\ (y \notin t) &= [(T_A, \tau'\{t/y\} \setminus y) \mid (T_A,\tau') \in \{t/y\}(S'_A \cdot S'_a) \Downarrow [t]a] \\ (y \notin S_A \cdot S_a) &= [(T_A, \tau' \setminus y) \mid S'_A \cdot S'_a \Downarrow [t]a] \\ &= [(S'_A \cdot S'_a\, t, \varepsilon)] \\ &= S'_A \cdot S'_a\, t \Downarrow \star \end{aligned}$$

  but otherwise $S_A \Downarrow \exists x.\,a\langle x\rangle\,;[x]a = [] \sqsubseteq S_A \cdot S_a\, t \Downarrow \star$.
- $[f(R)]a\,;a\langle f(U)\rangle \sim [R]a\,;a\langle U\rangle$

$$\begin{aligned} S_A \cdot S_a \Downarrow [f(R)]a\,;a\langle f(U)\rangle &= S_A \cdot S_a\, f(R) \Downarrow a\langle f(U)\rangle \\ &= S_A \cdot S_a\, R \Downarrow a\langle U\rangle \\ (*) &= S_A \cdot S_a \Downarrow [R]a\,;a\langle U\rangle \end{aligned}$$

  Note the final step $(*)$ is proven by repeated unfoldings of the "push" and sequencing rules.

- $[f(R)]a\,;a\langle g(U)\rangle \sim 0$

$$\begin{aligned} S_A \cdot S_a \Downarrow [f(R)]a\,;a\langle g(U)\rangle &= S_A \cdot S_a\, f(R) \Downarrow a\langle g(U)\rangle \\ &= [\,] = S_A \cdot S_a \Downarrow 0 \end{aligned}$$

- $[f(S)]a\,;a\langle x\rangle \sim 0 \ (x \in S)$

$$\begin{aligned} S_A \cdot S_a \Downarrow [f(S)]a\,;a\langle x\rangle &= S_A \cdot S_a\, f(S) \Downarrow a\langle x\rangle \\ &= [\,] = S_A \cdot S_a \Downarrow 0 \end{aligned}$$

- $[x]a\,;a\langle f(S)\rangle \sim 0 \ (x \in S)$

$$\begin{aligned} S_A \cdot S_a \Downarrow [x]a\,;a\langle f(S)\rangle &= S_A \cdot S_a\, x \Downarrow a\langle f(S)\rangle \\ &= [\,] = S_A \cdot S_a \Downarrow 0 \end{aligned}$$

- $[t]a\,;b\langle u\rangle \sim b\langle u\rangle\,;[t]a \ (a \neq b)$
  In the case $S_A \equiv S'_A \cdot S_a \cdot S_b\, s$

$$S'_A \cdot S_a \cdot S_b\, s \Downarrow [t]a\,;b\langle u\rangle = S'_A \cdot S_a\, t \cdot S_b\, s \Downarrow b\langle u\rangle$$

$$\begin{aligned} (\text{Prop 3.2}) &= [(\sigma(S'_A \cdot S_a\, t \cdot S_b), \sigma) \mid \sigma = \mathrm{mgu}(s \doteq u)] \\ &= [(\sigma U_A, \sigma) \mid (U_A, v) \in \sigma(S'_A \cdot S_a \cdot S_b) \Downarrow \sigma[t]a, \\ &\qquad \sigma = \mathrm{mgu}(s \doteq u)] \\ (\text{Prop 3.2}) &= S'_A \cdot S_a \cdot S_b\, s \Downarrow b\langle u\rangle\,;[t]a \end{aligned}$$

  but otherwise $S_A \Downarrow [t]a\,;b\langle u\rangle = [\,] = S_A \Downarrow b\langle u\rangle\,;[t]a$.

Theorem C.4 (Theorem 4.5 restatement). *The equational theory is sound with respect to machine equivalence:*

$$M = N \implies M \sim N\,.$$

Proof. Because machine equivalence is a congruence (Lemma 4.4), it suffices to check the statement for each generator of the equational theory. We in fact prove a stronger consequent, $\forall S_A.\, S_A \Downarrow \sigma M = S_A \Downarrow \sigma N$. We further use the proposition that $\lfloor S_A \Downarrow M \rfloor = \lfloor S_A \Downarrow N \rfloor$ implies $M \sim N$, to simplify the form of the statement to be proven. This claim is itself proved in Proposition C.2. Following Lemma C.3, it remains only to prove that for any terms M and N, and values t such that $x \notin t$, we have the following.

$$\exists x.\,(N\,;[t]a\,;a\langle x\rangle\,;M) \sim \{t/x\}(N\,;M)$$
$$\exists x.\,(N\,;[x]a\,;a\langle t\rangle\,;M) \sim \{t/x\}(N\,;M)$$

We first require to prove the following auxillary statement. For any term M and value t such that $x \notin t$, we have that

$$M\,;[t]a\,;a\langle x\rangle \sim [t]a\,;a\langle x\rangle\,;M\,.$$

We proceed by induction on the size of M. We will make use of the fact that $M = N$ implies $M \sim N$ from Lemma C.3.

- $M \equiv \star$

$$\star\,;[t]a\,;a\langle x\rangle = [t]a\,;a\langle x\rangle = [t]a\,;a\langle x\rangle\,;\star$$

- $M \equiv M_1\,;M_2$

$$\begin{aligned} M_1\,;M_2\,;[t]a\,;a\langle x\rangle \ (\text{I.H.}) &= M_1\,;[t]a\,;a\langle x\rangle\,;M_2 \\ (\text{I.H.}) &= [t]a\,;a\langle x\rangle\,;M_1\,;M_2 \end{aligned}$$

- $M \equiv 0$

$$0\,;[t]a\,;a\langle x\rangle = 0 = [t]a\,;a\langle x\rangle\,;0$$

- $M \equiv M_1 + M_2$

$$\begin{aligned}(M_1 + M_2)\,;[t]a\,;a\langle x\rangle &= M_1\,;[t]a\,;a\langle x\rangle + M_2\,;[t]a\,;a\langle x\rangle\\ \text{(I.H.)} &= [t]a\,;a\langle x\rangle\,;M_1 + [t]a\,;a\langle x\rangle\,;M_2\\ &= [t]a\,;a\langle x\rangle\,;(M_1 + M_2)\end{aligned}$$

- $M \equiv N^*$

$$S_A \Downarrow N^*\,;[t]\,;\langle x\rangle = \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow N^n\,;[t]\,;\langle x\rangle$$
$$(*) = \bigsqcup_{n\in\mathbb{N}} S_A \Downarrow [t]\,;\langle x\rangle\,;N^n = S_A \Downarrow [t]\,;\langle x\rangle\,;N^*$$

The equality $(*)$ is derived from n applications of the inductive hypothesis in the nth component of the sum.

- $M \equiv \exists y.\, M'$

$$\begin{aligned}(\exists y.\, M)\,;[t]a\,;a\langle x\rangle &=_\nu \exists y.\,(M\,;[t]a\,;a\langle x\rangle)\\ \text{(I.H.)} &= \exists y.\,([t]a\,;a\langle x\rangle\,;M)\\ &=_\nu [t]a\,;a\langle x\rangle\,;\exists y.\, M\end{aligned}$$

- $M \equiv [s]a$

$$\begin{aligned}&S_A \cdot S_a \Downarrow [s]a\,;[t]a\,;a\langle x\rangle\\ &= S_A \cdot S_a\, s \Downarrow [t]a\,;a\langle x\rangle\\ &= S_A \cdot S_a\, s\, t \Downarrow a\langle x\rangle\\ &= (\{t/x\}(S_A \cdot S_a\, s), \{t/x\})\\ &= [(U_A, v\{t/x\}) \mid (U_A, v) \in \{t/x\}(S_A \cdot S_a) \Downarrow [\{t/x\}s]a]\\ &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \cdot S_a\, t \Downarrow a\langle x\rangle,\\ &\qquad (U_A, v) \in T_A \Downarrow \tau[s]a]\\ &= S_A \cdot S_a\, t \Downarrow a\langle x\rangle\,;[s]a\\ &= S_A \cdot S_a \Downarrow [t]a\,;a\langle x\rangle\,;[s]a\end{aligned}$$

- $M \equiv [s]b$

$$[s]b\,;[t]a\,;a\langle x\rangle = [t]a\,;a\langle x\rangle\,;[s]b$$

Achieved by application of the Permutation Lemma 4.2.

- $M \equiv a\langle s\rangle$

$$\begin{aligned}&S_A \cdot S_a\, u \Downarrow a\langle s\rangle\,;[t]a\,;a\langle x\rangle\\ &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \cdot S_a\, u \Downarrow a\langle s\rangle\\ &\qquad (U_A, v) \in T_A \Downarrow \tau([t]a\,;a\langle x\rangle)]\\ &= [(U_A, v\sigma \mid (U_A, v) \in \sigma(S_A \cdot S_a) \Downarrow \sigma([t]a\,;a\langle x\rangle),\ \mid \sigma = \mathsf{mgu}(s \doteq u)]\\ &= [(U_A, v\sigma \mid (U_A, v) \in \sigma(S_A \cdot S_a)\, \sigma t \Downarrow a\langle \sigma x\rangle) \mid \sigma = \mathsf{mgu}(s \doteq u)]\\ &= [(\rho\sigma(S_A \cdot S_a), \rho\sigma) \mid \rho = \mathsf{mgu}(\sigma t \doteq \sigma x), \sigma = \mathsf{mgu}(s \doteq u)]\\ (*) &= [(\mu\{t/x\}(S_A \cdot S_a \mu\{t/x\}) \mid \mu = \mathsf{mgu}(u \doteq \{t/x\}s)]\\ &= [(U_A, v\{t/x\}) \mid (U_A, v) \in \{t/x\}(S_A \cdot S_a\, u) \Downarrow a\langle\{t/x\}s\rangle]\\ &= [(U_A, v\tau) \mid (T_A, \tau) \in S_A \cdot S_a\, u\, t \Downarrow a\langle x\rangle,\\ &\qquad (U_A, v) \in T_A \Downarrow \tau a\langle s\rangle]\\ &= S_A \cdot S_a\, u\, t \Downarrow a\langle x\rangle\,;a\langle s\rangle = S_A \cdot S_a\, u \Downarrow [t]a\,;a\langle x\rangle\,;a\langle s\rangle\end{aligned}$$

The equality $(*)$ is given by noting that

$$\rho\sigma = \mathsf{mgu}(s \doteq u, t \doteq x) = \mu\{t/x\}\,.$$

- $M \equiv b\langle s\rangle$

$$b\langle s\rangle\,;[t]a\,;a\langle x\rangle = [t]a\,;a\langle x\rangle\,;b\langle s\rangle$$

Achieved by application of the Permutation Lemma 4.2.

Using the previous result, $(*)$, we can now prove the statement.

- $\exists x.\,(N\,;[t]a\,;a\langle x\rangle\,;M) \sim \{t/x\}(N\,;M)$ $(x \notin t)$
By application of $(*)$, it suffices to prove that

$$\exists x.\,([t]a\,;a\langle x\rangle\,;M) \sim \{t/x\}M\,,$$

which we do by the following calculations.

$$\begin{aligned}&S_A \cdot S_a \Downarrow \exists x.\,([t]a\,;a\langle x\rangle\,;M)\\ (y \text{ fresh}) &= [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \cdot S_a \Downarrow \{y/x\}([t]a\,;a\langle x\rangle\,;M)]\\ (x \notin t) &= [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \cdot S_a \Downarrow [t]a\,;a\langle y\rangle\,;\{y/x\}M]\\ &= [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \cdot S_a\, t \Downarrow a\langle y\rangle\,;\{y/x\}M]\\ &= [(U_A, v\tau' \setminus y) \mid (T'_A, \tau') \in S_A \cdot S_a\, t \Downarrow a\langle y\rangle\\ &\qquad (U_A, v) \in T'_A \Downarrow \tau'\{y/x\}M]\\ &= [(U_A, v\{t/y\} \setminus y) \mid (U_A, v) \in \{t/y\}(S_A \cdot S_a) \Downarrow \{t/y\}\{y/x\}M]\\ &= S_A \cdot S_a \Downarrow \{t/x\}M\end{aligned}$$

The final equality uses the assumption of the global freshness of y.

- $\exists x.\,(N\,;[x]a\,;a\langle t\rangle\,;M) \sim \{t/x\}(N\,;M)$ $(x \notin t)$ By application of $(*)$, it suffices to prove that

$$\exists x.\,([x]a\,;a\langle t\rangle\,;M) \sim \{t/x\}M\,,$$

which we do by the following calculations.

$$\begin{aligned}&S_A \cdot S_a \Downarrow \exists x.\,([x]a\,;a\langle t\rangle\,;M)\\ (y \text{ fresh}) &= [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \cdot S_a \Downarrow \{y/x\}([x]a\,;a\langle t\rangle\,;M)]\\ (x \notin t) &= [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \cdot S_a \Downarrow [y]a\,;a\langle t\rangle\,;\{y/x\}M]\\ &= [(T_A, \tau \setminus y) \mid (T_A, \tau) \in S_A \cdot S_a\, y \Downarrow a\langle t\rangle\,;\{y/x\}M]\\ &= [(U_A, v\tau' \setminus y) \mid (T'_A, \tau') \in S_A \cdot S_a\, y \Downarrow a\langle t\rangle\\ &\qquad (U_A, v) \in T'_A \Downarrow \tau'\{y/x\}M]\\ &= [(U_A, v\{t/y\} \setminus y) \mid (U_A, v) \in \{t/y\}(S_A \cdot S_a) \Downarrow \{t/y\}\{y/x\}M]\\ &= S_A \cdot S_a \Downarrow \{t/x\}M\end{aligned}$$

The final equality uses the assumption of the global freshness of y. □

C.2 Proofs for Section 4.2: Reduction and Confluence

In this section we prove Theorem 4.7, which states that the reduction relation $\twoheadrightarrow$ is confluent. We recall the corresponding reduction rules given in Definition 4.6 below, adding labels to distinguish between applications of each rule.

$$
\begin{array}{rcll}
\exists x.\, L\{[t]a\,;a\langle x\rangle\,;M\} & \to^{\mathrm{r1}} & \exists x.\, \{t/x\}L\{M\} & (x \notin t)\\
\exists x.\, L\{[x]a\,;a\langle t\rangle\,;M\} & \to^{\mathrm{r2}} & \exists x.\, \{t/x\}L\{M\} & (x \notin t)\\
[x]a\,;a\langle x\rangle\,;M & \to^{\mathrm{r3}} & M & \\
[f(T)]a\,;a\langle f(S)\rangle\,;M & \to^{\mathrm{r4}} & [T]a\,;a\langle S\rangle\,;M & \\
[f(T)]a\,;a\langle g(S)\rangle\,;M & \to^{\mathrm{r5}} & 0 & (f \neq g)\\
[t]a\,;b\langle s\rangle\,;M & \to^{\mathrm{r6}} & b\langle s\rangle\,;[t]a\,;M & (a \neq b)\\
\exists x.\, (M+N) & \to^{\mathrm{r7}} & (\exists x.\, M)+(\exists x.\, N) & \\
M\,;\exists x.\, N & \to^{\mathrm{r8}} & \exists x.\, (M\,;N) & \\
(M+N)\,;P & \to^{\mathrm{r9}} & (M\,;P)+(N\,;P) & (P \neq M+N)\\
P\,;(M+N) & \to^{\mathrm{r10}} & (P\,;M)+(P\,;N) & \\
M+0 & \to^{\mathrm{r11}} & M & \\
0+M & \to^{\mathrm{r12}} & M & \\
\star\,;M & \to^{\mathrm{r13}} & M & \\
\exists x.\, M & \to^{\mathrm{r14}} & M & (x \notin M)\\
M\,;0 & \to^{\mathrm{r15}} & 0 & \\
0\,;M & \to^{\mathrm{r16}} & 0 & \\
M^* & \to^{\mathrm{r17}} & (\star)+(M\,;M^*) & (M \neq 0)
\end{array}
$$

Our proof of confluence makes use of techniques from the field of first-order term rewriting by treating the atomic terms $[t]a$, $a\langle t\rangle$, $\star$ and 0 as constants. Local confluence is shown by analysis of critical pairs, so we observe the star-free fragment is strongly normalising (by the measure given in the proof of Proposition 2.5) and hence confluent by Newman's Lemma. Confluence of the full reduction relation then follows from an application of the Hindley-Rosen Lemma.

We opt to view reduction as a first-order term-rewrite system $\mathcal{R} = \langle \mathcal{T}, \to\rangle$ (or, by abuse of notation, simply $\to$), treating binders $\exists x$ as distinct function symbols for each x and atomic terms $[t]a$, $a\langle t\rangle$, $\star$ and 0 as constants. Barendregt's convention is adopted to simplify our arguments: in particular, existential binders $\exists x.\, M$ in the same linear context are assumed to be distinct. Applications of $\to^{\mathrm{r17}}$ must therefore assign fresh variables appropriately when duplicating. We additionally assume each instance of an algebraic term variable is bound by an existential, and that sequential composition is right-associative with rightmost term $\star$. The star-free fragment of reduction $\to \setminus \to^{\mathrm{r17}}$ is denoted $\to^{\mathrm{T}}$, and the parallel reduction relations induced by $\to^{\mathrm{T}}$ and $\to^{\mathrm{r17}}$ are denoted $\Rightarrow^{\mathrm{T}}$ and $\Rightarrow^{\mathrm{r17}}$, respectively.

Lemma C.5. *$\to$ is locally confluent.*

Proof. We aim to show joinability of all critical pairs of $\to$, from which local confluence follows by the critical pair theorem. We begin with critical pairs arising from peaks ${}^{\mathrm{r1}}\!\leftarrow \cdot \to$ and ${}^{\mathrm{r2}}\!\leftarrow \cdot \to$. Note that substitution over an existential commutes with all rules except $\to^{\mathrm{r1}}$ and $\to^{\mathrm{r2}}$, so in these cases we need only consider critical peaks ${}^{\mathrm{r1}\,\cup\,\mathrm{r2}}\!\leftarrow \cdot \to^{-}$ such that $\to^{-}$ either arises from an outer overlap with the corresponding redex, or destroys/duplicates any part of the internal redex. These are handled for $\to^{\mathrm{r1}}$ below, extending straightforwardly to $\to^{\mathrm{r2}}$.

- Case (${}^{\mathrm{r1}}\!\leftarrow \cdot \to^{\mathrm{r8}}$): $M\,;\exists x.\, L\{[t]a\,;a\langle x\rangle\,;N\}$ gives the following critical pair.

$$\langle l, r\rangle = \langle M\,;\exists x.\, \{t/x\}L\{N\}, \exists x.\, (M\,;L\{[t]a\,;a\langle x\rangle\,;N\})\rangle$$

  By our assumption that $x \notin M$, we have the join:

$$l \to^{\mathrm{r8}} \exists x.\, (M\,;\{t/x\}L\{N\})\ {}^{\mathrm{r1}}\!\leftarrow r$$

- Case (${}^{\mathrm{r1}}\!\leftarrow \cdot \to^{\mathrm{r9}}$): $\exists x.\, L([t]a\,;a\langle x\rangle\,;(M+N))$ gives the following critical pair.

$$\langle l, r\rangle = \langle \exists x.\, \{t/x\}L\{M+N\}, \exists x.\, L\{[t]a\,;((a\langle x\rangle\,;M)+(a\langle x\rangle\,;N))\}\rangle$$

  Join:

$$l \to^{(\mathrm{r9}\,\cup\,\mathrm{r10})^*} \cdot \to^{\mathrm{r7}} (\exists x.\, \{t/x\}L\{M\})+(\exists x.\, \{t/x\}L\{N\})\ {}^{\mathrm{r1}}\!\leftarrow \cdot\ {}^{\mathrm{r1}}\!\leftarrow \cdot\ {}^{\mathrm{r7}}\!\leftarrow \cdot\ {}^{(\mathrm{r9}\,\cup\,\mathrm{r10})^*}\!\leftarrow r$$

- Case (${}^{\mathrm{r1}}\!\leftarrow \cdot \to^{\mathrm{r10}}$): $\exists x.\, L((M+N)\,;[t]a\,;a\langle x\rangle\,;P$ gives the following critical pair.

$$\langle l, r\rangle = \langle \exists x.\, \{t/x\}L\{(M+N)\,;P\}, \exists x.\, L\{((M\,;[t]a)+(N\,;[t]a))\,;a\langle x\rangle\,;P\}\rangle$$

  Join:

$$l \to^{(\mathrm{r9}\,\cup\,\mathrm{r10})^*} \cdot \to^{\mathrm{r7}} (\exists x.\, \{t/x\}L\{M\,;P\})+(\exists x.\, \{t/x\}L\{N\,;P\})\ {}^{\mathrm{r1}}\!\leftarrow \cdot\ {}^{\mathrm{r1}}\!\leftarrow \cdot\ {}^{\mathrm{r7}}\!\leftarrow \cdot\ {}^{(\mathrm{r9}\,\cup\,\mathrm{r10})^*}\!\leftarrow r$$

- Case (${}^{\mathrm{r1}}\!\leftarrow \cdot \to^{\mathrm{r5}\,\cup\,\mathrm{r15}\,\cup\,17}$): joinability follows immediately from the following observation.

$$L\{0\} \to^{(\mathrm{r15}\,\cup\,\mathrm{r16})^*} 0$$

We now look to join critical pairs arising from peaks of the form ${}^{\mathrm{r1}\,\cup\,\mathrm{r2}}\!\leftarrow \cdot \to^{\mathrm{r1}\,\cup\,\mathrm{r2}}$. Note that the order of application of two rewrite steps $\to^{\mathrm{r1}}$ and $\to^{\mathrm{r2}}$ is interchangeable whenever their corresponding binders differ, since their contexts are disjoint. Let $L\{M, N\}$ denote a linear context with two terms at distinct positions identified. Then, for peaks ${}^{\mathrm{r1}}\!\leftarrow \cdot \to^{\mathrm{r1}}$, the remaining critical pairs must of the following form.

$$\langle l, r\rangle = \langle \exists x.\, \{t/x\}L\{M, [s]\,;a\langle t\rangle\,;N\}, \exists x.\, \{s/x\}L\{[t]a\,;a\langle s\rangle\,;M, N\}\rangle$$

Suppose s and t have most general unifier σ. Then, by Proposition 3.2, we have the join

$$l \to^{\mathrm{T}^*} \exists x.\, \sigma\{t/x\}L\{M, N\}\ {}^{\mathrm{T}^*}\!\leftarrow r$$

Otherwise,

$$l \to^{\mathrm{T}^*} 0\ {}^{\mathrm{T}^*}\!\leftarrow r$$

The joinability of peaks ${}^{\mathrm{r1}\,\cup\,\mathrm{r2}}\!\leftarrow \cdot \to^{\mathrm{r1}\,\cup\,\mathrm{r2}}$ follows straightforwardly from the above reasoning. We now construct joins for all remaining critical pairs: that is, those of $\to \setminus(\to^{\mathrm{r1}\,\cup\,\mathrm{r2}})$.

- Case (${}^{\mathrm{r3}}\!\leftarrow \cdot \to^{\mathrm{r8}}$): $[x]a\,;a\langle x\rangle\,;\exists y.\, M$ gives the following critical pair.

$$\langle l, r\rangle = \langle \exists y.\, M, [x]a\,;(\exists y.\, (a\langle x\rangle\,;M))\rangle$$

  Join:

$$l = \exists y.\, M\ {}^{\mathrm{r3}}\!\leftarrow \cdot\ {}^{\mathrm{r8}}\!\leftarrow r$$

- Case (${}^{\mathrm{r3}}\!\leftarrow \cdot \to^{\mathrm{r10}}$): $[x]a\,;a\langle x\rangle\,;(M+N)$ gives the following critical pair.

$$\langle l, r\rangle = \langle M+N, [x]a\,;((a\langle x\rangle\,;M)+(a\langle x\rangle\,;N))\rangle$$

  Join:

$$l = M+N\ {}^{\mathrm{r3}}\!\leftarrow \cdot\ {}^{\mathrm{r3}}\!\leftarrow \cdot\ {}^{\mathrm{r10}}\!\leftarrow r$$

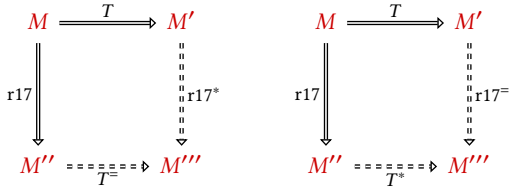
- Case $({}^{\mathrm{r3}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r15}})$: $[x]\,a\,;a\langle x\rangle\,;0$ gives the following critical pair.
$$\langle l, r\rangle = \langle 0, [x]\,a\,;0\rangle$$
Join:
$$l = 0\ {}^{\mathrm{r15}}\!\leftarrow r$$
- Case $({}^{\mathrm{r4}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r8}})$: $[f(T)]\,a\,;a\langle f(S)\rangle\,;\exists x.\,M$ gives the following critical pair.
$$\langle l, r\rangle = \langle [T]\,a\,;a\langle S\rangle\,;\exists x.\,M, [f(T)]\,a\,;(\exists x.\,(a\langle f(S)\rangle\,;M))\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r8}^*} \exists x.\,([T]\,a\,;a\langle S\rangle\,;M)\ {}^{\mathrm{r4}}\!\leftarrow \cdot\ {}^{\mathrm{r8}}\!\leftarrow r$$
- Case $({}^{\mathrm{r4}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r10}})$: $[f(T)]\,a\,;a\langle f(S)\rangle\,;(M+N)$ gives the following critical pair.
$$\langle l, r\rangle = \langle [T]\,a\,;a\langle S\rangle\,;(M+N), [f(T)]\,a\,;((a\langle f(S)\rangle\,;M)+(a\langle f(S)\rangle\,;N))\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r10}} \cdot \twoheadrightarrow^{\mathrm{r10}} ([T]\,a\,;a\langle S\rangle\,;M)+([T]\,a\,;a\langle S\rangle\,;N)\ {}^{\mathrm{r4}}\!\leftarrow\cdot\ {}^{\mathrm{r4}}\!\leftarrow\cdot\ {}^{\mathrm{r10}}\!\leftarrow r$$
- Case $({}^{\mathrm{r4}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r15}})$: $[f(T)]\,a\,;a\langle f(S)\rangle\,;0$ gives the following critical pair.
$$\langle l, r\rangle = \langle [T]\,a\,;a\langle S\rangle\,;0, [f(T)]\,a\,;0\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r15}^*} 0\ {}^{\mathrm{r15}}\!\leftarrow r$$
- Case $({}^{\mathrm{r5}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r8}})$: $[f(T)]\,a\,;a\langle g(S)\rangle\,;\exists x.\,M$ gives the following critical pair.
$$\langle l, r\rangle = \langle 0, [f(T)]\,a\,;(\exists x.\,(a\langle g(S)\rangle\,;M))\rangle$$
Join:
$$l = 0\ {}^{\mathrm{r14}}\!\leftarrow\cdot\ {}^{\mathrm{r5}}\!\leftarrow\cdot\ {}^{\mathrm{r8}}\!\leftarrow r$$
- Case $({}^{\mathrm{r5}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r10}})$: $[f(T)]\,a\,;a\langle g(S)\rangle\,;(M+N)$ gives the following critical pair.
$$\langle l, r\rangle = \langle 0, [f(T)]\,a\,;((a\langle g(S)\rangle\,;M)+(a\langle g(S)\rangle\,;N))\rangle$$
Join:
$$l = 0\ {}^{\mathrm{r11}}\!\leftarrow\cdot\ {}^{\mathrm{r5}}\!\leftarrow\cdot\ {}^{\mathrm{r5}}\!\leftarrow\cdot\ {}^{\mathrm{r10}}\!\leftarrow r$$
- Case $({}^{\mathrm{r5}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r15}})$: $[f(T)]\,a\,;a\langle g(S)\rangle\,;0$ gives the following critical pair.
$$\langle l, r\rangle = \langle 0\,;0, [f(T)]\,a\,;0\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r15}} 0\ {}^{\mathrm{r15}}\!\leftarrow r$$
- Case $({}^{\mathrm{r6}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r8}})$: $[t]\,a\,;b\langle s\rangle\,;\exists x.\,M$ gives the following critical pair.
$$\langle l, r\rangle = \langle b\langle s\rangle\,;[t]\,a\,;\exists x.\,M, b\langle s\rangle\,;(\exists x.\,([t]\,a\,;M))\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r8}} \cdot \twoheadrightarrow^{\mathrm{r8}} \exists x.\,(b\langle s\rangle\,;[t]\,a\,;M)\ {}^{\mathrm{r6}}\!\leftarrow\cdot\ {}^{\mathrm{r8}}\!\leftarrow r$$
- Case $({}^{\mathrm{r6}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r10}})$: $[t]\,a\,;b\langle s\rangle\,;(M+N)$ gives the following critical pair.
$$\langle l, r\rangle = \langle b\langle s\rangle\,;[t]\,a\,;(M+N), [t]\,a\,;((b\langle s\rangle\,;M)+(b\langle s\rangle\,;N))\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r10}} \cdot \twoheadrightarrow^{\mathrm{r10}} (b\langle s\rangle\,;[t]\,a\,;M)+(b\langle s\rangle\,;[t]\,a\,;N)\ {}^{\mathrm{r6}}\!\leftarrow\cdot\ {}^{\mathrm{r6}}\!\leftarrow\cdot\ {}^{\mathrm{r10}}\!\leftarrow r$$
- Case $({}^{\mathrm{r6}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r15}})$: $[t]\,a\,;b\langle s\rangle\,;0$ gives the following critical pair.
$$\langle l, r\rangle = \langle b\langle s\rangle\,;[t]\,a\,;0, [t]\,a\,;0\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r15}} \cdot \twoheadrightarrow^{\mathrm{r15}} 0\ {}^{\mathrm{r15}}\!\leftarrow r$$
- Case $({}^{\mathrm{r7}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r8}})$: $M\,;(\exists x.\,(N+P))$ gives the following critical pair.
$$\langle l, r\rangle = \langle M\,;((\exists x.\,N)+(\exists x.\,P)), \exists x.\,(M\,;(N+P))\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r10}} \cdot \twoheadrightarrow^{\mathrm{r8}} \cdot \twoheadrightarrow^{\mathrm{r8}} (\exists x.\,(M\,;N))+(\exists x.\,(M\,;P))\ {}^{\mathrm{r7}}\!\leftarrow\cdot\ {}^{\mathrm{r10}}\!\leftarrow r$$
- Case $({}^{\mathrm{r7}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r11}})$: $\exists x.\,(M+0)$ gives the following critical pair.
$$\langle l, r\rangle = \langle (\exists x.\,M)+(\exists x.\,0), \exists x.\,M\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r14}} \cdot \twoheadrightarrow^{\mathrm{r11}} \exists x.\,M = r$$
- Case $({}^{\mathrm{r7}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r12}})$: $\exists x.\,(0+M)$ gives the following critical pair.
$$\langle l, r\rangle = \langle (\exists x.\,0)+(\exists x.\,M), \exists x.\,M\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r14}} \cdot \twoheadrightarrow^{\mathrm{r12}} \exists x.\,M = r$$
- Case $({}^{\mathrm{r7}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r14}})$: $\exists x.\,(M+N)$ gives the following critical pair.
$$\langle l, r\rangle = \langle (\exists x.\,M)+(\exists x.\,N), M+N\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r14}} \cdot \twoheadrightarrow^{\mathrm{r14}} M+N = r$$
- Case $({}^{\mathrm{r8}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r9}})$: $(M+N)\,;\exists x.\,P$ gives the following critical pair.
$$\langle l, r\rangle = \langle \exists x.\,((M+N)\,;P), (M\,;\exists x.\,P)+(N\,;\exists x.\,P)\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r9}} \cdot \twoheadrightarrow^{\mathrm{r7}} (\exists x.\,(M\,;P))+(\exists x.\,(N\,;P))\ {}^{\mathrm{r8}}\!\leftarrow\cdot\ {}^{\mathrm{r8}}\!\leftarrow r$$
- Case $({}^{\mathrm{r8}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r16}})$: $0\,;\exists x.\,M$ gives the following critical pair.
$$\langle l, r\rangle = \langle \exists x.\,(0\,;M), 0\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r16}} \cdot \twoheadrightarrow^{\mathrm{r14}} 0 = r$$
- Case $({}^{\mathrm{r9}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r10}})$: $(M+N)\,;(P+Q)$ gives the following critical pair.
$$\langle l, r\rangle = \langle (M\,;(P+Q))+(N\,;(P+Q)), ((M+N)\,;P)+((M+N)\,;Q)\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r10}} \cdot \twoheadrightarrow^{\mathrm{r10}} (M\,;P)+(N\,;P)+(M\,;Q)+(N\,;Q)\ {}^{\mathrm{r9}}\!\leftarrow\cdot\ {}^{\mathrm{r9}}\!\leftarrow r$$
- Case $({}^{\mathrm{r9}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r11}})$: $(M+0)\,;N$ gives the following critical pair.
$$\langle l, r\rangle = \langle (M\,;N)+(0\,;N), M\,;N\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r16}} \cdot \twoheadrightarrow^{\mathrm{r11}} M\,;N = r$$
- Case $({}^{\mathrm{r9}}\!\leftarrow \cdot \twoheadrightarrow^{\mathrm{r12}})$: $(0+M)\,;N$ gives the following critical pair.
$$\langle l, r\rangle = \langle (0\,;N)+(M\,;N), M\,;N\rangle$$
Join:
$$l \twoheadrightarrow^{\mathrm{r16}} \cdot \twoheadrightarrow^{\mathrm{r12}} M\,;N = r$$

- Case ($\overset{\mathrm{r9}}{\leftarrow} \cdot \to^{\mathrm{r15}}$): $(M + N)\,;0$ gives the following critical pair.
$$\langle l, r\rangle = \langle (M\,;0) + (N\,;0), 0\rangle$$
Join:
$$l \to^{\mathrm{r15}} \cdot \to^{\mathrm{r15}} \cdot \to^{\mathrm{r11}} 0 = r$$
- Case ($\overset{\mathrm{r10}}{\leftarrow} \cdot \to^{\mathrm{r11}}$): $M\,;(N + 0)$ gives the following critical pair.
$$\langle l, r\rangle = \langle (M\,;N) + (M\,;0), M\,;N\rangle$$
Join:
$$l \to^{\mathrm{r15}} \cdot \to^{\mathrm{r11}} M\,;N = r$$
- Case ($\overset{\mathrm{r10}}{\leftarrow} \cdot \to^{\mathrm{r12}}$): $M\,;(0 + N)$ gives the following critical pair.
$$\langle l, r\rangle = \langle (M\,;0) + (M\,;N), M\,;N\rangle$$
Join:
$$l \to^{\mathrm{r15}} \cdot \to^{\mathrm{r12}} M\,;N = r$$
- Case ($\overset{\mathrm{r10}}{\leftarrow} \cdot \to^{\mathrm{r13}}$): $\star\,;(M + N)$ gives the following critical pair.
$$\langle l, r\rangle = \langle (\star\,;M) + (\star\,;N), M + N\rangle$$
Join:
$$l \to^{\mathrm{r13}} \cdot \to^{\mathrm{r13}} M + N = r$$
- Case ($\overset{\mathrm{r10}}{\leftarrow} \cdot \to^{\mathrm{r16}}$): $0\,;(M + N)$ gives the following critical pair.
$$\langle l, r\rangle = \langle (0\,;M) + (0\,;N), 0\rangle$$
Join:
$$l \to^{\mathrm{r16}} \cdot \to^{\mathrm{r16}} \cdot \to^{\mathrm{r11}} 0 = r$$

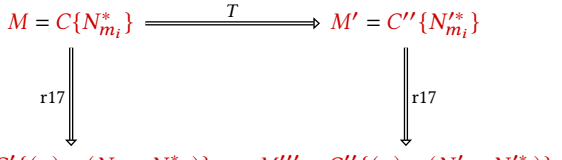
We conclude that all critical pairs of $\to$ are joinable, hence $\to$ is locally confluent. □

Lemma C.6. *$\Rightarrow^{T}$ and $\Rightarrow^{r17}$ commute.*

Proof. It suffices to show that $\Rightarrow^{\mathrm{T}}$ and $\Rightarrow^{\mathrm{r17}}$ strongly commute. That is, that we have the following commutative diagrams.



We begin by assigning a unique marking m from a finite alphabet Σ to each occurrence of a redex N^* in M in the reduction $M \Rightarrow^{\mathrm{r17}} M''$. Note that $M \Rightarrow^{\mathrm{T}} M'$ induces a partial mapping ϕ from markings m to their corresponding (non-overlapping, possibly duplicated) reducts N'^*_m in M'. Let $\{m_i\}$ be the family of all markings m such that $m \in \phi^{-1}(\Sigma)$. Given a marking $m \in \{m_i\}$, each of M, M', M'' and M''' corresponds to a collection of occurrences of identical terms N^*_m inside a context C, which, by abuse of notation, we denote $C\{{N_m}^*\}$. Since $\Rightarrow^{18}$ does not destroy redexes, we are then able to construct the following commutative diagram.



□

Theorem C.7. *$\to$ is confluent.*

Proof. Note that $\to^{\mathrm{r17}}$ was not used to join any critical pairs in Lemma C.5, so $\to^{\mathrm{T}}$ is locally confluent. By Proposition 2.5, we have that $\to^{\mathrm{T}}$ is terminating, and so, by Newman's lemma, confluent. $\to^{\mathrm{r17}}$ is trivially confluent, and so we have confluence of both $\Rightarrow^{\mathrm{T}}$ and $\Rightarrow^{\mathrm{r17}}$. Applying the Hindley-Rosen lemma to Lemma C.6, we recover confluence of $\Rightarrow^{\mathrm{T}} \cup \Rightarrow^{\mathrm{r17}} = \to$. □

D PROOFS FOR SECTION 5: SIMPLE TYPES

We use the following Lemma in the "pop" case of the proof of machine progress.

Lemma D.1. *Given $\vec{x} : \vec{\rho} \vdash s_i, t_i : \tau_i$, there exists $\vec{x} : \vec{\rho} \vdash R : \vec{\rho}$ such that $\{R/\vec{x}\} = mgu(\vec{s} \doteq \vec{t})$.*

Proof. Induction as in Proposition 3.2, using the same measure. The only non-trivial case is as follows. Let $\vec{x} : \vec{\rho}, y : \upsilon \vdash \vec{s} \doteq \vec{t}, u \doteq y$, $y \notin u$. Then by inductive hypothesis, we have that there exists $\vec{x} : \vec{\rho} \vdash R : \vec{\rho}$ such that $\{R/\vec{x}\} = \mathrm{mgu}(\{u/y\}\vec{s} \doteq \{u/y\}\vec{t})$. Therefore we have that $\{R/\vec{x}\}\{u/y\} = \mathrm{mgu}(\vec{s} \doteq \vec{t}, u = y)$. Note that $\{R/\vec{x}\}\{u/y\} = \{R/\vec{x}, \{R/\vec{x}\}u/y\}$ and so $\vec{x} : \vec{\rho}, y : \upsilon \vdash R\{R/\vec{x}\}u : \vec{\rho}\upsilon$ as required. □

Theorem D.2 (Machine progress). *For any closed typed term $M : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$ we have*
$$\vdash M : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A \implies \forall (\vdash S_A : \vec{\sigma}_A).\ \lfloor S_A \Downarrow M \rfloor : \vec{\tau}_A\ .$$

Proof. In fact, we prove a stronger analogue of the statement for multisets. We strengthen the statement by considering open terms as follows: for every typed term $\vec{x} : \vec{\rho} \vdash M : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$, we have
$$\forall (\vec{y} : \vec{\rho}' \vdash S_A : \vec{\sigma}_A, R : \vec{\rho}).\ \forall (T_A, \tau) \in S_A \Downarrow \{R/\vec{x}\}M.$$
$$\exists R', \vec{z} : \vec{\rho}''.\ (\vec{z} : \vec{\rho}'' \vdash T_A : \vec{\tau}_A, R' : \vec{\rho}') \wedge \tau = \{R'/\vec{y}\}$$

Proceed by induction on the type derivation of $\vec{x} : \vec{\rho} \vdash M : \overleftarrow{\sigma}_A \Rightarrow \overrightarrow{\tau}_A$. Let $\vec{y} : \vec{\rho}' \vdash S_A : \vec{\sigma}_A, R : \vec{\rho}$ be arbitrary, and let $\rho = \{R/\vec{x}\}$. We will use without mention that substitution preserves typing (Lemma 5.2).

- $M \equiv \star$ follows from the definition $S_A \Downarrow \rho\star = [(S_A, \varepsilon)]$, recalling that $\varepsilon = \{\vec{y}/\vec{y}\}$ for $\vec{y} : \vec{\rho}' \vdash \vec{y} : \vec{\rho}'$.
- $M \equiv M_1\,;M_2$:
$$S_A \Downarrow \rho(M\,;N) = [(U_A, \upsilon\tau) \mid (T_A, \tau) \in S_A \Downarrow \{R/\vec{x}\}M,\ (U_A, \upsilon) \in T_A \Downarrow \tau\{R/\vec{x}\}N]$$
By applying the inductive hypothesis to M, S_A and R we have that
$$\exists R', \vec{z} : \vec{\rho}''.\ (\vec{z} : \vec{\rho}'' \vdash T_A : \vec{\tau}_A, R' : \vec{\rho}') \wedge \tau = \{R'/\vec{y}\}.$$

We thus have that $\vec{z} : \vec{\rho}'' \vdash \{R'/\vec{y}\}R : \vec{\rho}$, and therefore $\tau\{R/\vec{x}\}N = \{\{R'/\vec{y}\}R/\vec{x}\}N$. Thus, applying the inductive hypothesis to N, T_A and $\{\{R'/\vec{y}\}R/\vec{x}\}$ we have that

$$\exists R', \vec{w} : \vec{\rho}'''. (\vec{w} : \vec{\rho}''' \vdash U_A : \vec{v}_A, R'' : \vec{\rho}'') \wedge \upsilon = \{R''/\vec{z}\}.$$

The statement follows by noting that $\vec{w} : \vec{\rho}''' \vdash \{R''/\vec{z}\}R' : \vec{\rho}'$ and that $\upsilon\tau = \{\{R''/\vec{z}\}R'/\vec{y}\}$, using the assumption that $\vec{z}$ is globally fresh and thus not free in the continuation.

- $M \equiv 0$: follows vacuously from the definition $S_A \Downarrow \rho 0 = [\,]$.
- $M \equiv M_1 + M_2$: follows from the definition $S_A \Downarrow \rho(M + N) = (S_A \Downarrow \rho M) \sqcup (S_A \Downarrow \rho N)$ and the inductive hypothesis.
- $M \equiv [t]a$ follows from the definition $S_A \cdot S_a \Downarrow \rho[t]a = [(S_A \cdot S_a\, \rho t, \varepsilon)]$ and the observation that $S_A \cdot S_a\, \rho t : \vec{\sigma}_A \cdot \sigma_a\, \tau$, recalling that $\varepsilon = \{\vec{y}/\vec{y}\}$ for $\vec{y} : \vec{\rho}' \vdash \vec{y} : \vec{\rho}'$.
- $M \equiv a\langle t \rangle$: By Proposition 3.2 we have

$$S_A \cdot S_a\, s \Downarrow \rho a\langle t \rangle = [(\sigma(S_A \cdot S_a), \sigma) \mid \sigma = \mathrm{mgu}(s \doteq \rho t)]\,.$$

If the most general unifer σ doesn't exist, the statement holds vacuously. Otherwise, we require that

$$\exists R', \vec{z} : \vec{\rho}''. (\vec{z} : \vec{\rho}'' \vdash R' : \vec{\rho}') \wedge \sigma = \{R'/\vec{y}\}\,,$$

and so $\vec{z} : \vec{\rho}'' \vdash \sigma(S_A \cdot S_a) : \vec{\sigma}_A \cdot \sigma_a$ as required. This follows from Lemma D.1.

- $M \equiv \exists w.\, M'$: noting that $w \notin \mathrm{dom}(\rho)$, we have

$$S_A \Downarrow \rho \exists w.\, M' = [(T_A, \tau \setminus u) \mid (T_A, \tau) \in S_A \Downarrow \{R/\vec{x}\}\{u/w\}M']\,,$$

with u fresh. Noting that $\{R/\vec{x}\}\{u/w\} = \{Ru/\vec{x}w\}$, we can apply the inductive hypothesis on $\vec{x} : \vec{\rho}, w : \upsilon \vdash M : \vec{\sigma}_A \Rightarrow \vec{\tau}_A$ and $\vec{y} : \vec{\rho}', u : \upsilon \vdash S_A : \vec{\sigma}_A$ (achieved by applying weakening to S_A) and $\vec{y} : \rho', u : \upsilon \vdash Ru : \vec{\rho}\upsilon$ (achieved by concatenating R and u to find

$$\exists R', \vec{z} : \vec{\rho}''. (\vec{z} : \vec{\rho}'' \vdash T_A : \vec{\tau}_A, R' : \vec{\rho}'\upsilon) \wedge \tau = \{R'/\vec{y}u\}$$

The result follows by noting $\{R'/\vec{y}u\} \setminus u = \{\pi_0 R'/\vec{y}\}$, where $\pi_0 R'$ denotes the tail of R', and that $\vec{z} : \vec{\rho}'' \vdash \pi_0 R' : \vec{\rho}'$. □

Theorem D.3 (Theorem 4.9 restatement). *We have the following:*

- *for any closed, star-free term M, we have $M \leq M\,;M^\dagger\,;M$;*
- *if M is also sum-free then $M = M\,;M^\dagger\,;M$;*

Proof. We prove the statements in reverse order. In each case, we consider an equivalent term in normal form.

To prove the second statement, assume that M is in normal form, so that $M \equiv \exists \overleftarrow{x}. (\langle \overleftarrow{t} \rangle\,;[\vec{u}])$. Thus, we have

$$\begin{aligned}
&M\,;M^\dagger\,;M \\
&= \exists \overleftarrow{x}. (\langle \overleftarrow{t} \rangle\,;[\vec{u}])\,;\exists \overleftarrow{x}. (\langle \overleftarrow{u} \rangle\,;[\vec{t}])\,;\exists \overleftarrow{x}. (\langle \overleftarrow{t} \rangle\,;[\vec{u}]) \\
&= \exists \overleftarrow{xyz}. (\langle \overleftarrow{t} \rangle\,;\underline{[\vec{u}]}\,;\langle \{\vec{y}/\vec{x}\}\vec{u} \rangle\,;[\{\vec{y}/\vec{x}\}\vec{t}]\,;\langle \{\vec{z}/\vec{x}\}\overleftarrow{t} \rangle\,;[\{\vec{z}/\vec{x}\}\vec{u}]) \\
(*) &= \exists \overleftarrow{xz}. \{\vec{x}/\vec{y}\}(\langle \overleftarrow{t} \rangle\,;[\{\vec{y}/\vec{x}\}\vec{t}]\,;\langle \{\vec{z}/\vec{x}\}\overleftarrow{t} \rangle\,;[\{\vec{z}/\vec{x}\}\vec{u}]) \\
&= \exists \overleftarrow{xz}. (\langle \overleftarrow{t} \rangle\,;\underline{[\vec{t}]}\,;\langle \{\vec{z}/\vec{x}\}\overleftarrow{t} \rangle\,;[\{\vec{z}/\vec{x}\}\vec{u}]) \\
(*) &= \exists \overleftarrow{x}. \{\vec{x}/\vec{z}\}(\langle \overleftarrow{t} \rangle\,;[\{\vec{z}/\vec{x}\}\vec{u}]) \\
&= \exists \overleftarrow{x}. (\langle \overleftarrow{t} \rangle\,;[\vec{u}])
\end{aligned}$$

where the steps $(*)$ are given by repeated application of β and unification rules. Unification is guaranteed to succeed, because the constant symbols are matched by definition, and the freshness of $\vec{y}$ and $\vec{z}$ guarantee no occurs check fails.

We prove the first statement, using the second. Again, we assume that M is in normal form, so that $M \equiv M_1 + \ldots + M_n$ where each M_i is sum-free. We prove the case for $n = 2$, but the proof generalizes easily to arbitrary n. Then we have

$$\begin{aligned}
M_1 + M_2 &= M_1\,;M_1^\dagger M_1 \;+\; M_2\,;M_2^\dagger M_2 \\
(*) &\leq M_1\,;M_1^\dagger M_1 + M_1\,;M_1^\dagger M_2 + M_1\,;M_2^\dagger M_1 + M_1\,;M_2^\dagger M_2 + \\
&\quad M_2\,;M_1^\dagger M_1 + M_2\,;M_1^\dagger M_2 + M_2\,;M_2^\dagger M_1 + M_2\,;M_2^\dagger M_2 + \\
&= (M_1 + M_2)\,;(M_1^\dagger + M_2^\dagger)\,;(M_1 + M_2) \\
&= (M_1 + M_2)\,;(M_1 + M_2)^\dagger\,;(M_1 + M_2)
\end{aligned}$$

where $(*)$ is the repeated application of distributivities. □

Corollary D.4 (Corollary 5.4 restatement). *For any closed typed term $M : \overleftarrow{\sigma}_A \Rightarrow \vec{\tau}_A$ and memory $S_A : \vec{\sigma}_A$, (S_A, M, ε) makes progress.*

Proof. Induction on type derivations, applying Theorem 5.3 in the case of sequencing in order to ensure the continuation receives the correctly typed input stack. □

Theorem D.5 (Theorem 5.7 restatement). *The typed equational theory is sound with respect to typed machine equivalence:*

$$\Gamma \vdash M = N : \overleftarrow{\sigma}_A \Rightarrow \vec{\tau}_A \;\Rightarrow\; \Gamma \vdash M \sim N : \overleftarrow{\sigma}_A \Rightarrow \vec{\tau}_A\,.$$

Proof. Typed machine equivalence is easily seen to identify more similarly typed terms than untyped machine equivalence, since it has available fewer memories with which to seperate terms. The η-axiom is the only strengthening of the typed equational theory, so it suffices to check that the stengthened η equation, which is easily done following the η case in the proof of Theorem 4.5 using the property of machine progress and the requirement that for typed machine equivalence a term is tested only on appropriately typed inputs. □

E PROOFS FOR SECTION 6: CATEGORICAL SEMANTICS

Lemma E.1 (Interchange). *Given closed terms $M : \overleftarrow{\rho} \Rightarrow \vec{\sigma}$ and $N : \overleftarrow{\tau} \Rightarrow \vec{\upsilon}$ and $\vec{x} : \vec{\tau}$, $\vec{y} : \vec{\upsilon}$, we have*

$$\exists \overleftarrow{x}. (\langle \overleftarrow{x} \rangle\,;M\,;[\vec{x}])\,;N = N\,;\exists \vec{y}. (\langle \vec{y} \rangle\,;M\,;[\vec{y}])\,.$$

Proof. Using the notation of Definition 6.3, we will write $M \otimes \vec{\tau}$ to denote $\exists \overleftarrow{x}. (\langle \overleftarrow{x} \rangle\,;M\,;[\vec{x}])$ We proceed by induction on the type derivation of N.

- $N \equiv \star$: follows immediately from $\star\,;P = P = P\,;\star$.
- $N \equiv 0$: follows immediately from $0\,;P = 0 = P\,;0$.
- $N \equiv [t] : \vec{\sigma} \Rightarrow \vec{\sigma}\tau$:

$$\begin{aligned}
[t]\,;(M \otimes \vec{\sigma}\tau) &= [t]\,;\exists y. \langle y \rangle\,;(M \otimes \vec{\sigma})\,;[y] \\
&=_\nu \exists y. [t]\,;\langle y \rangle\,;(M \otimes \vec{\sigma})\,;[y] \\
&=_\beta (M \otimes \vec{\sigma})\,;[t]
\end{aligned}$$

- $N \equiv \langle t \rangle : \tau \overleftarrow{\sigma} \Rightarrow \overrightarrow{\sigma}$:

$$\begin{aligned}\langle t \rangle ; (M \otimes \overrightarrow{\sigma}) &=_\beta \exists y.\, (\langle y \rangle ; (M \otimes \overrightarrow{\sigma}) ; [y] ; \langle t \rangle) \\ &=_\nu \exists y.\, (\langle y \rangle ; (M \otimes \overrightarrow{\sigma}) ; [y]) ; \langle t \rangle \\ &= (M \otimes \overrightarrow{\sigma\tau}) ; \langle t \rangle\end{aligned}$$

- $N \equiv N_0 ; N_1$:

$$\begin{aligned}N_0 ; N_1 ; (M \otimes \overrightarrow{\tau}) \ \text{(I.H.)} &= N_0 ; (M \otimes \overrightarrow{\tau'}) ; N_1 \\ \text{(I.H.)} &= (M \otimes \overrightarrow{\tau''}) ; N_0 ; N_1\end{aligned}$$

- $N \equiv N_0 + N_1$:

$$\begin{aligned}(N_0 + N_1) ; (M \otimes \overrightarrow{\tau}) &= N_0 ; (M \otimes \overrightarrow{\tau}) + N_1 ; (M \otimes \overrightarrow{\tau}) \\ \text{(I.H.)} \times 2 &= (M \otimes \overrightarrow{\tau'}) ; N_0 + (M \otimes \overrightarrow{\tau'}) ; N_1 \\ &= M \otimes \overrightarrow{\tau'} ; (N_0 + N_1)\end{aligned}$$

- $N \equiv \exists x.\, N'$:

$$\begin{aligned}(\exists x.\, N') ; (M \otimes \overrightarrow{\tau}) &= \exists x.\, N' ; (M \otimes \overrightarrow{\tau}) \\ \text{(I.H.)} &= \exists x.\, (M \otimes \overrightarrow{\tau'}) ; N' \\ &=_\nu (M \otimes \overrightarrow{\tau'}) ; \exists x.\, N'\end{aligned}$$

where the side-condition $x \notin \mathrm{fv}(M)$ is indeed satisfied. □

Proposition E.2 (RMC is symmetric monoidal). *The Relational Machine Category RMC$[\Sigma_0]$ is a symmetric monoidal category, with the equipment given in Definition 6.3*

Proof. It is immediate that RMC$[\Sigma_0]$ is indeed a category, by the axioms of KA that make $(M, ;, \star)$ a monoid. In the following calculuations, we often omit to mention when we apply the axioms of KA, *e.g.*, associativity.

We now prove that RMC$[\Sigma_0]$ is a symmetric monoidal category. First, we prove that the left- and right-actions of $\otimes$ are functorial. Taking $M : \overleftarrow{\rho} \Rightarrow \overrightarrow{\sigma}$ and $\overrightarrow{x}, \overrightarrow{y} : \overrightarrow{\tau}$, it suffices to calculate the following.

$$\begin{aligned}(\overrightarrow{\tau} \otimes \star) &= \star \\ (\star \otimes \overrightarrow{\tau}) &= \exists \overrightarrow{x}.\, (\langle \overrightarrow{x} \rangle ; \star ; [\overrightarrow{x}]) =_\eta \star \\ (\overrightarrow{\tau} \otimes M); (\overrightarrow{\tau} \otimes N) &= M; N = \overrightarrow{\tau} \otimes M ; N \\ (M \otimes \overrightarrow{\tau}); (N \otimes \overrightarrow{\tau}) &= \exists \overrightarrow{x}.\, (\langle \overrightarrow{x} \rangle ; M ; [\overrightarrow{x}]) ; \exists \overrightarrow{y}.\, (\langle \overrightarrow{y} \rangle ; N ; [\overrightarrow{y}]) \\ &=_\nu \exists \overrightarrow{xy}.\, (\langle \overrightarrow{x} \rangle ; M ; [\overrightarrow{x}] ; \langle \overrightarrow{y} \rangle ; N ; [\overrightarrow{y}]) \\ &=_\beta \exists \overrightarrow{x}.\, (\langle \overrightarrow{x} \rangle ; M ; N ; [\overrightarrow{x}]) \\ &= M ; N \otimes \overrightarrow{\tau}\end{aligned}$$

We now show that $\otimes$ is *bi*-functorial. Taking now $N : \overleftarrow{\tau} \Rightarrow \overrightarrow{v}$, it suffces to prove the following *interchange* law.

$$(M \otimes \overrightarrow{\tau}); (\overrightarrow{\sigma} \otimes N) = (\overrightarrow{\rho} \otimes N); (M \otimes \overrightarrow{v})$$

This is proved in Lemma E.1.

We now prove that RMC$[\Sigma_0]$ is a *symmetric* monoidal category. We first check the symmetry equation.

$$\begin{aligned}&\mathrm{sym}_{\overrightarrow{\sigma},\overrightarrow{\tau}}; \mathrm{sym}_{\overrightarrow{\tau},\overrightarrow{\sigma}} \\ &= \exists \overrightarrow{wx}.\, (\langle \overrightarrow{w} \rangle ; \langle \overrightarrow{x} \rangle ; [\overrightarrow{w}] ; [\overrightarrow{x}]) ; \exists \overrightarrow{yz}.\, (\langle \overrightarrow{y} \rangle ; \langle \overrightarrow{z} \rangle ; [\overrightarrow{y}] ; [\overrightarrow{z}]) \\ &=_\nu \exists \overrightarrow{wxyz}.\, (\langle \overrightarrow{w} \rangle ; \langle \overrightarrow{x} \rangle ; [\overrightarrow{w}] ; [\overrightarrow{x}] ; \langle \overrightarrow{y} \rangle ; \langle \overrightarrow{z} \rangle ; [\overrightarrow{y}] ; [\overrightarrow{z}] \\ &=_\beta \exists \overrightarrow{wx}.\, (\langle \overrightarrow{w} \rangle ; \langle \overrightarrow{x} \rangle ; [\overrightarrow{x}] ; [\overrightarrow{w}] =_\eta \star = \mathrm{id}_{\overrightarrow{\sigma},\overrightarrow{\tau}}\end{aligned}$$

We now check the hexagon identity.

$$\begin{aligned}&(\mathrm{sym}_{\overrightarrow{\rho},\overrightarrow{\sigma}} \otimes \overrightarrow{\tau}); (\overrightarrow{\sigma} \otimes \mathrm{sym}_{\overrightarrow{\rho},\overrightarrow{\tau}}) \\ &= \exists x.\, (\langle x \rangle ; \exists yz.\, (\langle y \rangle ; \langle z \rangle ; [y] ; [z]) ; [x]) ; \exists vw.\, (\langle v \rangle ; \langle w \rangle ; [v] ; [w]) \\ &=_\nu \exists vwxyz.\, (\langle x \rangle ; \langle y \rangle ; \langle z \rangle ; [y] ; [z] ; [x] ; \langle v \rangle ; \langle w \rangle ; [v] ; [w]) \\ &=_\beta \exists wxyz.\, (\langle x \rangle ; \langle y \rangle ; \langle z \rangle ; [y] ; [z] ; \langle w \rangle ; [x] ; [w]) \\ &=_\beta \exists xyz.\, (\langle x \rangle ; \langle y \rangle ; \langle z \rangle ; [y] ; [x] ; [z]) \\ &= \exists xyz.\, (\langle xy \rangle \langle z \rangle ; [yx] ; [z]) = \mathrm{sym}_{\overrightarrow{\rho},\overrightarrow{\sigma\tau}}\end{aligned}$$

Finally, we check the naturality of symmetry. It suffices to give a proof of naturality in each argument separately, since $\otimes$ is bifunctorial. Thus, we prove the following, with naturality in the other component following by duality: for $N : \overleftarrow{\rho} \Rightarrow \overrightarrow{\sigma}$, we have

$$(\overrightarrow{\tau} \otimes N); \mathrm{sym}_{\overrightarrow{\tau},\overrightarrow{\sigma}} = \mathrm{sym}_{\overrightarrow{\tau},\overrightarrow{\rho}}; (N \otimes \overrightarrow{\tau})$$

We proceed by induction on the type derivation of N. We omit subscripts on natural transformations when it is clear from context.

- $N \equiv \star$: follows immediately from $\star ; P = P = P ; \star$.
- $N \equiv 0$: follows immediately from $0 ; P = 0 = P ; 0$.
- $N \equiv [t] : \overleftarrow{\sigma} \Rightarrow \overrightarrow{\sigma\tau}$:

$$\begin{aligned}&(\overrightarrow{\rho} \otimes [t]); \mathrm{sym}_{\overrightarrow{\rho},\overrightarrow{\sigma\tau}} \\ &= [t] ; \exists \overrightarrow{rst'}.\, (\langle t' \overrightarrow{s} \rangle ; \langle \overrightarrow{r} \rangle ; [\overrightarrow{st'}] ; [\overrightarrow{r}]) \\ &=_\nu \exists \overrightarrow{rst'}.\, ([t] ; \langle t' \overrightarrow{s} \rangle ; \langle \overrightarrow{r} \rangle ; [\overrightarrow{st'}] ; [\overrightarrow{r}]) \\ &=_\beta \exists \overrightarrow{rs}.\, (\langle \overrightarrow{s} \rangle ; \langle \overrightarrow{r} \rangle ; [\overrightarrow{st}] ; [\overrightarrow{r}]) \\ &=_\beta \exists \overrightarrow{rr's}\, (\langle \overrightarrow{s} \rangle ; \langle \overrightarrow{r} \rangle ; [\overrightarrow{s}] ; [\overrightarrow{r}] ; \langle \overrightarrow{r'} \rangle ; [t] ; [\overrightarrow{r'}]) \\ &=_\nu \exists \overrightarrow{rs} (\langle \overrightarrow{s} \rangle ; \langle \overrightarrow{r} \rangle ; [\overrightarrow{s}] ; [\overrightarrow{r}]) ; \exists \overrightarrow{r'}.\, (\langle \overrightarrow{r'} \rangle ; [t] ; [\overrightarrow{r'}]) \\ &= \mathrm{sym}_{\overrightarrow{\rho},\overrightarrow{\sigma}}; ([t] \otimes \overrightarrow{\rho})\end{aligned}$$

- $N \equiv \langle t \rangle : \tau \overleftarrow{\sigma} \Rightarrow \overleftarrow{\sigma}$:

$$\begin{aligned}&(\overrightarrow{\rho} \otimes \langle t \rangle); \mathrm{sym}_{\overrightarrow{\rho},\overrightarrow{\sigma}} \\ &= \langle t \rangle ; \exists \overrightarrow{rs}.\, (\langle \overrightarrow{s} \rangle ; \langle \overrightarrow{r} \rangle ; [\overrightarrow{s}] ; [\overrightarrow{r}]) \\ &=_\nu \exists \overrightarrow{rs}.\, (\langle t \overrightarrow{s} \rangle ; \langle \overrightarrow{r} \rangle ; [\overrightarrow{s}] ; [\overrightarrow{r}]) \\ &=_\beta \exists \overrightarrow{rr's}.\, (\langle t' \overrightarrow{s} \rangle ; \langle \overrightarrow{r} \rangle ; [\overrightarrow{st'}] ; \langle t \rangle ; [r]) \\ &=_\beta \exists \overrightarrow{rr's}.\, (\langle t' \overrightarrow{s} \rangle ; \langle \overrightarrow{r'} \rangle ; [\overrightarrow{st'}] ; [\overrightarrow{r'}] ; \langle \overrightarrow{r} \rangle ; \langle t \rangle ; [r]) \\ &=_\nu \exists \overrightarrow{r's}.\, (\langle t' \overrightarrow{s} \rangle ; \langle \overrightarrow{r'} \rangle ; [\overrightarrow{st'}] ; [\overrightarrow{r'}]) ; \exists \overrightarrow{r}.\, (\langle \overrightarrow{r} \rangle ; \langle t \rangle ; [r]) \\ &= \mathrm{sym}_{\rho,\overrightarrow{\sigma\tau}}; (\langle t \rangle \otimes \overrightarrow{\rho})\end{aligned}$$

- $N \equiv N_0 ; N_1$:

$$\begin{aligned}(\overrightarrow{\tau} \otimes N_0 ; N_1); \mathrm{sym} &= (\overrightarrow{\tau} \otimes N_0); (\overrightarrow{\tau} \otimes N_1); \mathrm{sym} \\ &= (\overrightarrow{\tau} \otimes N_0); (\overrightarrow{\tau} \otimes N_1); \mathrm{sym} \\ \text{(I.H.)} &= (\overrightarrow{\tau} \otimes N_0); \mathrm{sym}; (N_1 \otimes \overrightarrow{\tau}) \\ \text{(I.H.)} &= \mathrm{sym}; (\overrightarrow{\tau} \otimes N_0); (N_1 \otimes \overrightarrow{\tau}) \\ &= \mathrm{sym}; (\overrightarrow{\tau} \otimes N_0 ; N_1)\end{aligned}$$

- $N \equiv N_0 + N_1$:

$$\begin{aligned}(\vec{\tau} \otimes N_0 + N_1); \mathsf{sym} &= ((\vec{\tau} \otimes N_0) + (\vec{\tau} \otimes N_1)); \mathsf{sym} \\ &= ((\vec{\tau} \otimes N_0); \mathsf{sym}) + ((\vec{\tau} \otimes N_1); \mathsf{sym}) \\ \text{(I.H.)} \times 2 &= (\mathsf{sym}; (N_0 \otimes \vec{\tau})) + (\mathsf{sym}; (N_1 \otimes \vec{\tau})) \\ &= \mathsf{sym}; ((N_0 \otimes \vec{\tau}) + (N_1 \otimes \vec{\tau})) \\ &= \mathsf{sym}; (N_0 + N_1 \otimes \vec{\tau})\end{aligned}$$

- $N \equiv \exists x.\, N'$:

$$\begin{aligned}(\vec{\tau} \otimes \exists x.\, M); \mathsf{sym} &= \exists x.\, (\vec{\tau} \otimes M); \mathsf{sym} \\ \text{(I.H.)} &= \exists x.\, \mathsf{sym}; (M \otimes \vec{\tau}) \\ &=_\nu \mathsf{sym}; \exists x.\, (M \otimes \vec{\tau}) \\ &=_\nu \mathsf{sym}; (\exists x.\, M \otimes \vec{\tau})\end{aligned}$$

The associators and unitors are given by the identity, so the remaining coherences hold trivially. □

Theorem E.3 (Theorem 6.4 restatement). *The Relational Machine Category RMC$[\Sigma_0]$ is a extra-hypergraph category with the equipment given in Definition 6.3*

Proof. We previously showed that RMC$[\Sigma_0]$ is a symmetric monoidal category. We now show that RMC$[\Sigma_0]$ is additionally an *extra-hypergraph* category. Note that, although in our definition of a Frobenius monoid we listed three terms in the Frobenius equations (to emphasize the duality of the definition), in fact any two of these equations implies the third. Further, because the objects of RMC$[\Sigma]$ are inductively generated from Σ_0, it suffices to define a Frobenius monoid on each base type, with Frobenius monoids at compound types defined using the compatibility conditions of Definition 6.2. Thus it suffices to check the following equations. Co-associativity, co-unitality, and co-commutativity follow from dual arguments.

- Associativity:

$$\begin{aligned}(\mu \otimes \tau); \mu &= \exists x.\, (\langle x\rangle ; \exists y.\, (\langle y\rangle ; \langle y\rangle ; [y]) ; [x]) ; \exists z.\, (\langle z\rangle ; \langle z\rangle ; [z]) \\ &=_\nu \exists xyz.\, (\langle x\rangle ; \langle y\rangle ; \langle y\rangle ; [y] ; [x] ; \langle z\rangle ; \langle z\rangle ; [z]) \\ &=_\beta \exists xy.\, (\langle x\rangle ; \langle y\rangle ; \langle y\rangle ; [y] ; \langle x\rangle ; [x]) \\ &=_\beta \exists x.\, (\langle x\rangle ; \langle x\rangle ; \langle x\rangle ; [x]) \\ &=_\nu \exists xy.\, (\langle x\rangle ; \langle x\rangle ; [x] ; \langle y\rangle ; \langle y\rangle ; [y]) \\ &= \exists x.\, (\langle x\rangle ; \langle x\rangle ; [x]) ; \exists y.\, (\langle y\rangle ; \langle y\rangle ; [y]) \\ &= (\tau \otimes \mu); \mu\end{aligned}$$

- Unitality:

$$\begin{aligned}(\tau \otimes \eta); \mu &= \exists x.\, ([x]) ; \exists y.\, (\langle y\rangle ; \langle y\rangle ; [y]) \\ &=_\nu \exists xy.\, ([x] ; \langle y\rangle ; \langle y\rangle ; [y]) \\ &=_\beta \exists x.\, (\langle x\rangle ; [x]) \\ &=_\eta \star = \mathsf{id}\end{aligned}$$

- Commutativity:

$$\begin{aligned}\mathsf{sym}; \mu &= \exists xy.\, (\langle x\rangle ; \langle y\rangle ; [x] ; [y]) ; \exists z.\, (\langle z\rangle ; \langle z\rangle ; [z]) \\ &=_\nu \exists xyz.\, (\langle x\rangle ; \langle y\rangle ; [x] ; [y] ; \langle z\rangle ; \langle z\rangle ; [z]) \\ &=_\beta \exists xy.\, (\langle x\rangle ; \langle y\rangle ; [x] ; \langle y\rangle ; [y]) \\ &=_\beta \exists x.\, (\langle x\rangle ; \langle x\rangle ; [x]) = \mu\end{aligned}$$

- Frobenius:

$$\begin{aligned}&(\tau \otimes \delta); (\mu \otimes \tau) \\ &= \exists x.\, (\langle x\rangle ; [x] ; [x]) ; \exists y.\, (\langle y\rangle ; \exists z.\, (\langle z\rangle ; \langle z\rangle ; [z]) ; [y]) \\ &=_\nu \exists xyz.\, (\langle x\rangle ; [x] ; [x] ; \langle y\rangle ; \langle z\rangle ; \langle z\rangle ; [z] ; [y]) \\ &=_\beta \exists xz.\, (\langle x\rangle ; [x] ; \langle z\rangle ; \langle z\rangle ; [z] ; [x]) \\ &=_\beta \exists x.\, (\langle x\rangle ; \langle x\rangle ; [x] ; [x]) \\ &=_\nu \exists xy.\, (\langle x\rangle ; \langle x\rangle ; [x] ; \langle y\rangle ; [y] ; [y]) \\ &= \exists x.\, (\langle x\rangle ; \langle x\rangle ; [x]) ; \exists y.\, (\langle y\rangle ; [y] ; [y]) = \mu; \delta\end{aligned}$$

- Specialness:

$$\begin{aligned}\delta; \mu &= \exists x.\, (\langle x\rangle ; [x] ; [x]) ; \exists y.\, (\langle y\rangle ; \langle y\rangle ; [y]) \\ &=_\nu \exists xy.\, (\langle x\rangle ; [x] ; [x] ; \langle y\rangle ; \langle y\rangle ; [y]) \\ &=_\beta \exists x.\, (\langle x\rangle ; [x] ; \langle x\rangle ; [x]) \\ &=_\beta \exists x.\, (\langle x\rangle ; [x]) =_\eta \star\end{aligned}$$ □

Proposition E.4 (Proposition 6.6 restatement). *For any closed term $M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$ we have the following.*

$$M^\dagger = \exists \overleftarrow{x}\overrightarrow{y}.\, (\langle \overleftarrow{x}\rangle ; [\vec{y}] ; M ; \langle \overleftarrow{x}\rangle ; [\vec{y}]) : \overleftarrow{\tau} \Rightarrow \vec{\sigma}$$

where $\overrightarrow{x} : \vec{\tau}$ and $\vec{y} : \vec{\sigma}$.

Proof. By induction on the type derivation of $M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$.

- $M \equiv \star : \overleftarrow{\tau} \Rightarrow \vec{\tau}$

$$\begin{aligned}&\exists \overleftarrow{x}\overrightarrow{y}.\, (\langle \overleftarrow{x}\rangle ; [\vec{y}] ; \star ; \langle \overleftarrow{x}\rangle ; [\vec{y}]) \\ = \;&\exists \overleftarrow{x}\overrightarrow{y}.\, (\langle \overleftarrow{x}\rangle ; [\vec{y}] ; \langle \overleftarrow{x}\rangle ; [\vec{y}]) \\ =_\beta\; &\exists \overrightarrow{y}.\, (\langle \overrightarrow{y}\rangle ; [\vec{y}]) =_\eta \star = \star^\dagger\end{aligned}$$

- $M \equiv M_1 ; M_2 : \overleftarrow{\sigma} \Rightarrow \vec{v}$

$$\begin{aligned}&\exists \overleftarrow{x}\overrightarrow{y}.\, (\langle \overleftarrow{x}\rangle ; [\vec{y}] ; M_1 ; M_2 ; \langle \overleftarrow{x}\rangle ; [\vec{y}]) \\ \text{(I.H.)} = \;&\exists \overleftarrow{x}\overrightarrow{y}.\, (\langle \overleftarrow{x}\rangle ; [\vec{y}] ; \exists \overrightarrow{u}\overleftarrow{v}.\, (\langle \overrightarrow{u}\rangle ; [\vec{v}] ; {M_1}^\dagger ; \langle \overrightarrow{u}\rangle ; [\vec{v}]) ; \\ &\exists \overrightarrow{w}\overleftarrow{z}.\, (\langle \overleftarrow{w}\rangle ; [\vec{z}] ; {M_2}^\dagger ; \langle \overleftarrow{w}\rangle ; [\vec{z}]) ; \langle \overleftarrow{x}\rangle ; [\vec{y}]) \\ =_\nu \;&\exists \overrightarrow{u}\overleftarrow{v}\overrightarrow{w}\overleftarrow{x}\overrightarrow{y}\overleftarrow{z}.\, (\langle \overleftarrow{x}\rangle ; \underline{[\vec{y}] ; \langle \overrightarrow{u}\rangle} ; [\vec{v}] ; {M_1}^\dagger ; \langle \overrightarrow{u}\rangle ; [\vec{v}] ; \\ &\langle \overleftarrow{w}\rangle ; [\vec{z}] ; {M_2}^\dagger ; \langle \overleftarrow{w}\rangle ; [\vec{z}] ; \langle \overleftarrow{x}\rangle ; [\vec{y}]) \\ =_\beta \;&\exists \overleftarrow{v}\overrightarrow{w}\overleftarrow{x}\overrightarrow{y}\overleftarrow{z}.\, (\langle \overleftarrow{x}\rangle ; [\vec{v}] ; {M_1}^\dagger ; \langle \overrightarrow{y}\rangle ; \underline{[\vec{v}] ;} \\ &\underline{\langle \overleftarrow{w}\rangle} ; [\vec{z}] ; {M_2}^\dagger ; \langle \overleftarrow{w}\rangle ; [\vec{z}] ; \langle \overleftarrow{x}\rangle ; [\vec{y}]) \\ =_\beta \;&\exists \overleftarrow{v}\overleftarrow{x}\overrightarrow{y}\overleftarrow{z}.\, (\langle \overleftarrow{x}\rangle ; [\vec{v}] ; {M_1}^\dagger ; \langle \overrightarrow{y}\rangle ; \\ &[\vec{z}] ; {M_2}^\dagger ; \langle \overleftarrow{v}\rangle ; \underline{[\vec{z}] ; \langle \overleftarrow{x}\rangle} ; [\vec{y}]) \\ =_\beta \;&\exists \overleftarrow{v}\overrightarrow{y}\overleftarrow{z}.\, (\langle \overleftarrow{z}\rangle ; [\vec{v}] ; {M_1}^\dagger ; \langle \overrightarrow{y}\rangle ; [\vec{z}] ; {M_2}^\dagger ; \langle \overleftarrow{v}\rangle ; [\vec{y}]) \\ =_\nu \;&\exists \overleftarrow{v}\overleftarrow{z}.\, (\langle \overleftarrow{z}\rangle ; [\vec{v}] ; {M_1}^\dagger ; \exists \overrightarrow{y}.\, (\langle \overrightarrow{y}\rangle ; [\vec{z}] ; {M_2}^\dagger ; \langle \overleftarrow{v}\rangle ; [\vec{y}])) \\ \text{(Lemma } E.1) = \;&\exists \overleftarrow{v}\overleftarrow{z}.\, (\langle \overleftarrow{z}\rangle ; [\vec{z}] ; {M_2}^\dagger ; \langle \overleftarrow{v}\rangle ; [\vec{v}] ; {M_1}^\dagger) \\ =_\nu \;&\vec{z}.\, (\langle \overleftarrow{z}\rangle ; [\vec{z}]) ; {M_2}^\dagger ; \exists \overleftarrow{v}.\, (\langle \overleftarrow{v}\rangle ; [\vec{v}]) ; {M_1}^\dagger \\ =_\eta \;&{M_2}^\dagger ; {M_1}^\dagger = (M_1 ; M_2)^\dagger\end{aligned}$$

  where $\overrightarrow{x}, \vec{z} : \vec{\sigma}$, $\vec{v}, \overrightarrow{w} : \vec{\tau}$ and $\vec{y}, \vec{u} : \vec{v}$.

- $M \equiv 0 : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$

$$\exists \overleftarrow{x}\overrightarrow{y}.\, (\langle \overleftarrow{x}\rangle ; [\vec{y}] ; 0 ; \langle \overleftarrow{x}\rangle ; [\vec{y}]) = 0 = 0^\dagger$$

- $M \equiv M_1 + M_2 : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$

$$\begin{aligned} & \exists \vec{x}\vec{y}.\, (\langle \vec{x} \rangle ; [\vec{y}] ; (M_1 + M_2) ; \langle \vec{x} \rangle ; [\vec{y}]) \\ =_\nu\ & \exists \vec{x}\vec{y}.\, (\langle \vec{x} \rangle ; [\vec{y}] ; M_1 ; \langle \vec{x} \rangle ; [\vec{y}]) + \\ & \exists \vec{x}\vec{y}.\, (\langle \vec{x} \rangle ; [\vec{y}] ; M_2 ; \langle \vec{x} \rangle ; [\vec{y}]) \\ (\text{I.H.}) =\ & M_1{}^\dagger + M_2{}^\dagger = (M_1 + M_2)^\dagger \end{aligned}$$

- $M \equiv [t] : \overleftarrow{\sigma} \Rightarrow \vec{\sigma} \cdot \tau$

$$\begin{aligned} & \exists x\vec{y}\vec{z}.\, (\langle x \rangle ; \langle \vec{y} \rangle ; [\vec{z}] ; [t] ; \langle x \rangle ; \langle \vec{y} \rangle ; [\vec{z}]) \\ =_\beta\ & \exists \vec{y}\vec{z}.\, (\langle t \rangle ; \langle \vec{y} \rangle ; [\vec{z}] ; \langle \vec{y} \rangle ; [\vec{z}]) \\ =_\nu\ & \langle t \rangle ; \exists \vec{y}\vec{z}.\, (\langle \vec{y} \rangle ; [\vec{z}] ; \langle \vec{y} \rangle ; [\vec{z}]) \\ =_\beta\ & \langle t \rangle ; \exists \vec{z}.\, (\langle \vec{z} \rangle ; [\vec{z}]) =_\eta \langle t \rangle = [t]^\dagger \end{aligned}$$

  where $x : \tau$, $t : \tau$ and $\vec{y}, \vec{z} : \vec{\sigma}$.

- $M \equiv \langle t \rangle a : \tau\overleftarrow{\sigma} \Rightarrow \vec{\sigma}$

$$\begin{aligned} & \exists x\vec{y}\vec{z}.\, (\langle \vec{y} \rangle ; [\vec{z}] ; [x] ; \langle t \rangle ; \langle \vec{y} \rangle ; [\vec{z}] ; [x]) \\ =_\beta\ & \exists \vec{y}\vec{z}.\, (\langle \vec{y} \rangle ; [\vec{z}] ; \langle \vec{y} \rangle ; [\vec{z}] ; [t]) \\ =_\nu\ & \exists \vec{y}\vec{z}.\, (\langle \vec{y} \rangle ; [\vec{z}] ; \langle \vec{y} \rangle ; [\vec{z}]) ; [t] \\ =_\beta\ & \exists \vec{z}.\, (\langle \vec{z} \rangle ; [\vec{z}]) ; [t] =_\eta [t] = \langle t \rangle^\dagger \end{aligned}$$

  where $x : \tau$, $t : \tau$ and $\vec{y}, \vec{z} : \vec{\sigma}$. □

Theorem E.5 (Theorem 6.5 restatement). *The subcategory of sum-free, star-free terms of RMC$[\Sigma_0]$ is equivalent to the free extra-hypergraph category.*

Proof. Let Hyp$[\Sigma_0]$ be the free extra-hypergraph category. We show the free functor from Hyp$[\Sigma_0] \to$ RMC$[\Sigma_0]$ induced by the soundness proof and the interpretation of Σ_0 as itself is full and faithful by a normal form argument. The functor is also surjective on objects, so this suffices to prove it is an equivalence. First, recall the result of [30] , which says that Hyp$[\Sigma_0]$ is equivalent to the category of co-relations between finite sets, where a co-relation $\alpha : [n] \to [m]$ is a partition of $[n] + [m]$, where $[n] = \{1, \ldots, n\}$. The normal forms of closed terms of the sum-free, star-free subcategory of RMC$[\Sigma_0]$ are given by

$$\exists \vec{z}.\, (\langle \vec{x} \rangle ; [\vec{y}]) : \tau^n \Rightarrow \tau^m$$

where $n = |\vec{x}|$ and $m = |\vec{y}|$, and $\vec{z} = \mathrm{fv}(\vec{x}) \cup \mathrm{fv}(\vec{y})$, and in particular we allow a variable $z \in \vec{z}$ to occur repeatedly in both $\vec{x}$ and $\vec{y}$. Note, we work with a typed version of normal forms, where normal forms are η-expanded. There is a bijection between normal forms and co-relations given by sending the normal form above to a co-relation $\alpha : [n] \to [m]$ where for each $z \in \vec{z}$ there is a partition of $[n] + [m]$ defined as the set $\{inl(i) \mid x_i = z\} \cup \{inr(j) \mid y_j = z\}$. □

F PROOFS FOR SECTION 7: RELATIONAL SEMANTICS

Lemma F.1 (Lemma 7.3 restatement). *For any $\Gamma, x : \rho \vdash M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$, we have*

$$[\![\Gamma, x : \rho \vdash M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_{v\{x \leftarrow t\}} = [\![\Gamma \vdash \{t/x\}M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}]\!]_v .$$

Proof. We proceed by induction on the type derivation of $\Gamma, x : \rho \vdash M : \overleftarrow{\sigma} \Rightarrow \vec{\tau}$.

- $M \equiv \star$

$$[\![\star]\!]_{v\{x \leftarrow t\}}(S) = [S] = [\![\{t/x\}\star]\!]_v(S)$$

- $M \equiv M_1 ; M_2$

$$\begin{aligned} [\![M_1 ; M_2]\!]_{v\{x \leftarrow t\}}(S) &= [U \mid T \in [\![M_1]\!]_{v\{x \leftarrow t\}},\ U \in [\![M_2]\!]_{v\{x \leftarrow t\}}] \\ (\text{I.H.}) &= [U \mid T \in [\![\{t/x\}M_1]\!]_v,\ U \in [\![\{t/x\}M_2]\!]_v] \\ &= [\![\{t/x\}(M_1 ; M_2)]\!]_v \end{aligned}$$

- $M \equiv 0$

$$[\![0]\!]_{v\{x \leftarrow t\}}(S) = [] = [\![\{t/x\}0]\!]_v(S)$$

- $M \equiv M_1 + M_2$

$$\begin{aligned} [\![M_1 + M_2]\!]_{v\{x \leftarrow t\}}(S) &= [\![M_1]\!]_{v\{x \leftarrow t\}}(S) \sqcup [\![M_2]\!]_{v\{x \leftarrow t\}}(S) \\ (\text{I.H.}) &= [\![\{t/x\}M_1]\!]_v(S) \sqcup [\![\{t/x\}M_2]\!](S) \\ &= [\![\{t/x\}(M_1 + M_2)]\!](S) \end{aligned}$$

- $M \equiv [s]$

$$\begin{aligned} [\![[s]]\!]_{v\{x \leftarrow t\}}(S) &= [Sv\{x \leftarrow t\}(s)] \\ &= [Sv(\{t/x\}s)] = [\![\{t/x\}[s]]\!]_v(S) \end{aligned}$$

- $M \equiv \langle s \rangle$

$$\begin{aligned} [\![\langle s \rangle]\!]_{v\{x \leftarrow t\}}(S\, r) &= \begin{cases} [S] & \text{if } r = v\{x \leftarrow t\}(s) \\ [] & \text{otherwise} \end{cases} \\ &= \begin{cases} [S] & \text{if } r = v(\{t/x\}s) \\ [] & \text{otherwise} \end{cases} \\ &= [\![\{t/x\}\langle s \rangle]\!]_v(S\, r) \end{aligned}$$

- $M \equiv \exists y.\, M'$

$$\begin{aligned} [\![\exists y.\, M']\!]_{v\{x \leftarrow t\}}(S) &= \bigsqcup_{t' \in T0} [\![M']\!]_{v\{x \leftarrow t\}\{y \leftarrow t'\}}(S) \\ (\text{I.H.}) &= \bigsqcup_{t' \in T0} [\![\{t/x\}M']\!]_{v\{y \leftarrow t'\}}(S) \\ &= [\![\exists y.\, \{t/x\}M']\!]_v(S) \\ &= [\![\{t/x\}\exists y.\, M']\!]_v(S) \end{aligned}$$

□

Proposition F.2 (Lemma 7.4 restatement). *The relational semantics $[\![-]\!] : RMC[\Sigma] \to Rel$ is a well-defined extra-hypergraph functor.*

Proof. We show the functor is well-defined: $M = N$ implies that $[\![M]\!]_v = [\![N]\!]_v$, for any valuation v. The cases of equations for Kleene algebra are analogous to those of Theorem 4.5, and we omit them for brevity. The remaining equations are as follows.

- $[\![\exists x.\, M]\!]_v = [\![M]\!]_v$ $(x \notin \mathrm{fv}(M))$

$$\begin{aligned} [\![\exists x.\, M]\!]_v(S) &= \bigsqcup_{t \in T0} [\![M]\!]_{v\{x \leftarrow t\}}(S) \\ (\text{Lemma 7.3}) &= \bigcup_{t \in T0} [\![M]\!]_v(S) \\ &= [\![M]\!]_v(S) \end{aligned}$$

- $[\![\exists x.\,(M\,;N)]\!]_v = [\![M\,;(\exists x.\,N)]\!]_v$ $(x \notin \mathrm{fv}(M))$

$$
\begin{aligned}
&[\![\exists x.\,(M\,;N)]\!]_v(S)\\
&= \bigsqcup_{t\in T0} [\![M\,;N]\!]_{v\{x\leftarrow t\}}(S)\\
&= \bigsqcup_{t\in T0} [U \mid T \in [\![M]\!]_{v\{x\leftarrow t\}}(S),\ U \in [\![N]\!]_{v\{x\leftarrow t\}}(T)]\\
\text{(Lemma 7.3)} &= \bigsqcup_{t\in T0} [U \mid T \in [\![M]\!]_{v}(S),\ U \in [\![N]\!]_{v\{x\leftarrow t\}}(T)]\\
&= [U \mid T \in [\![M]\!]_{v}(S),\ U \in \bigsqcup_{t\in T0} [\![N]\!]_{v\{x\leftarrow t\}}(T)]\\
&= [U \mid T \in [\![M]\!]_{v}(S),\ U \in [\![\exists x.\,N]\!]_{v}(T)]\\
&= [\![M\,;(\exists x.\,N)]\!]_v(S)
\end{aligned}
$$

- $[\![\exists x.\,(M\,;N)]\!]_v = [\![(\exists x.\,M)\,;N]\!]_v$ $(x \notin \mathrm{fv}(N))$

$$
\begin{aligned}
&[\![\exists x.\,(M\,;N)]\!]_v(S)\\
&= \bigsqcup_{t\in T0} [\![M\,;N]\!]_{v\{x\leftarrow t\}}(S)\\
&= \bigsqcup_{t\in T0} [U \mid T \in [\![M]\!]_{v\{x\leftarrow t\}}(S),\ U \in [\![N]\!]_{v\{x\leftarrow t\}}(T)]\\
\text{(Lemma 7.3)} &= \bigsqcup_{t\in T0} [U \mid T \in [\![M]\!]_{v\{x\leftarrow t\}}(S),\ U \in [\![N]\!]_{v}(T)]\\
&= [U \mid T \in \bigsqcup_{t\in T0} [\![M]\!]_{v\{x\leftarrow t\}}(S),\ U \in [\![N]\!]_{v}(T)]\\
&= [U \mid T \in [\![\exists x.\,M]\!]_{v}(S),\ U \in [\![N]\!]_{v}(T)]\\
&= [\![(\exists x.\,M)\,;N]\!]_v(S)
\end{aligned}
$$

- $[\![\exists x.\,(M+N)]\!]_v = [\![\exists x.\,M + \exists x.\,N]\!]_v$

$$
\begin{aligned}
&[\![\exists x.\,(M+N)]\!]_v(S)\\
&= \bigsqcup_{t\in T0} [\![M+N]\!]_{v\{x\leftarrow t\}}(S)\\
&= \bigsqcup_{t\in T0} ([\![M]\!]_{v\{x\leftarrow t\}}(S) \sqcup [\![N]\!]_{v\{x\leftarrow t\}}(S))\\
&= \bigsqcup_{t\in T0} [\![M]\!]_{v\{x\leftarrow t\}}(S) \sqcup \bigsqcup_{t\in T0} [\![N]\!]_{v\{x\leftarrow t\}}(S)\\
&= [\![\exists x.\,M]\!]_v(S) \sqcup [\![\exists x.\,N]\!]_v(S)\\
&= [\![\exists x.\,M + \exists x.\,N]\!]_v
\end{aligned}
$$

- $[\![\exists x.\,\exists y.\,M]\!]_v = [\![\exists y.\,\exists x.\,M]\!]_v$

$$
\begin{aligned}
&[\![\exists x.\,\exists y.\,M]\!]_v(S)\\
&= \bigsqcup_{t\in T0} [\![\exists y.\,M]\!]_{v\{x\leftarrow t\}}(S)\\
&= \bigsqcup_{t\in T0} \bigsqcup_{t'\in T0} [\![M]\!]_{v\{x\leftarrow t\}\{y\leftarrow t'\}}(S)\\
&= \bigsqcup_{t'\in T0} \bigsqcup_{t\in T0} [\![M]\!]_{v\{y\leftarrow t'\}\{x\leftarrow t\}}(S)\\
&= \bigsqcup_{t'\in T0} \bigsqcup_{t\in T0} [\![\exists x.\,M]\!]_{v\{y\leftarrow t'\}}(S)\\
&= [\![\exists y.\,\exists x.\,M]\!]_v(S)
\end{aligned}
$$

- $[\![\exists x.\,(\langle x\rangle\,;[x])]\!]_v = [\![\star]\!]_v$

$$
\begin{aligned}
&[\![\exists x.\,(\langle x\rangle\,;[x])]\!]_v(S\,s)\\
&= \bigsqcup_{t\in T0} [\![\langle x\rangle\,;[x]]\!]_{v\{x\leftarrow t\}}(S\,s)\\
&= [\![[x]]\!]_{v\{x\leftarrow s\}}(S) = Ss = [\![\star]\!]_v(Ss)
\end{aligned}
$$

- $[\![\exists x.\,(N\,;[t]\,;\langle x\rangle\,;M)]\!]_v = [\![\{t/x\}(N\,;M)]\!]_v$ $(x \notin t)$

$$
\begin{aligned}
&[\![\exists x.\,(N\,;[t]\,;\langle x\rangle\,;M)]\!]_v(S)\\
&= \bigsqcup_{t'\in T0} [\![N\,;[t]\,;\langle x\rangle\,;M]\!]_{v'}(S)\\
&= \bigsqcup_{t'\in T0} ([\![N]\!]_{v'};[\![[t]\,;\langle x\rangle]\!]_{v'};[\![M]\!]_{v'})(S)\\
(*) &= ([\![N]\!]_{v\{x\leftarrow t\}};[\![M]\!]_{v\{x\leftarrow t\}})(S)\\
&= ([\![N\,;M]\!]_{v\{x\leftarrow t\}})(S)\\
\text{(Lemma 7.3)} &= ([\![\{t/x\}(N\,;M)]\!]_v)(S)
\end{aligned}
$$

where $v' = v\{x \leftarrow t'\}$, we make use of the observation that $[\![M\,;N]\!]_v = [\![M]\!]_v;[\![N]\!]_v$, and $(*)$ guarantees that $t = t'$.

- $[\![\exists x.\,(N\,;[x]\,;\langle t\rangle\,;M)]\!]_v = [\![\{t/x\}(N\,;M)]\!]_v$ $(x \notin t)$

$$
\begin{aligned}
&[\![\exists x.\,(N\,;[x]\,;\langle t\rangle\,;M)]\!]_v(S)\\
&= \bigsqcup_{t'\in T0} [\![N\,;[x]\,;\langle t\rangle\,;M]\!]_{v'}(S)\\
&= \bigsqcup_{t'\in T0} ([\![N]\!]_{v'};[\![[x]\,;\langle t\rangle]\!]_{v'};[\![M]\!]_{v'})(S)\\
(*) &= ([\![N]\!]_{v\{x\leftarrow t\}};[\![M]\!]_{v\{x\leftarrow t\}})(S)\\
&= ([\![N\,;M]\!]_{v\{x\leftarrow t\}})(S)\\
\text{(Lemma 7.3)} &= ([\![\{t/x\}(N\,;M)]\!]_v)(S)
\end{aligned}
$$

where $v' = v\{x \leftarrow t'\}$, we make use of the observation that $[\![M\,;N]\!]_v = [\![M]\!]_v;[\![N]\!]_v$, and $(*)$ guarantees that $t = t'$.

- $[\![[x]\,;\langle x\rangle]\!]_v = [\![\star]\!]_v$

$$
\begin{aligned}
&[\![[x]\,;\langle x\rangle]\!]_{v\{x\leftarrow t\}}(S)\\
&= [\![\langle x\rangle]\!]_{v\{x\leftarrow t\}}(St)\\
&= S = [\![\star]\!]_{v\{x\leftarrow t\}}(S)
\end{aligned}
$$

- $[\![[f(R)]\,;\langle f(T)\rangle]\!]_v = [\![[R]\,;\langle T\rangle]\!]_v$

$$
\begin{aligned}
[\![[f(R)]\,;\langle f(T)\rangle]\!]_v(S) &= [\![\langle f(T)\rangle]\!]_v(S\,v(f(R)))\\
&= \begin{cases} [S] & \text{if } v(f(T)) = v(f(R))\\ [] & \text{otherwise}\end{cases}\\
&= \begin{cases} [S] & \text{if } v(T) = v(R)\\ [] & \text{otherwise}\end{cases}\\
(*) &= [\![\langle T\rangle]\!]_v(S\,v(R)) = [\![[R]\,;\langle T\rangle]\!]_v(S)
\end{aligned}
$$

where $(*)$ is given by repeated application of the definitions of push, pop, and sequencing.

- $[\![[f(R)] ; \langle g(T) \rangle]\!]_v = [\![0]\!]_v$

$$[\![[f(R)] ; \langle g(T) \rangle]\!]_v(S) = [\![\langle g(T) \rangle]\!]_v(S\, f(R)) = \begin{cases} [S] & \text{if } v(g(T)) = f(R) \\ [] & \text{otherwise} \end{cases} = [] = [\![0]\!]_v(S)$$

- $[\![[f(R)] ; \langle x \rangle]\!]_v = [\![0]\!]_v \ (x \in f(R))$

$$[\![[f(R)] ; \langle x \rangle]\!]_{v\{x \leftarrow t\}}(S) = [\![\langle x \rangle]\!]_{v\{x \leftarrow t\}}(S\, v(f(R))) = \begin{cases} [S] & \text{if } t = v\{x \leftarrow t\}(f(R)) \\ [] & \text{otherwise} \end{cases}$$

$$(t \in v\{x \leftarrow t\}(f(R))) = [] = [\![0]\!]_{v\{x \leftarrow t\}}(S)$$

- $[\![[x] ; \langle f(R) \rangle]\!]_v = [\![0]\!]_v \ (x \in f(R))$

$$[\![[x] ; \langle f(R) \rangle]\!]_{v\{x \leftarrow t\}}(S) = [\![\langle f(R) \rangle]\!]_{v\{x \leftarrow t\}}(S\, t) = \begin{cases} [S] & \text{if } t = v\{x \leftarrow t\}(f(R)) \\ [] & \text{otherwise} \end{cases}$$

$$(t \in v\{x \leftarrow t\}(f(R))) = [] = [\![0]\!]_{v\{x \leftarrow t\}}(S)$$

It remains only to check that $[\![-]\!]$ is a monoidal functor — indeed, we have that $[\![\star]\!] = \mathrm{id}$, $[\![M ; N]\!] = [\![M]\!]; [\![N]\!]$, and $[\![\vec{\sigma} \cdot \vec{\tau}]\!] = [\![\vec{\sigma}]\!] \otimes [\![\vec{\tau}]\!]$ — which preserves symmetry and extra-hypergraph structure. This is an easy exercise. □